Concrete Stream Calculus

An extended study

RALF HINZE
Computing Laboratory, University of Oxford,
Wolfson Building, Parks Road, Oxford, OX1 3QD, England
ralf.hinze@comlab.ox.ac.uk

Abstract

This paper shows how to reason about streams concisely and precisely. Streams, infinite sequences of elements, live in a coworld: they are given by a coinductive datatype, operations on streams are implemented by corecursive programs, and proofs are typically concocted using coinduction. This paper offers an alternative to coinduction. Suitably restricted, stream equations possess *unique solutions*. This property gives rise to a simple and attractive proof technique, essentially bringing equational reasoning to the coworld. We redevelop the theory of recurrences, finite calculus and generating functions using streams and stream operators, building on the cornerstone of unique solutions. The paper contains a smörgåsbord of examples: we study recursion elimination, investigate the binary carry sequence, explore Sprague-Grundy numbers and present two proofs of Moessner's Theorem. The calculations benefit from the rich structure of streams. As the type of streams is an applicative functor we can effortlessly lift operations and their properties to streams. In combination with Haskell's facilities for overloading, this greatly contributes to conciseness of notation. The development is indeed constructive: streams and stream operators are implemented in Haskell, usually by one-liners. The resulting calculus or library, if you wish, is elegant and fun to use.

1 Introduction

The cover of my favourite maths book displays a large Σ imprinted in concrete (Graham *et al.*, 1994). There is a certain beauty to it, but sure enough, when the letter first appears in the text, it is decorated with formulas. Sigma denotes summation and, traditionally, summation is a binder introducing an index variable that ranges over some set. More often than not, the index variable then appears as a subscript referring to an element of some other set or sequence. If you turn the pages of this paper, you will not find many index variables or subscripts though we deal with recurrences, summations and power series. Index variables have their rôle, but often they can be avoided by taking a holistic view, treating the set or the sequence they refer to as a single entity. Manipulating a single entity is almost always simpler and more elegant than manipulating a cohort of singletons.

Adopting this holistic view, the present paper shows how to reason effectively about streams, combining concision with precision. Streams, infinite sequences of elements, are a simple example of codata: they are given by a coinductive datatype, operations on streams are implemented by corecursive programs. In a lazy functional language, such as Haskell (Peyton Jones, 2003), streams are easy to define and many textbooks on Haskell

reproduce the folklore examples of Fibonacci or Hamming numbers defined by recursion equations over streams. One has to be a bit careful in formulating a recursion equation, basically ensuring that the sequence defined does not swallow its own tail. However, if this care is exercised, the equation possesses a *unique solution*, a fact that is not widely appreciated. Uniqueness can be exploited to prove that two streams are equal: if they satisfy the same recursion equation, then they are! This technique makes an interesting alternative to coinduction, the prevalent proof technique for codata. Building on the cornerstone of unique solutions, we redevelop the theory of recurrences, finite calculus and generating functions. We hope that this will contribute to popularising this attractive proof technique.

The paper contains a smörgåsbord of examples: we study recursion elimination, investigate the binary carry sequence, explore Sprague-Grundy numbers and present two proofs of Moessner's Theorem. Rather attractively, the calculations have a strong equational flavour. Furthermore, they benefit greatly from the rich structure of streams: the type of streams is an applicative functor or idiom; streams can be seen as memo-tables or tabulations. The idiomatic structure allows us to effortlessly *lift operations* and their properties to streams, a device we use extensively in combination with Haskell's facilities for overloading. The development is indeed constructive: streams and stream operators are implemented in Haskell, usually by one-liners. Technically, we view Haskell as a meta-language for Set, the category of sets and total functions, avoiding the need to consider $\bot$.

The paper draws heavily from previous work. The two major sources of inspiration were Rutten's work on stream calculus (2003; 2005) and the aforementioned textbook on concrete mathematics (Graham *et al.*, 1994) — hence the title of the paper. Rutten shows that stream equations, which he calls behavioural differential equations, have indeed unique solutions. A detailed discussion of related work can be found in Section 6.

The rest of the paper is structured as follows. Section 2 introduces the basic stream operations and their properties, organised around the different structural views of streams: idioms, tabulations and final coalgebras. Furthermore, it introduces the fundamental proof technique, which exploits uniqueness of solutions. Section 3 shows how to capture recurrences as streams and solves a multitude of recreational puzzles. We study recursion elimination, investigate the binary carry sequence and explore Sprague-Grundy numbers. Section 4 redevelops the theory of finite calculus using streams and stream operators. As a major application, we present a proof of Moessner's Theorem. Section 5 introduces generating functions and explains how to solve recurrences. Moessner's Theorem is reformulated in terms of generating functions, and an application to combinatorial counting is presented. Finally, Section 6 reviews related work and Section 7 concludes.

2 Streams

A stream is an infinite sequence of elements. The type of streams, called $Stream\ \alpha$, is like Haskell's list datatype $[\alpha]$, except that there is no base constructor so we cannot construct a finite stream. The *Stream* type is not an inductive type, but a *coinductive type*.

type name — creates a new datatype.

$$\begin{aligned}&\mathbf{data}\ Stream\ \alpha = Cons\ \{head :: \alpha, tail :: Stream\ \alpha\}\\&(\prec)\ \ :: \alpha \to Stream\ \alpha \to Stream\ \alpha\\&a \prec s = Cons\ a\ s\end{aligned}$$

stream type — var — type — type

Table 1. *Precedences and fixities of operators (including prelude operators).*

precedence	left associative	non-associative	right-associative
9	$\diamond$		$\cdot$ $\circ$
7	$*$ / **div** **mod** $\setminus$ $\times$ $\div$ $\otimes$		
6	$+$ $-$ $\cup$		
5			: $\mathbin{+\!\!+}$ $\prec$ $\curlyvee$ $\preccurlyeq$
4		$\in$ $==$ $\neq$ $<$ $\leqslant$ $>$ $\geqslant$ $\doteq$ etc	
0			$\star$

Streams are constructed using $\prec$, which prepends an element to a stream. They are destructed using *head*, which yields the first element, and *tail*, which returns the stream without the first element. We let s, t and u range over streams.

Though the type of streams enjoys a deceptively simple definition, it is an infinite product, it exhibits a rich structure: *Stream* is an idiom, a tabulation, a monad, a final coalgebra, and a comonad. The following section, which introduces the most important generic operations on streams, is organised around these different views. Table 1 summarises the operators, listing their precedences and fixities.

2.1 Operations

Idiom Most definitions we encounter later on make use of operations lifted to streams. We obtain these liftings almost for free, as *Stream* is a so-called *applicative functor* or *idiom* (McBride & Paterson, 2008).

$$\begin{aligned}&\textbf{class } \mathit{Idiom}\ \iota \textbf{ where}\\ &\quad \mathit{pure} :: \alpha \to \iota\ \alpha\\ &\quad (\diamond)\ \ :: \iota\ (\alpha \to \beta) \to (\iota\ \alpha \to \iota\ \beta)\end{aligned}$$

The constructor class introduces an operation for embedding a value into an idiomatic structure, and an application operator that takes a structure of functions to a function that maps a structure of arguments to a structure of results. The identity type constructor, $\mathit{Id}\ \alpha = \alpha$, is an idiom. Idioms are closed both under type composition, $(\iota \circ \kappa)\ \alpha = \iota\ (\kappa\ \alpha)$, and type pairing, $(\iota \mathbin{\dot\times} \kappa)\ \alpha = (\iota\ \alpha, \kappa\ \alpha)$. Here are the definitions that turn *Stream* into an idiom.

$$\begin{aligned}&\textbf{instance } \mathit{Idiom\ Stream} \textbf{ where}\\ &\quad \mathit{pure}\ a = s \textbf{ where } s = a \prec s\\ &\quad s \diamond t \quad\ = (\mathit{head}\ s)\ (\mathit{head}\ t) \prec (\mathit{tail}\ s) \diamond (\mathit{tail}\ t)\end{aligned}$$

The function *pure* constructs a *constant stream*; *pure* 0, for example, yields the infinite sequence of zeros (*A*000004[1]). The method is given by a first-order equation: $s = a \prec s$. Alternatively, it can be defined by a second-order equation: $\mathit{pure}\ a = a \prec \mathit{pure}\ a$, which

[1] Most if not all integer sequences defined in this paper are recorded in Sloane's On-Line Encyclopedia of Integer Sequences (Sloane, 2009). Keys of the form *Annnnnn* refer to entries in that database. Somewhat surprisingly, *pure* 0 is not *A*000000. Just in case you were wondering, the first sequence (*A*000001) lists the number of groups of order n.

amounts to the λ-lifted version of the actual definition (Danvy, 1999). The first-order equation is preferable, because it constructs a cyclic structure, using only a constant amount of space (Bird, 1998). We let c range over constant streams. Idiomatic application lifts function application pointwise to streams. We refer to *pure* as a *parametrised stream* and to $\diamond$ as a *stream operator*.

Using nested idiomatic applications, we can lift an arbitrary function pointwise to an idiomatic structure. Here are generic combinators for lifting $n$ary operations ($n = 0, 1, 2$).

$$\begin{aligned}
&\mathit{repeat} && :: (\mathit{Idiom}\ \iota) \Rightarrow \alpha \to \iota\ \alpha \\
&\mathit{repeat}\ a && = \mathit{pure}\ a \\
&\mathit{map} && :: (\mathit{Idiom}\ \iota) \Rightarrow (\alpha \to \beta) \to (\iota\ \alpha \to \iota\ \beta) \\
&\mathit{map}\ f\ s && = \mathit{pure}\ f \diamond s \\
&\mathit{zip} && :: (\mathit{Idiom}\ \iota) \Rightarrow (\alpha \to \beta \to \gamma) \to (\iota\ \alpha \to \iota\ \beta \to \iota\ \gamma) \\
&\mathit{zip}\ g\ s\ t && = \mathit{pure}\ g \diamond s \diamond t
\end{aligned}$$

The context $(\mathit{Idiom}\ \iota) \Rightarrow$ constrains the type variable ι to instances of the class *Idiom*. Using *zip* we can, for instance, lift pairing to idioms.

$$\begin{aligned}
&(\star) :: (\mathit{Idiom}\ \iota) \Rightarrow \iota\ \alpha \to \iota\ \beta \to \iota\ (\alpha, \beta) \\
&(\star) = \mathit{zip}\ (,)
\end{aligned}$$

The quizzical '$(,)$' is Haskell's pairing constructor. We need to introduce a new operator for lifted pairing as we cannot overload '$(,)$'. Overloading pairing would not be a terribly good idea anyway, since it is a polymorphic operation. If '$(,)$' has both the type $\alpha \to \beta \to (\alpha, \beta)$ and $\mathit{Stream}\ \alpha \to \mathit{Stream}\ \beta \to \mathit{Stream}\ (\alpha, \beta)$, then expressions such as (s,t) are inherently ambiguous: is (s,t) a pair of streams or a stream of pairs?

For convenience and conciseness of notation, we overload the arithmetic operations to also denote the corresponding operations lifted to streams. In Haskell, this is easily accomplished using the numeric type classes. Here is an excerpt of the code.[2]

$$\begin{aligned}
&\textbf{instance}\ (\mathit{Num}\ \alpha) \Rightarrow \mathit{Num}\ (\mathit{Stream}\ \alpha)\ \textbf{where} \\
&\quad (+) && = \mathit{zip}\ (+) \\
&\quad (-) && = \mathit{zip}\ (-) \\
&\quad (*) && = \mathit{zip}\ (*) \\
&\quad \mathit{negate} && = \mathit{map}\ \mathit{negate} \quad \text{-- unary minus} \\
&\quad \mathit{fromInteger}\ i && = \mathit{repeat}\ (\mathit{fromInteger}\ i)
\end{aligned}$$

This instance declaration allows us, in particular, to use integer constants as streams — in Haskell, the literal 3 abbreviates $\mathit{fromInteger}\ (3 :: \mathit{Integer})$. This convention effectively overloads integer literals. Depending on the context, i, j, k, m and n either denote an integer or a constant stream of integers.

We shall make intensive use of overloading, going beyond Haskell's predefined numeric classes. For instance, we also lift exponentiation and binomial coefficients to streams: s^t

[2] Since *repeat*, *map* and *zip* work for an arbitrary idiom, we should actually be able to define a generic instance of the form $(\mathit{Idiom}\ \iota, \mathit{Num}\ \alpha) \Rightarrow \mathit{Num}\ (\iota\ \alpha)$. Unfortunately, this does not quite work with the Standard Haskell libraries, as *Num* has two super-classes, *Eq* and *Show*, which cannot sensibly be defined generically.

and $\binom{s}{t}$. To reduce clutter, we omit the necessary class and instance declarations — they are entirely straightforward — and just note the use of overloading as we go along.

Of course, the elements of a stream are by no means confined to numbers. Streams of Booleans are equally useful. For this element type, it is convenient to lift and overload the Boolean constants, *false* and *true*, and the Boolean operators, $\neg$, $\wedge$ and $\vee$, to streams. Again, the definitions are straightforward and omitted. One might be tempted to also overload $==$ to denote lifted equality of streams. However, the predefined *Eq* class dictates that $==$ returns a Boolean. Consequently, lifted equality, which returns a stream of Booleans, cannot be made an instance of that class. Instead, we introduce a new operator.

$$\begin{aligned}&(\doteq) :: (\mathit{Idiom}\ \iota, \mathit{Eq}\ \alpha) \Rightarrow \iota\ \alpha \to \iota\ \alpha \to \iota\ \mathit{Bool}\\&(\doteq) = \mathit{zip}\ (==)\end{aligned}$$

Likewise, we also define dotted variants of $\neq$, $<$, $\leqslant$, $>$ and $\geqslant$.

Using this vocabulary we can already define the usual suspects: the natural numbers (*A*001477), the factorial numbers (*A*000142), and the Fibonacci numbers (*A*000045).

$$\begin{aligned}\mathit{nat} &= 0 \prec \mathit{nat} + 1\\ \mathit{fac} &= 1 \prec (\mathit{nat} + 1) * \mathit{fac}\\ \mathit{fib} &= 0 \prec \mathit{fib}'\\ \mathit{fib}' &= 1 \prec \mathit{fib} + \mathit{fib}'\end{aligned}$$

Note that $\prec$ binds less tightly than $+$, see Table 1. For instance, $0 \prec \mathit{nat} + 1$ is grouped $0 \prec (\mathit{nat} + 1)$. The three sequences are given by recursion equations adhering to a strict scheme: each equation defines the head and the tail of the sequence, the latter possibly in terms of the entire sequence. As an aside, we will use the convention that the identifier x' denotes the tail of x, and x'' the tail of x'. The Fibonacci numbers provide an example of mutual recursion: fib' refers to *fib* and vice versa. Actually, in this case mutual recursion is not necessary, as a quick calculation shows: $\mathit{fib}' = 1 \prec \mathit{fib} + \mathit{fib}' = (1 \prec \mathit{fib}) + (0 \prec \mathit{fib}') = (1 \prec \mathit{fib}) + \mathit{fib}$. So, an alternative definition is

$$\mathit{fib} = 0 \prec (1 \prec \mathit{fib}) + \mathit{fib}\ .$$

It is fun to play with the sequences. Here is a short interactive session.

```
≫ fib
⟨0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,...⟩
≫ nat * nat
⟨0,1,4,9,16,25,36,49,64,81,100,121,144,169,196,225,...⟩
≫ fib'^2 - fib * fib''
⟨1,-1,1,-1,1,-1,1,-1,1,-1,1,-1,1,-1,1,-1,...⟩
≫ fib'^2 - fib * fib'' ≐ (-1)^nat
⟨True,True,True,True,True,True,True,True,True,True,True,True,True,...⟩
```

The part after the prompt, ≫ , is the user's input. The result of each submission is shown in the subsequent line. This document has been produced using lhs2TEX(Hinze & Löh, 2008). The session displays the actual output of the Haskell interpreter, generated automatically with lhs2TEX's active features.

Interleaving Another important operator is *interleaving* of two streams.

$$\begin{aligned}&(\curlyvee) \;::\; \mathit{Stream}\;\alpha \to \mathit{Stream}\;\alpha \to \mathit{Stream}\;\alpha\\&s \curlyvee t = \mathit{head}\;s \prec t \curlyvee \mathit{tail}\;s\end{aligned}$$

Though the symbol is symmetric, $\curlyvee$ is not commutative. Neither is it associative. Let us look at an example application. The above definition of the naturals is based on the unary number system. Using interleaving, we can alternatively base the sequence on the binary number system.

$$\mathit{bin} = 0 \prec 2 * \mathit{bin} + 1 \curlyvee 2 * \mathit{bin} + 2$$

Since $\curlyvee$ has lower precedence than the arithmetic operators, see Table 1, the right-hand side of the equation above is grouped $0 \prec ((2 * \mathit{bin} + 1) \curlyvee (2 * \mathit{bin} + 2))$.

Now that we have two definitions of the natural numbers, the question naturally arises as to whether they are actually equal. Reassuringly, the answer is "Yes!" Proving the equality of streams or of stream operators is one of our main activities in this paper. However, we postpone a proof of $\mathit{nat} = \mathit{bin}$, until we have the prerequisites at hand.

Many numeric sequences are actually interleavings in disguise: for instance, $(-1)^{\mathit{nat}} = 1 \curlyvee -1$, $\mathit{nat}\;\mathbf{div}\;2 = \mathit{nat} \curlyvee \mathit{nat}$, and $\mathit{nat}\;\mathbf{mod}\;2 = 0 \curlyvee 1$.

Just as binary numbers can be generalised to *n*ary numbers, $\curlyvee$ can be generalised to interleave *n* argument streams, represented by a list of streams.

$$\begin{aligned}&\mathit{interleave} \;::\; [\mathit{Stream}\;\alpha] \to \mathit{Stream}\;\alpha\\&\mathit{interleave}\;(s : x) = \mathit{head}\;s \prec \mathit{interleave}\;(x +\!\!+ [\mathit{tail}\;s])\end{aligned}$$

Tabulation The Fibonacci function is the folklore example of a function whose straightforward definition leads to a very inefficient program.

$$\begin{aligned}\mathscr{F}_0 &= 0\\ \mathscr{F}_1 &= 1\\ \mathscr{F}_{n+2} &= \mathscr{F}_n + \mathscr{F}_{n+1}\end{aligned}$$

To compute $\mathscr{F}_n$ a total of $\mathscr{F}_{n+1} - 1$ additions are necessary. In other words, the running time of $\mathscr{F}$ is proportional to the result. By contrast, the stream definition, *fib*, does not suffer from this problem: to determine the *n*th element only $n \mathbin{\dot{-}} 1 = \mathit{max}\,\{n-1, 0\}$ additions are required. The following display visualises the workings of *fib*.

$$\begin{array}{ccccccccccccc}
0 & 1 & 1 & 2 & 3 & 5 & 8 & 13 & 21 & 34 & 55 & \cdots & \quad \mathit{fib}\\
\| & \| & \| & \| & \| & \| & \| & \| & \| & \| & \| & \cdots & \quad \|\\
0 & 1 & 0 & 1 & 1 & 2 & 3 & 5 & 8 & 13 & 21 & \cdots & \quad 0 \prec (1 \prec \mathit{fib})\\
 & + & + & + & + & + & + & + & + & + & + & \cdots & \quad\quad\quad\quad\quad\quad + \\
 & 0 & 1 & 1 & 2 & 3 & 5 & 8 & 13 & 21 & 34 & \cdots & \quad\quad\quad\quad\quad\quad \mathit{fib}
\end{array}$$

The speed-up is, of course, not a coincidence: *fib* is the memoised version of $\mathscr{F}$. In fact, streams are in one-to-one correspondence to functions from the natural numbers:

$$\mathit{Stream}\;\alpha \quad\cong\quad \mathit{Nat} \to \alpha \;\;.$$

A stream can be seen as the tabulation of a function from the natural numbers. Conversely, a function of type $\mathit{Nat} \to \alpha$ can be implemented by looking up a memo-table. Here are the

functions that witness the isomorphism.

$$\begin{aligned}
&\textbf{data}\ \mathit{Nat} = 0 \mid \mathit{Nat} + 1\\
&\mathit{tabulate} \quad :: (\mathit{Nat} \to \alpha) \to \mathit{Stream}\ \alpha\\
&\mathit{tabulate}\ f = f\ 0 \prec \mathit{tabulate}\ (f \cdot (+1))\\
&\mathit{lookup} \qquad\quad :: \mathit{Stream}\ \alpha \to (\mathit{Nat} \to \alpha)\\
&\mathit{lookup}\ s\ 0 \qquad = \mathit{head}\ s\\
&\mathit{lookup}\ s\ (n+1) = \mathit{lookup}\ (\mathit{tail}\ s)\ n
\end{aligned}$$

We have $\mathscr{F} = \mathit{lookup}\ \mathit{fib}$ and $\mathit{tabulate}\ \mathscr{F} = \mathit{fib}$. Section 3.1 shows how to systematically turn functions given by recurrence relations into streams defined by recursion equations.

For conciseness of notation, we sometimes abbreviate $\mathit{lookup}\ s\ n$ by $(s)_n$, so that, in particular, $\mathit{head}\ s = (s)_0$.

Many idioms are, in fact, tabulations: the diagonal functor $\Delta = \mathit{Id} \mathbin{\dot\times} \mathit{Id}$, for instance, tabulates functions from the Booleans as $\Delta\ \alpha \cong \mathit{Bool} \to \alpha$. In general, the type constructor ι is said to tabulate $\tau \to$ iff $\iota\ \alpha \cong \tau \to \alpha$.

The type constructor $\tau \to$ itself[3], the so-called *environment monad*, also gives rise to an idiom.

$$\begin{aligned}
&\textbf{instance}\ \mathit{Idiom}\ (\tau \to)\ \textbf{where}\\
&\quad \mathit{pure}\ a = \lambda x \to a\\
&\quad f \diamond g \quad = \lambda x \to (f\ x)\ (g\ x)
\end{aligned}$$

Here, *pure* is the *K* combinator and $\diamond$ is the *S* combinator from combinatory logic. Like for streams, lifting is defined pointwise: $\mathit{zip}\ (+)\ f\ g = \mathit{pure}\ (+) \diamond f \diamond g = \lambda x \to f\ x + g\ x$.

Monad Since $\tau \to$ is a monad, *Stream* can also be turned into a monad: *return* is *pure* and *join* is given by

$$\begin{aligned}
&\mathit{join} \quad :: \mathit{Stream}\ (\mathit{Stream}\ \alpha) \to \mathit{Stream}\ \alpha\\
&\mathit{join}\ s = \mathit{head}\ (\mathit{head}\ s) \prec \mathit{join}\ (\mathit{tail}\ (\mathit{tail}\ s)) \quad .
\end{aligned}$$

We will, however, not make use of the monadic structure.

Final coalgebra The stream *nat* is constructed by repeatedly mapping a function over a stream. We can capture this recursion scheme using a combinator.

$$\begin{aligned}
&\mathit{recurse} \qquad :: (\alpha \to \alpha) \to (\alpha \to \mathit{Stream}\ \alpha)\\
&\mathit{recurse}\ f\ a = s\ \textbf{where}\ s = a \prec \mathit{map}\ f\ s
\end{aligned}$$

So, $\mathit{nat} = \mathit{recurse}\ (+1)\ s$.

Alternatively, we can build a stream by repeatedly applying a given function to a given initial seed.

$$\begin{aligned}
&\mathit{iterate} \qquad :: (\alpha \to \alpha) \to (\alpha \to \mathit{Stream}\ \alpha)\\
&\mathit{iterate}\ f\ a = a \prec \mathit{iterate}\ f\ (f\ a)
\end{aligned}$$

So, $\mathit{iterate}\ (+1)\ 0$ is yet another definition of the naturals. In fact, $\mathit{recurse}\ f\ a$ and $\mathit{iterate}\ f\ a$ always yield the same stream, see Section 2.5.

[3] In Haskell, the type constructor $\to$ is curried, it has kind $* \to * \to *$. The operator section $\tau \to$ is shorthand for $(\to)\ \tau$. Unfortunately, the section is not legal Haskell syntax.

The iterator is a special case of the *unfold* or anamorphism for the *Stream* datatype, which captures another common corecursion pattern.

$$\begin{aligned} &\mathit{unfold} && :: (\sigma \to \alpha) \to (\sigma \to \sigma) \to (\sigma \to \mathit{Stream}\ \alpha) \\ &\mathit{unfold}\ g\ f\ a = g\ a \prec \mathit{unfold}\ g\ f\ (f\ a) \end{aligned}$$

The type σ can be seen as a type of states. The anamorphism implements an automaton over this state space: the third argument is the initial state; the second argument is the transition function that maps the current state to the next state; and the first argument is the output function that maps the current state to an 'output symbol'.

The functions *iterate* and *unfold* are inter-definable: $\mathit{iterate}\ f = \mathit{unfold}\ \mathit{id}\ f$ and conversely $\mathit{unfold}\ h\ t = \mathit{map}\ h \cdot \mathit{iterate}\ t$. We prefer *iterate* over *unfold*, since the extra generality is barely needed. Section 3.1 explains why.

Comonad Finally, *Stream* has the structure of a comonad, the categorical dual of a *monad*: *head* is the counit and *tails*, defined below, is the comultiplication.

$$\begin{aligned} &\mathit{tails} :: \mathit{Stream}\ \alpha \to \mathit{Stream}\ (\mathit{Stream}\ \alpha) \\ &\mathit{tails} = \mathit{iterate}\ \mathit{tail} \end{aligned}$$

We shall neither make use of the comonadic structure.

2.2 Stream comprehensions

Throughout the paper we emphasise the holistic view on streams. However, from time to time it is useful to define a stream pointwise or to relate a given stream to a pointwise definition. Parallel stream comprehensions allow us to do exactly this.

$$\langle e \mid x_1 \leftarrow e_1 \mid \ldots \mid x_n \leftarrow e_n \rangle = \mathit{pure}\ (\lambda x_1 \ldots x_n \to e) \diamond e_1 \diamond \cdots \diamond e_n$$

In fact, since the notation is defined in terms of the idiomatic primitives, it makes sense for an arbitrary idiom. The notational advantage of a comprehension over the idiomatic expression on the right-hand side is that the bound variable x_i and the stream it iterates over e_i are adjacent. For streams, the order of the binders is not significant: any permutation of $x_1 \leftarrow e_1, \ldots, x_n \leftarrow e_n$ generates the same stream. This does not hold for arbitrary idioms: in the *IO* idiom, for instance, the order is significant.

The functional programming language Miranda[4] supports a second form of generators, so-called iterative generators, which capture recurrence relations.

$$\langle e_1 \mid x \leftarrow e_2, e_3 \,..\rangle = \mathit{pure}\ (\lambda x \to e_1) \diamond \mathit{iterate}\ (\lambda x \to e_3)\ e_2$$

The iterative generator avoids the duplication of the binder '$\lambda x \to$' on the right-hand side. In parallel stream comprehensions, iterative generators can be freely mixed with ordinary generators. As an example, here is an alternative definition of *fib*

$$\mathit{fib} = \langle a \mid (a,b) \leftarrow (0,1), (b, a+b) \,..\rangle \ ,$$

which uses tupling to achieve linear running time. We will derive this version in Section 3.2.1.

[4] Miranda is a registered trademark of Research Software Limited.

2.3 Definitions

Not every legal Haskell definition of type *Stream* τ actually defines a stream. Two simple counterexamples are $s = \mathit{tail}\ s$ and $s = \mathit{head}\ s \prec \mathit{tail}\ s$. Both of them loop in Haskell; when viewed as stream equations they are ambiguous.[5] In fact, they admit infinitely many solutions: every constant stream is a solution of the first equation, every stream is a solution of the second one. This situation is undesirable from both a practical and a theoretical standpoint. Fortunately, it is not hard to restrict the *syntactic* form of equations so that they possess *unique solutions* (Rutten, 2003, Theorem 3.1). We insist that equations adhere to the following form:

$$x = h \prec t \ \ ,$$

where x is an identifier of type *Stream* τ, h is a constant expression of type τ, and t is an expression of type *Stream* τ possibly referring to x or other stream identifiers. However, neither h nor t may contain *head* or *tail*.

If x is a parametrised stream or a stream operator,

$$x\, x_1\ \ldots\ x_n = h \prec t$$

then h and t may use $\mathit{head}\ x_i$ or $\mathit{tail}\ x_i$ provided x_i is of the right type. Apart from that, no other uses of *head* or *tail* are permitted. Equations of this form are called *admissible*.

Looking back, we find that the definitions we have encountered so far, including those of *pure*, $\diamond$ and Υ, are admissible. In general, we consider systems of admissible equations to cater for mutually recursive definitions. In fact, most of the definitions cannot be read in isolation. For instance, *nat*'s defining equation, $\mathit{nat} = 0 \prec \mathit{nat} + 1$, silently refers to *pure* and $\diamond$, so *nat* is actually given by a system of three equations

$$\begin{aligned}
\mathit{nat} \quad &= 0 \prec \mathit{pure}\ (+) \diamond \mathit{nat} \diamond \mathit{pure}\ 1 \\
\mathit{pure}\ a &= a \prec \mathit{pure}\ a \\
s \diamond t \quad &= (\mathit{head}\ s)\ (\mathit{head}\ t) \prec (\mathit{tail}\ s) \diamond (\mathit{tail}\ t) \ \ ,
\end{aligned}$$

all of which are admissible.

If the requirements are violated, then a simple rewrite sometimes saves the day. As an example, the folklore definition of *fib*

$$\mathit{fib} = 0 \prec 1 \prec \mathit{fib} + \mathit{tail}\ \mathit{fib}$$

is not admissible, since *tail* is applied to the stream defined. We can, however, eliminate the call to *tail* by introducing a name for it. If we replace $\mathit{tail}\ \mathit{fib}$ by fib', we obtain the two equations given in Section 2.1. The alternative definition $\mathit{fib} = 0 \prec (1 \prec \mathit{fib}) + \mathit{fib}$ is also admissible. Nested occurrences of $\prec$ are unproblematic since they can be seen as shorthand for mutually recursive equations: $\mathit{fib} = 0 \prec \mathit{cons}\ 1\ \mathit{fib} + \mathit{fib}$ and $\mathit{cons}\ h\ t = h \prec t$.

[5] There is a slight mismatch between the theoretical framework of streams and the Haskell implementation of streams. Since products are lifted in Haskell, *Stream* τ additionally contains partial streams such as $\bot$, $a_0 \prec \bot$, $a_0 \prec a_1 \prec \bot$ and so forth. We ignore this extra complication, viewing Haskell as a meta-language for Set.

Sometimes the situation is more subtle. The two stream operators

$$\begin{array}{lll} even, odd :: Stream\ \alpha \to Stream\ \alpha \\ even\ s & = head\ s \prec odd\ \ (tail\ s) \\ odd\ s & = \qquad\qquad even\ (tail\ s) \end{array}$$

are well-defined, even though *odd* does not meet the syntactic requirements. If $s = x \curlyvee y$, then $even\ s = x$ and $odd\ s = y$. The operators are ruled out, however, since their *use* in other recursive definitions is possibly problematic: the equation $x = 0 \prec even\ x$, for instance, is admissible, yet it has infinitely many solutions.

Not all is lost, however. Consider as another example the equation

$$s = a \prec f\ (even\ s) \curlyvee g\ (odd\ s)$$

or, equivalently,

$$x \curlyvee y = a \prec f\ x \curlyvee g\ y \ , \tag{1}$$

where f and g are some stream operators. In this case, the equation uniquely determines s, and hence x and y, since it is equivalent to the recursive equation $x = a \prec g\ (f\ x)$ and the non-recursive $y = f\ x$. So, we may view Equation (1) as a notational shorthand for the two latter equations. The scheme nicely captures alternations, for instance,

$$d \curlyvee i = 1 \prec d * 2 \curlyvee i + 1$$

alternates between doubling and incrementing (*A*075427).

By the way, non-recursive definitions like

$$nat' = nat + 1$$

are unproblematic and unrestricted as they can always be inlined.

2.4 Laws

This section provides an account of the most important properties of streams, serving mainly as a handy reference. If you wish, you can skim through the material or proceed directly to Section 2.5.

We have noted before that the type *Stream* has a rich structure: it is an idiom, a tabulation and a final coalgebra. The laws are categorised according to these different views.

First of all, streams enjoy the following *extensionality* property.

$$s \quad = \quad head\ s \prec tail\ s \tag{2}$$

Idiom Streams satisfy the four *idiom laws*.

$$\begin{array}{lcll} pure\ id \diamond u & = & u & \text{(identity)} \\ pure\ (\cdot) \diamond u \diamond v \diamond w & = & u \diamond (v \diamond w) & \text{(composition)} \\ pure\ f \diamond pure\ x & = & pure\ (f\ x) & \text{(homomorphism)} \\ u \diamond pure\ x & = & pure\ (\lambda f \to f\ x) \diamond u & \text{(interchange)} \end{array}$$

The first two imply the well-known *functor laws*: *map* preserves identity and composition (hence the names of the idiom laws).

$$\begin{array}{lcl} \mathit{map}\ \mathit{id} & = & \mathit{id} \\ \mathit{map}\ (f \cdot g) & = & \mathit{map}\ f \cdot \mathit{map}\ g \end{array}$$

Index manipulations are often applications of the second functor law, which becomes evident, if we rephrase the laws using stream comprehensions.

$$\langle a \mid a \leftarrow s \rangle = s \tag{3}$$

$$\langle f\ (g\ a) \mid a \leftarrow s \rangle = \langle f\ b \mid b \leftarrow \langle g\ a \mid a \leftarrow s \rangle \rangle \tag{4}$$

To illustrate, here is a short calculation that demonstrates a simple index manipulation.

$$\begin{array}{ll} & \langle 2 * i \mid i \leftarrow \mathit{nat} + 1 \rangle \\ = & \quad \{\ \mathit{nat} + 1 = \langle j + 1 \mid j \leftarrow \mathit{nat} \rangle\ \} \\ & \langle 2 * i \mid i \leftarrow \langle j + 1 \mid j \leftarrow \mathit{nat} \rangle \rangle \\ = & \quad \{\ \text{functor law (4)}\ \} \\ & \langle 2 * (j + 1) \mid j \leftarrow \mathit{nat} \rangle \end{array}$$

Every structure comes equipped with structure-preserving maps; so do idioms: a natural transformation $h :: \iota\ \alpha \rightarrow \kappa\ \alpha$ is an *idiom homomorphism* iff

$$h\ (\mathit{pure}\ a) = \mathit{pure}\ a\ ; \tag{5}$$

$$h\ (x \diamond y) = h\ x \diamond h\ y\ . \tag{6}$$

The two conditions imply, in fact, the naturality property: $\mathit{map}\ f \cdot h = h \cdot \mathit{map}\ f$. The identity function is an idiom homomorphism, and homomorphisms are closed under composition. The projection function *head* is a homomorphism (from *Stream* to the identity idiom *Id*), so is *tail*. The latter projection is an *idiom endomorphism*, a homomorphism from an idiom to itself. Also *pure* itself is a homomorphism (from *Id* to the idiom). Condition (6) for *pure* is equivalent to the third idiom law (hence its name).

This leaves us to explain the fourth idiom law: it captures the fact that a pure, effect-free computation commutes with an impure, effectful computation.

Lifted pairing $\star$ satisfies

$$\mathit{map}\ \mathit{fst}\ (s \star t) = s\ ; \tag{7}$$

$$\mathit{map}\ \mathit{snd}\ (s \star t) = t\ ; \tag{8}$$

$$\mathit{map}\ \mathit{fst}\ p \star \mathit{map}\ \mathit{snd}\ p = p\ . \tag{9}$$

It is important to note that the Laws (7)–(9) do not hold for arbitrary idioms. Perhaps surprisingly, (the uncurried version of) $\star$ is also a homomorphism (from $\Delta \cdot \mathit{Stream}$ to $\mathit{Stream} \cdot \Delta$). In the case of streams or, more generally, *commutative idioms*, $\star$ is even an idiom isomorphism, an idiom homomorphism which has an inverse.

Since the arithmetic operations are defined pointwise, the familiar arithmetic laws also hold for streams. In proofs we will signal their use by the hint *arithmetic*.

Interleaving The interleaving operator interacts nicely with lifting.

$$\begin{aligned} \mathit{pure}\ a \curlyvee \mathit{pure}\ a &= \mathit{pure}\ a && (10)\\ (s_1 \diamond s_2) \curlyvee (t_1 \diamond t_2) &= (s_1 \curlyvee t_1) \diamond (s_2 \curlyvee t_2) && (11) \end{aligned}$$

A simple consequence is $(s \curlyvee t) + 1 = s + 1 \curlyvee t + 1$ or, more generally, $\mathit{map}\ f\ (s \curlyvee t) = \mathit{map}\ f\ s \curlyvee \mathit{map}\ g\ t$. The two laws show, in fact, that interleaving is a homomorphism (from $\Delta \cdot \mathit{Stream}$ to Stream). Interleaving is even an isomorphism; the reader is encouraged to work out the details.

Property (11) is also called *abide law* because of the following two-dimensional way of writing the law, in which the two operators are written either *ab*ove or bes*ide* each other.

$$\begin{array}{ccc} s_1 & \diamond & s_2 \\ & \curlyvee & \\ t_1 & \diamond & t_2 \end{array} \quad = \quad \begin{array}{ccc} s_1 & & s_2 \\ \curlyvee & \diamond & \curlyvee \\ t_1 & & t_2 \end{array} \qquad\qquad \begin{array}{c|c} s_1 & s_2 \\ \hline t_1 & t_2 \end{array} \quad = \quad \begin{array}{c|c} s_1 & s_2 \\ \hline\hline t_1 & t_2 \end{array}$$

The two-dimensional arrangement is originally due to Hoare, the catchy name is due to Bird (1987). The geometrical interpretation can be emphasised further by writing the two operators $|$ and $-$, like on the right-hand side (Fokkinga, 1992).

Tabulation The functions *lookup* and *tabulate* are mutually inverse and they satisfy the following naturality properties.

$$\begin{aligned} \mathit{map}\ f \cdot \mathit{tabulate} &= \mathit{tabulate} \cdot (f \cdot)\\ (f \cdot) \cdot \mathit{lookup} &= \mathit{lookup} \cdot \mathit{map}\ f \end{aligned}$$

Note that post-composition $(f \cdot)$ is the mapping function for the functor $\tau \to$. The laws are somewhat easier to memorise, if we write them in a pointwise style.

$$\begin{aligned} \mathit{map}\ f\ (\mathit{tabulate}\ g) &= \mathit{tabulate}\ (f \cdot g) && (12)\\ f \cdot \mathit{lookup}\ t &= \mathit{lookup}\ (\mathit{map}\ f\ t) && (13) \end{aligned}$$

Furthermore, *tabulate* and *lookup* are idiom isomorphisms between *Stream* and *Nat* $\to$. We will exploit this fact in Section 3.2.1, when we look at tabulation in more detail.

Final coalgebra On streams, recursion and iteration coincide.

$$\mathit{recurse}\ f\ a = \mathit{iterate}\ f\ a$$

The function *iterate* satisfies an important fusion law, which amounts to the free theorem of $(\alpha \to \alpha) \to (\alpha \to \mathit{Stream}\ \alpha)$.

$$\mathit{map}\ h \cdot \mathit{iterate}\ f_1 = \mathit{iterate}\ f_2 \cdot h \quad \Longleftarrow \quad h \cdot f_1 = f_2 \cdot h \qquad (14)$$

The *unfold* or anamorphism of the *Stream* datatype is characterised by the following *universal property*.

$$h = \mathit{unfold}\ g\ f \quad \Longleftrightarrow \quad \mathit{head} \cdot h = g \ \text{ and } \ \mathit{tail} \cdot h = h \cdot f \qquad (15)$$

Read from left to right the universal property states that $\mathit{unfold}\ g\ f$ is a solution of the equations $\mathit{head} \cdot h = g$ and $\mathit{tail} \cdot h = h \cdot f$. Read from right to left the property asserts that $\mathit{unfold}\ g\ f$ is indeed the unique solution.

The universal property has a multitude of consequences. Instantiating g to *head* and f to *tail*, we obtain the *reflection law*.

$$\mathit{unfold}\ \mathit{head}\ \mathit{tail} \quad = \quad \mathit{id}$$

Instantiating h to $\mathit{unfold}\ g\ f$, gives the two *computation rules*, which are point-free versions of the defining equation of *unfold*.

$$\begin{aligned} \mathit{head} \cdot \mathit{unfold}\ g\ f &= g \\ \mathit{tail} \cdot \mathit{unfold}\ g\ f &= \mathit{unfold}\ g\ f \cdot f \end{aligned}$$

Since the type of *unfold* involves two type variables, *unfold* enjoys two fusion laws: *fusion* and *functor fusion*.

$$\mathit{unfold}\ g\ f \cdot h = \mathit{unfold}\ g'\ f' \quad \Longleftarrow \quad g \cdot h = g' \ \text{and}\ \ f \cdot h = h \cdot f'$$

$$\mathit{map}\ h \cdot \mathit{unfold}\ g\ f \quad = \quad \mathit{unfold}\ (h \cdot g)\ f$$

Both are, in fact, also consequences of the *iterate*-fusion law (14).

As an aside, the pair of functions $(\mathit{head}, \mathit{tail})$ forms a so-called coalgebra for the functor $F\ X = A \times X$. The stream coalgebra is final as there is a unique coalgebra homomorphism from every other coalgebra (g, f) to the stream coalgebra: $\mathit{unfold}\ g\ f$ — this is the import of the universal property (15).

2.5 Proofs

The last section has listed a multitude of laws. It is high time that we show how to prove them correct. In Section 2.3 we have planted the seeds by restricting the syntactic form of equations so that they possess unique solutions. It is now time to reap the harvest. If $x = \phi\ x$ is an admissible equation, we denote its unique solution by $\mathit{fix}\ \phi$. (The equation implicitly defines a function in x. A solution of the equation is a fixed point of this function and vice versa.) The fact that the solution is unique is captured by the following property.

$$\mathit{fix}\ \phi = s \quad \Longleftrightarrow \quad \phi\ s = s$$

Read from left to right it states that $\mathit{fix}\ \phi$ is indeed a solution of $x = \phi\ x$. Read from right to left it asserts that any solution is equal to $\mathit{fix}\ \phi$. The universal property of *unfold* is an instance of this scheme. Now, if we want to prove $s = t$ where $s = \mathit{fix}\ \phi$, then it suffices to show that $\phi\ t = t$.

As a first example, let us prove the *idiom homomorphism law*.

$$\begin{aligned} & \mathit{pure}\ f \diamond \mathit{pure}\ a \\ = \quad & \{\ \text{definition of}\ \diamond\ \} \\ & (\mathit{head}\ (\mathit{pure}\ f))\ (\mathit{head}\ (\mathit{pure}\ a)) \prec \mathit{tail}\ (\mathit{pure}\ f) \diamond \mathit{tail}\ (\mathit{pure}\ a) \\ = \quad & \{\ \text{definition of}\ \mathit{pure}\ \} \\ & f\ a \prec \mathit{pure}\ f \diamond \mathit{pure}\ a \end{aligned}$$

Consequently, $\mathit{pure}\ f \diamond \mathit{pure}\ a$ equals the unique solution of $x = f\ a \prec x$, which by definition is $\mathit{pure}\ (f\ a)$.

That was easy. The next proof is not much harder. We show that the natural numbers are even and odd numbers interleaved: $nat = 2 * nat \curlyvee 2 * nat + 1$.

$$\begin{array}{ll} & 2 * nat \curlyvee 2 * nat + 1 \\ = & \quad \{ \text{ definition of } nat \ \} \\ & 2 * (0 \prec nat + 1) \curlyvee 2 * nat + 1 \\ = & \quad \{ \text{ arithmetic } \} \\ & (0 \prec 2 * nat + 2) \curlyvee 2 * nat + 1 \\ = & \quad \{ \text{ definition of } \curlyvee \ \} \\ & 0 \prec 2 * nat + 1 \curlyvee 2 * nat + 2 \\ = & \quad \{ \text{ arithmetic } \} \\ & 0 \prec (2 * nat \curlyvee 2 * nat + 1) + 1 \end{array}$$

Inspecting the second but last term, we note that the result implies $nat = 0 \prec 2 * nat + 1 \curlyvee 2 * nat + 2$, which in turn proves $nat = bin$.

Now, if both s and t are given as fixed points, $s = fix\ \phi$ and $t = fix\ \psi$, then there are at least four possibilities to prove $s = t$:

$$\begin{array}{lllll} \phi\ (\psi\ s) = \psi\ s & \Longrightarrow & \psi\ s = s & \Longrightarrow & s = t \ ; \\ \psi\ (\phi\ t) = \phi\ t & \Longrightarrow & \phi\ t = t & \Longrightarrow & s = t \ . \end{array}$$

We may be lucky and establish one of the equations. Unfortunately, there is no success guarantee. The following approach is often more promising. We show $s = \chi\ s$ and $\chi\ t = t$. If χ has a unique fixed point, then $s = t$. The point is that we discover the function χ on the fly during the calculation. Proofs in this style are laid out as follows.

$$\begin{array}{ll} & s \\ = & \quad \{ \text{ why? } \} \\ & \chi\ s \\ \subset & \quad \{ \ x = \chi\ x \text{ has a unique solution } \} \\ & \chi\ t \\ = & \quad \{ \text{ why? } \} \\ & t \end{array}$$

The symbol $\subset$ is meant to suggest a link connecting the upper and the lower part. Overall, the proof establishes that $s = t$.

Let us illustrate the technique by proving *Cassini's identity*: $fib'^2 - fib * fib'' = (-1)^{nat}$ (recall that exponentiation is also lifted to streams).

$$\begin{array}{ll} & fib'^2 - fib * fib'' \\ = & \quad \{ \text{ definition of } fib'' \text{ and arithmetic } \} \\ & fib'^2 - (fib^2 + fib * fib') \\ = & \quad \{ \text{ definition of } fib \text{ and definition of } fib' \ \} \\ & 1 \prec (fib''^2 - (fib'^2 + fib' * fib'')) \end{array}$$

$$
\begin{aligned}
&= && \{\, \mathit{fib}'' - \mathit{fib}' = \mathit{fib} \text{ and arithmetic} \,\} \\
&\quad 1 \prec (-1) * (\mathit{fib}'^2 - \mathit{fib} * \mathit{fib}'') \\
&\subset && \{\, x = 1 \prec (-1) * x \text{ has a unique solution} \,\} \\
&\quad 1 \prec (-1) * (-1)^{\mathit{nat}} \\
&= && \{\, \text{definition of } \mathit{nat} \text{ and arithmetic} \,\} \\
&\quad (-1)^{\mathit{nat}}
\end{aligned}
$$

When reading $\subset$-proofs, it is easiest to start at both ends working towards the link. Each part follows a typical pattern, which we will see time and time again: starting with e we unfold the definitions obtaining $e_1 \prec e_2$; then we try to express e_2 in terms of e.

So far, we have been concerned with proofs about streams. However, the proof techniques apply equally well to parametric streams and stream operators! As an example, let us prove the abide law by showing $f = g$ where

$$f\ s_1\ s_2\ t_1\ t_2 = (s_1 \diamond s_2) \curlyvee (t_1 \diamond t_2) \quad \text{and} \quad g\ s_1\ s_2\ t_1\ t_2 = (s_1 \curlyvee t_1) \diamond (s_2 \curlyvee t_2) \ .$$

The proof is straightforward involving only bureaucratic steps.

$$
\begin{aligned}
&\quad f\ a\ b\ c\ d \\
&= \quad \{\, \text{definition of } f \,\} \\
&\quad (a \diamond b) \curlyvee (c \diamond d) \\
&= \quad \{\, \text{definition of } \diamond \text{ and definition of } \curlyvee \,\} \\
&\quad \mathit{head}\ a \diamond \mathit{head}\ b \prec (c \diamond d) \curlyvee (\mathit{tail}\ a \diamond \mathit{tail}\ b) \\
&= \quad \{\, \text{definition of } f \,\} \\
&\quad \mathit{head}\ a \diamond \mathit{head}\ b \prec f\ c\ d\ (\mathit{tail}\ a)\ (\mathit{tail}\ b) \\
&\subset \quad \{\, x\ s_1\ s_2\ t_1\ t_2 = \mathit{head}\ s_1 \diamond \mathit{head}\ s_2 \prec x\ t_1\ t_2\ (\mathit{tail}\ s_1)\ (\mathit{tail}\ s_2) \text{ has a unique solution} \,\} \\
&\quad \mathit{head}\ a \diamond \mathit{head}\ b \prec g\ c\ d\ (\mathit{tail}\ a)\ (\mathit{tail}\ b) \\
&= \quad \{\, \text{definition of } g \,\} \\
&\quad \mathit{head}\ a \diamond \mathit{head}\ b \prec (c \curlyvee \mathit{tail}\ a) \diamond (d \curlyvee \mathit{tail}\ b) \\
&= \quad \{\, \text{definition of } \diamond \text{ and definition of } \curlyvee \,\} \\
&\quad (a \curlyvee c) \diamond (b \curlyvee d) \\
&= \quad \{\, \text{definition of } g \,\} \\
&\quad g\ a\ b\ c\ d
\end{aligned}
$$

Henceforth, we leave the two functions implicit sparing ourselves two rolling and two unrolling steps. On the downside, this makes the common pattern around the link more difficult to spot.

A popular benchmark for the effectiveness of proof methods for corecursive programs is the *iterate*-fusion law (14). The 'unique fixed-point proof' is short and sweet; it compares favourably to the ones given by Gibbons and Hutton (2005).

$$
\begin{aligned}
&\quad \mathit{map}\ h\ (\mathit{iterate}\ f_1\ a) \\
&= \quad \{\, \text{definition of } \mathit{iterate} \text{ and definition of } \mathit{map} \,\}
\end{aligned}
$$

$$
\begin{array}{ll}
 & h\ a \prec map\ h\ (iterate\ f_1\ (f_1\ a)) \\
\subset & \quad \{\ x\ a = h\ a \prec x\ (f_1\ a) \text{ has a unique solution } \} \\
 & h\ a \prec iterate\ f_2\ (h\ (f_1\ a)) \\
= & \quad \{\ \text{assumption: } h \cdot f_1 = f_2 \cdot h\ \} \\
 & h\ a \prec iterate\ f_2\ (f_2\ (h\ a)) \\
= & \quad \{\ \text{definition of } iterate\ \} \\
 & iterate\ f_2\ (h\ a)
\end{array}
$$

Unsurprisingly, the linking equation $g\ a = h\ a \prec g\ (f_1\ a)$ corresponds to the *unfold* operator, which as we have noted can be defined in terms of *map* and *iterate*.

The fusion law implies $map\ f \cdot iterate\ f = iterate\ f \cdot f$, which is the key for proving $nat = iterate\ (+1)\ 0$, or, more generally, $recurse\ f\ a = iterate\ f\ a$. We show that $iterate\ f\ a$ is the unique solution of $x = a \prec map\ f\ x$.

$$
\begin{array}{ll}
 & iterate\ f\ a \\
= & \quad \{\ \text{definition of } iterate\ \} \\
 & a \prec iterate\ f\ (f\ a) \\
= & \quad \{\ \text{iterate fusion law (14): } h = f_1 = f_2 = f\ \} \\
 & a \prec map\ f\ (iterate\ f\ a)
\end{array}
$$

Let us conclude by noting that the proof techniques are not always applicable. For example, they cannot be used to show the *extensionality* law (2), $s = head\ s \prec tail\ s$, simply because s is not given by a recursion equation. In this case, we have to resort to coinduction.

3 Recurrences ($\prec$, Υ)

Using $\prec$ and Υ we can easily capture recurrences or functions from the natural numbers. Though functions from the naturals and streams are in one-to-one correspondence, a stream is often easier to manipulate than a function or a recurrence, because a stream is a single entity, typically defined by a single equation. Before we consider concrete examples, we first explore tabulation in more depth.

3.1 Tabulation

The simplest recurrences are of the form $a_0 = k$ and $a_{n+1} = f(a_n)$, for some natural number k and some function f on the naturals. This recurrence is captured by the stream equation $a = k \prec map\ f\ a$, or more succinctly by $recurse\ f\ k$. Though fairly obvious, the relation is worth exploring. On the face of it, the linear recurrence corresponds to the *fold* or *catamorphism* of the inductive type *Nat*.

$$
\begin{array}{ll}
fold & :: (\alpha \to \alpha) \to \alpha \to (Nat \to \alpha) \\
fold\ s\ z\ 0 & = z \\
fold\ s\ z\ (n+1) & = s\ (fold\ s\ z\ n)
\end{array}
$$

Catamorphisms are dual to anamorphisms, enjoying a dual characterisation.

$$h = \mathit{fold}\ s\ z \quad \Longleftrightarrow \quad h\ 0 = z \ \text{ and } \ h \cdot (+1) = s \cdot h$$

Some simple consequences of the universal property are the *reflection law*, $\mathit{fold}\ (+1)\ 0 = \mathit{id}$, and the *computation rules*, $\mathit{fold}\ s\ z\ 0 = z$ and $\mathit{fold}\ s\ z \cdot (+1) = s \cdot \mathit{fold}\ s\ z$.

We already noted that tabulating $\mathit{fold}\ s\ z$ gives $\mathit{recurse}\ s\ z$. The proof of this fact makes crucial use of *tabulate*'s naturality property.

$$\begin{array}{ll} & \mathit{tabulate}\ (\mathit{fold}\ s\ z) \\ = & \quad \{\ \text{definition of } \mathit{tabulate}\ \} \\ & \mathit{fold}\ s\ z\ 0 \prec \mathit{tabulate}\ (\mathit{fold}\ s\ z \cdot (+1)) \\ = & \quad \{\ \text{computation rules, see above}\ \} \\ & z \prec \mathit{tabulate}\ (s \cdot \mathit{fold}\ s\ z) \\ = & \quad \{\ \text{naturality of } \mathit{tabulate}\ (12)\ \} \\ & z \prec \mathit{map}\ s\ (\mathit{tabulate}\ (\mathit{fold}\ s\ z)) \end{array}$$

Consequently, there are, at least, four equivalent ways of expressing the linear recurrence $a_0 = z$ and $a_{n+1} = s(a_n)$.

$$\mathit{tabulate}\ (\mathit{fold}\ s\ z) = \mathit{recurse}\ s\ z = \mathit{iterate}\ s\ z = \langle x \mid x \leftarrow z, s\ x\ldots \rangle$$

Using the reflection law for *Nat*, this furthermore implies that *nat* is the tabulation of the identity function: $\mathit{tabulate}\ \mathit{id} = \mathit{tabulate}\ (\mathit{fold}\ (+1)\ 0) = \mathit{iterate}\ (+1)\ 0 = \mathit{nat}$.

If we 'un-tabulate' *iterate*, setting $\mathit{loop}\ f = \mathit{lookup} \cdot \mathit{iterate}\ f$, we obtain a *tail-recursive* function, which can be seen as the counterpart of Haskell's *foldl* for natural numbers.

$$\begin{array}{ll} \mathit{loop} & :: (\alpha \to \alpha) \to \alpha \to (\mathit{Nat} \to \alpha) \\ \mathit{loop}\ f\ a\ 0 & = a \\ \mathit{loop}\ f\ a\ (n+1) & = \mathit{loop}\ f\ (f\ a)\ n \end{array}$$

Mathematically speaking, *loop* programs n-fold composition: $\mathit{loop}\ f\ a\ n = f^{(n)}\ a$.

The fact that *nat* tabulates *id* has a very simple, yet important consequence:

$$\begin{array}{ll} & \mathit{tabulate}\ f \\ = & \quad \{\ f \cdot \mathit{id} = f\ \} \\ & \mathit{tabulate}\ (f \cdot \mathit{id}) \\ = & \quad \{\ \text{naturality of } \mathit{tabulate}\ (12)\ \} \\ & \mathit{map}\ f\ (\mathit{tabulate}\ \mathit{id}) \\ = & \quad \{\ \mathit{tabulate}\ \mathit{id} = \mathit{nat}\ \} \\ & \mathit{map}\ f\ \mathit{nat} \\ = & \quad \{\ \text{stream comprehensions}\ \} \\ & \langle f\ n \mid n \leftarrow \mathit{nat} \rangle \ . \end{array}$$

Setting $f = \mathit{lookup}\ s$ in the equation above, gives the following *extensionality law*

$$s \quad = \quad \langle (s)_n \mid n \leftarrow \mathit{nat} \rangle \ , \tag{16}$$

which allows us to switch swiftly between a pointwise and a point-free or holistic view.

Now, let us tackle a slightly more involved class of recurrences. The sequence given by the 'binary' recurrence $a_0 = k$, $a_{2n+1} = f(a_n)$ and $a_{2n+2} = g(a_n)$ corresponds to the stream $a = k \prec \mathit{map}\ f\ a \curlyvee \mathit{map}\ g\ a$. Turning to the proof, observe that the extensionality law (16) implies $a = \langle a_n \mid n \leftarrow \mathit{nat} \rangle = \langle a_n \mid n \leftarrow \mathit{bin} \rangle$. The calculation below then proves that the latter expression satisfies the stream equation above.

$$
\begin{aligned}
& \langle a_n \mid n \leftarrow \mathit{bin} \rangle \\
= & \quad \{ \text{ definition of } \mathit{bin} \text{ and abide law (11) } \} \\
& a_0 \prec \langle a_n \mid n \leftarrow 2 * \mathit{bin} + 1 \rangle \curlyvee \langle a_n \mid n \leftarrow 2 * \mathit{bin} + 2 \rangle \\
= & \quad \{ \text{ functor law (4) } \} \\
& a_0 \prec \langle a_{2*n+1} \mid n \leftarrow \mathit{bin} \rangle \curlyvee \langle a_{2*n+2} \mid n \leftarrow \mathit{bin} \rangle \\
= & \quad \{ \text{ definition of } a \ \} \\
& k \prec \langle f(a_n) \mid n \leftarrow \mathit{bin} \rangle \curlyvee \langle g(a_n) \mid n \leftarrow \mathit{bin} \rangle \\
= & \quad \{ \text{ functor law (4) } \} \\
& k \prec \mathit{map} f\ \langle a_n \mid n \leftarrow \mathit{bin} \rangle \curlyvee \mathit{map}\ g\ \langle a_n \mid n \leftarrow \mathit{bin} \rangle
\end{aligned}
$$

The crucial step in the derivation is easily overlooked: it is the application of $\mathit{nat} = \mathit{bin}$ that makes the calculation fly.

To tabulate an arbitrary recurrence or an arbitrary function from the naturals, we can either apply *tabulate* or simply map the function over the stream of natural numbers. Of course, if f is recursively defined, $f = \phi\ f$, then this is a wasted opportunity, as the recursive calls do not re-use any of the tabulated values. In order to also replace these calls by table look-ups, we invoke the *rolling rule* of fixed-point calculus (Backhouse, 2001).

$$
\begin{aligned}
& \mathit{fix}\ \phi \\
= & \quad \{ \ \mathit{lookup} \cdot \mathit{tabulate} = \mathit{id} \ \} \\
& \mathit{fix}\ (\mathit{lookup} \cdot \mathit{tabulate} \cdot \phi) \\
= & \quad \{ \text{ rolling rule: } \mathit{fix}\ (f \cdot g) = f\ (\mathit{fix}\ (g \cdot f)) \ \} \\
& \mathit{lookup}\ (\mathit{fix}\ (\mathit{tabulate} \cdot \phi \cdot \mathit{lookup}))
\end{aligned}
$$

The calculation transforms a fixed point over functions into a fixed point over streams. Introducing names for the participants, we obtain $f = \mathit{lookup}\ s$ and $s = \mathit{tabulate}\ (\phi\ f)$. While the derivation is completely general, each call of f now involves a linear look-up. By contrast, the tabulations for linear and 'binary' recurrences avoid the look-up by maintaining 'pointers' into the memo table. As an aside, the derivation above is an instance of the *worker-wrapper transformation* (Gill & Hutton, 2009).

3.2 Examples

In the previous section we have considered abstract recurrences. Let us tackle some concrete examples now. In the next section we re-visit our all-time-favourite, the Fibonacci numbers, and then move on to examine some less known examples in the sections thereafter.

3.2.1 Fibonacci numbers

There and Back Again
John Ronald Reuel Tolkien (1937)

The recurrence of $\mathscr{F}$, see Section 2.1, does not fit the schemes previously discussed, so we have to start afresh. (We could apply the general scheme for recursive functions, but this involves linear look-ups.) The calculations are effortless, however, if we make use of the fact that *tabulate* is an idiom homomorphism. Using the environment idiom we can rewrite the last equation of $\mathscr{F}$ in a point-free style: $\mathscr{F} \cdot (+2) = \mathscr{F} + \mathscr{F} \cdot (+1)$.

$$
\begin{aligned}
& \mathit{tabulate}\ \mathscr{F} \\
= \quad & \{ \text{ definition of } \mathit{tabulate}\text{, twice } \} \\
& \mathscr{F}_0 \prec \mathscr{F}_1 \prec \mathit{tabulate}\ (\mathscr{F} \cdot (+2)) \\
= \quad & \{ \text{ definition of } \mathscr{F} \} \\
& 0 \prec 1 \prec \mathit{tabulate}\ (\mathscr{F} + \mathscr{F} \cdot (+1)) \\
= \quad & \{ \mathit{tabulate} \text{ is an idiom homomorphism } \} \\
& 0 \prec 1 \prec \mathit{tabulate}\ \mathscr{F} + \mathit{tabulate}\ (\mathscr{F} \cdot (+1)) \\
= \quad & \{ \text{ definition of } \mathit{tabulate} \} \\
& 0 \prec 1 \prec \mathit{tabulate}\ \mathscr{F} + \mathit{tail}\ (\mathit{tabulate}\ \mathscr{F})
\end{aligned}
$$

The only non-trivial step is the second but last one, which employs $\mathit{tabulate}\ (f+g) = \mathit{tabulate}\ f + \mathit{tabulate}\ g$, which in turn is syntactic sugar for $\mathit{tabulate}\ (\mathit{pure}\ (+) \diamond f \diamond g) = \mathit{pure}\ (+) \diamond \mathit{tabulate}\ f \diamond \mathit{tabulate}\ g$. Since *tabulate* preserves the idiomatic structure, the derivation goes through nicely.

Tabulation and look-up allow us to switch swiftly between functions from the naturals and streams. So, even if coinductive structures are not available in your language of choice, you can still use stream calculus for program transformations. Let us illustrate this point by deriving an efficient *iterative* implementation of $\mathscr{F}$. To this end we have to transform *fib* into a form that fits the recursion equation of *iterate* or *recurse*. This can be easily achieved using the standard technique of *tupling* (Bird, 1998), which boils down to an application of lifted pairing $\star$.

$$
\begin{aligned}
& \mathit{fib} \star \mathit{fib}' && \text{(A)} \\
= \quad & \{ \text{ definition of } \mathit{fib} \text{ and definition of } \mathit{fib}' \} \\
& (0 \prec \mathit{fib}') \star (1 \prec \mathit{fib} + \mathit{fib}') \\
= \quad & \{ \text{ definition of } \star \} \\
& (0,1) \prec \mathit{fib}' \star (\mathit{fib} + \mathit{fib}') \\
= \quad & \{ \text{ introduce } \mathit{step}\ (a,b) = (b, a+b) \text{ and lift the equality to streams } \} \\
& (0,1) \prec \mathit{map}\ \mathit{step}\ (\mathit{fib} \star \mathit{fib}')
\end{aligned}
$$

The last step makes use of the fact that equations between elements, such as $\mathit{step}\ (a,b) = (b, a+b)$, can be lifted to equations between streams, $\mathit{map}\ \mathit{step}\ (s \star t) = t \star (s+t)$, see also (Hinze, 2010a). The calculation shows that $\mathit{fib} \star \mathit{fib}'$ satisfies the recursion equation of

iterate, we have $fib \star fib' = iterate\ step\ (0,1)$. Consequently,

$$fib = map\ fst\ (iterate\ step\ (0,1)) = unfold\ fst\ step\ (0,1) = \langle a \mid (a,b) \leftarrow (0,1), (b,a+b)\,..\rangle.$$

To obtain an iterative or tail-recursive program, we simply 'un-tabulate' the derived stream expression.

$$\begin{array}{ll} & lookup\ fib \\ = & \quad \{\ \text{see above}\ \} \\ & lookup\ (map\ fst\ (iterate\ step\ (0,1))) \\ = & \quad \{\ \text{naturality of } lookup\ (12)\ \} \\ & fst\ (lookup\ (iterate\ step\ (0,1))) \\ = & \quad \{\ \text{definition of } loop\ \} \\ & fst\ (loop\ step\ (0,1)) \end{array}$$

An alternative approach is based on the observation that the tails of *fib* are linear combinations of *fib* and *fib*′. Let i and j be constant streams, then

$$\begin{array}{ll} & i * fib + j * fib' \qquad \text{(B)} \\ = & \quad \{\ \text{definition of } fib \text{ and definition of } fib'\ \} \\ & i * (0 \prec fib') + j * (1 \prec fib + fib') \\ = & \quad \{\ \text{arithmetic}\ \} \\ & j \prec i * fib' + j * (fib + fib') \\ = & \quad \{\ \text{arithmetic}\ \} \\ & j \prec j * fib + (i+j) * fib' \ . \end{array}$$

Viewing $i * fib + j * fib'$ as a function in (i,j), the calculation shows that the linear combination satisfies the recursion equation of *unfold*, we have $i * fib + j * fib' = unfold\ snd\ step\ (i,j)$ and hence $fib = unfold\ snd\ step\ (1,0)$.

The two iterative versions of *fib* are tantalisingly close. We may, in fact, be fooled into thinking that we can obtain one from the other by fusing with $swap\ (a,b) = (b,a)$. A moment's reflection shows that this does not work since the 'next' function is the same in both cases. However, the two derivations imply a simple formula for a generalised version of *fib* that abstracts away from the two initial values.

$$\begin{array}{l} pfib\ i\ j = s \ \textbf{where}\ s = i \prec s' \\ \qquad\qquad\qquad\qquad\quad s' = j \prec s + s' \end{array}$$

We can express *pfib i j* in terms of *fib* and *fib*′, since

$$\begin{array}{ll} & tail\ (pfib\ i\ j) \\ = & \quad \{\ \text{tupling, analogous to the first derivation (A)}\ \} \\ & map\ snd\ (iterate\ step\ (i,j)) \\ = & \quad \{\ map\ g \cdot iterate\ f = unfold\ g\ f\ \} \\ & unfold\ snd\ step\ (i,j) \\ = & \quad \{\ \text{second derivation (B)}\ \} \end{array}$$

$$i * fib + j * fib' \quad .$$

Furthermore, $pfib\ i\ j = (j-i) * fib + i * fib'$. So, in a sense, the extra generality of *pfib* is not needed.

As an example application, let us calculate the number of binary strings of a given length that do not contain adjacent 1s. These binary strings are used in the Fibonacci number system (Graham *et al.*, 1994). The empty string clearly satisfies the condition; a leading 0 can be followed by any string that satisfies the criterion; if the string starts with a 1, then it must be followed by a string that does not start with a 1. The description can be captured by the following grammar in Backus-Naur Form (*Nao* is short for no adjacent ones).

$$\begin{aligned} Nao &= \varepsilon \mid 0\ Nao \mid 1\ Nap \\ Nap &= \varepsilon \mid 0\ Nao \end{aligned}$$

Here, *Nap* produces *Nao* strings that do not start with a 1. Now, to solve the original problem we have to group the strings by length. Let *nao* (*nap*) be the stream whose *n*th element specifies the number of words of length *n* produced by *Nao* (*Nap*). The language *Nao* contains one string of length 0 and $nao + nap$ strings of length $n+1$, motivating the following stream definitions.

$$\begin{aligned} nao &= 1 \prec nao + nap \\ nap &= 1 \prec nao \end{aligned}$$

By the token above, $nao = tail\ (pfib\ 1\ 1) = 1 * fib + 1 * fib' = fib''$. In Section 5.5 we will show how to systematise the translation of grammars into stream equations.

The elements of a stream are by no means confined to numbers. Using a lifted version of equality, $\doteq$, we can, for instance, generate a stream of Booleans. To illustrate the use of $\doteq$, let us prove that every third Fibonacci number is even: $(fib\ \mathbf{mod}\ 2 \doteq 0) = (nat\ \mathbf{mod}\ 3 \doteq 0)$. The proof makes essential use of the fact that equality on *Bool* is associative (Backhouse & Fokkinga, 2001). We show that *even fib* satisfies $x = true \prec false \prec false \prec x$, where *even* is lifted to streams — recall that, like integer constants, *false* and *true* are overloaded.

$$\begin{aligned} & even\ fib \\ = & \quad \{ \text{ definition of } fib, \text{ definition of } fib' \text{ and definition of } even \ \} \\ & true \prec false \prec even\ (fib + fib') \\ = & \quad \{ even\ (a+b) = even\ a = even\ b, \text{ and lifting } \} \\ & true \prec false \prec (even\ fib \doteq even\ fib') \\ = & \quad \{ \text{ definition of } fib \text{ and definition of } fib' \ \} \\ & true \prec false \prec ((true \prec even\ fib') \doteq (false \prec (even\ fib \doteq even\ fib'))) \\ = & \quad \{ \text{ definition of } \doteq \ \} \\ & true \prec false \prec false \prec (even\ fib' \doteq even\ fib \doteq even\ fib') \\ = & \quad \{ a = a = true \text{ (twice), and lifting } \} \\ & true \prec false \prec false \prec even\ fib \end{aligned}$$

3.2.2 Bit fiddling

Bitte ein Bit
Slogan of "Bitburger Brauerei" (1951)

Let us move on to some less known examples, which are meant to illustrate the use of interleaving. All of the examples involve fiddling with bits, so we start off defining the stream of all bit strings.

$$bit = [\,] \prec 0 +\!\!+ bit \curlyvee 1 +\!\!+ bit$$

Here list concatenation $+\!\!+$ is lifted to streams. Furthermore, the literals 0 and 1 are doubly lifted, first to lists and then to streams: n expands to the stream of singleton lists $pure\ [n]$. The stream of positive numbers in binary representation, least significant bit first, is then $bit +\!\!+ 1$, and the stream of binary naturals is $[\,] \prec bit +\!\!+ 1$.

$$\gg\ bit$$
$$\langle [\,], [0], [1], [0,0], [1,0], [0,1], [1,1], [0,0,0], [1,0,0], [0,1,0], [1,1,0], [0,0,1], \ldots \rangle$$
$$\gg\ [\,] \prec bit +\!\!+ 1$$
$$\langle [\,], [1], [0,1], [1,1], [0,0,1], [1,0,1], [0,1,1], [1,1,1], [0,0,0,1], [1,0,0,1], \ldots \rangle$$

The function $lookup\ ([\,] \prec bit +\!\!+ 1)$ maps a natural number to its binary representation. Its inverse is the evaluation function $val = foldr\ (\lambda b\ n \to b + 2 * n)\ 0$.

It will be useful to define a variant of *bit* where all the bits have been zeroed out.

$$null = [\,] \prec 0 +\!\!+ null \curlyvee 0 +\!\!+ null$$

Now, how can we characterise the *pots*, the positive numbers that are powers of two (*A*036987)? A simple specification is

$$pot = (bit +\!\!+ 1 \doteq null +\!\!+ 1)\ .$$

All the bits of a pot are zero, except for the most significant bit. The specification above is equivalent to $pot = (bit \doteq null)$ — recall that identities between elements, such as $x +\!\!+ z == y +\!\!+ z \Longleftrightarrow x == y$, can be lifted to identities between streams. From the simplified specification we can easily calculate a definition of *pot*.

$$\begin{aligned}
& bit \doteq null \\
= \quad & \{\ \text{definition of } bit \text{ and definition of } null\ \} \\
& ([\,] \prec 0 +\!\!+ bit \curlyvee 1 +\!\!+ bit) \doteq ([\,] \prec 0 +\!\!+ null \curlyvee 0 +\!\!+ null) \\
= \quad & \{\ \text{definition of } \doteq \text{ and abide law (11)}\ \} \\
& ([\,] == [\,]) \prec (0 +\!\!+ bit \doteq 0 +\!\!+ null) \curlyvee (1 +\!\!+ bit \doteq 0 +\!\!+ null) \\
= \quad & \{\ \text{equality}\ \} \\
& true \prec (bit \doteq null) \curlyvee false
\end{aligned}$$

Joining the first with the last line we obtain a definition for *pot*.

$$pot = true \prec pot \curlyvee false$$

Using a similar approach we can characterise the most significant bit of a positive number ($0 \prec msb$ is *A*053644):

$$msb = val\ (null +\!\!+ 1)\ .$$

A similar derivation to the one above gives

$$msb = 1 \prec 2 * msb \curlyvee 2 * msb \ .$$

Put differently, *msb* is the largest pot at most nat'. Here and in what follows we also lift relations, "x and y are related by R", to streams. So, instead of "$n+1$ is odd iff n is even", where n is implicitly universally quantified, we say "$nat+1$ is odd iff *nat* is even", where the relations are implicitly lifted to streams.

We can use *msb* to bit-reverse a positive number: all bits except the most significant one are reversed. The stream *brp* is specified (*A*059893)

$$brp \quad = \quad val\ (reverse\ bit +\!\!+ 1) \ ,$$

where *val* and *reverse* are both lifted to streams. The derivation makes use of a simple property of *val*: if the bit strings x and y have the same length, $length\ x = length\ y$, then

$$val\ (x +\!\!+ z) - val\ x \quad = \quad val\ (y +\!\!+ z) - val\ y \ .$$

Since $val\ (null +\!\!+ 1) - val\ null = msb$, this implies the property

$$val\ (reverse\ bit +\!\!+ 1) - val\ (reverse\ bit) \quad = \quad msb \tag{17}$$

between streams. The derivation proceeds as follows

$$\begin{array}{ll} & val\ (reverse\ bit +\!\!+ 1) \\ = & \quad \{\ (17)\ \} \\ & val\ (reverse\ bit) + msb \\ = & \quad \{\ \text{definition of } bit\text{, definition of } reverse\text{, and definition of } val\ \} \\ & (0 \prec val\ (reverse\ bit) \curlyvee val\ (reverse\ bit +\!\!+ 1)) + msb \\ = & \quad \{\ \text{definition of } msb \text{ and abide law (11)}\ \} \\ & 1 \prec val\ (reverse\ bit) + 2 * msb \curlyvee val\ (reverse\ bit +\!\!+ 1) + 2 * msb \\ = & \quad \{\ (17)\ \} \\ & 1 \prec val\ (reverse\ bit +\!\!+ 1) + msb \curlyvee val\ (reverse\ bit +\!\!+ 1) + 2 * msb \ . \end{array}$$

Consequently, *brp* is defined

$$brp = 1 \prec brp + msb \curlyvee brp + 2 * msb \ .$$

Another example along these lines is the 1s-counting sequence (*A*000120), also known as the *binary weight*. The binary representation of the even number $2 * nat$ has the same number of 1s as *nat*; the odd number $2 * nat + 1$ has one 1 more. Hence, the sequence satisfies $ones = ones \curlyvee ones + 1$. Adding two initial values, we can turn the property into a definition.

$$\begin{array}{l} ones \ = 0 \prec ones' \\ ones' = 1 \prec ones' \curlyvee ones' + 1 \end{array}$$

It is important to note that $x = x \curlyvee x + 1$ does not have a unique solution. However, all solutions are of the form $ones + c$.

Alternatively, $ones'$ can be derived from the specification $ones' = map\ sum\ (bit +\!\!+ 1)$, where *sum* computes the sum of a list of numbers. The details are left to the reader.

Let us inspect the sequences.

$$\begin{array}{l} \gg\ msb \\ \langle 1,2,2,4,4,4,4,8,8,8,8,8,8,8,8,8,16,\ldots\rangle \\ \gg\ nat' - msb \\ \langle 0,0,1,0,1,2,3,0,1,2,3,4,5,6,7,0,\ldots\rangle \\ \gg\ brp \\ \langle 1,2,3,4,6,5,7,8,12,10,14,9,13,11,15,16,\ldots\rangle \\ \gg\ ones \\ \langle 0,1,1,2,1,2,2,3,1,2,2,3,2,3,3,4,\ldots\rangle \end{array}$$

The sequence $nat' - msb$ ($A053645$) exhibits a nice pattern; it describes the distance to the largest pot at most nat'.

3.2.3 Binary carry sequence

Here is a sequence that every computer scientist should know: the *binary carry sequence* or *ruler function* ($A007814$). The sequence gives the exponent of the largest pot dividing nat', that is, the number of leading zeros in the binary representation (least significant bit first). Put differently, it quantifies the running time of the binary increment. To specify the sequence we make use of the divisibility relation on numbers: $a \setminus b \Longleftrightarrow \exists n \in \mathbb{N} \,.\, a * n = b$. The stream *carry* is then specified[6]

$$\forall s \,.\quad carry \mathbin{\dot{\geqslant}} s \quad = \quad 2^s \setminus nat' \ , \tag{18}$$

Note that the variable s ranges over integer streams. Ordering and divisibility are lifted pointwise to streams — recall that the lifted variant of $\geqslant$ is denoted $\dot{\geqslant}$. The specification precisely captures the verbal description: read from left to right it implies $2^{carry} \setminus nat'$; read from right to left it formalises that *carry* is the largest such sequence. Now we calculate

$$\begin{array}{ll} & 2^s \setminus nat' \\ = & \quad \{\ nat' = 2 * nat + 1 \curlyvee 2 * nat' \text{ and } s = s_1 \curlyvee s_2\ \} \\ & 2^{s_1 \curlyvee s_2} \setminus (2 * nat + 1 \curlyvee 2 * nat') \\ = & \quad \{\ (10) \text{ and abide law } (11)\ \} \\ & (2^{s_1} \setminus (2 * nat + 1)) \curlyvee (2^{s_2} \setminus (2 * nat')) \\ = & \quad \{\ 2^a \setminus (2 * b + 1) \Longleftrightarrow 0 \geqslant a, \text{ and lifting}\ \} \\ & (0 \mathbin{\dot{\geqslant}} s_1) \curlyvee (2^{s_2} \setminus (2 * nat')) \\ = & \quad \{\ 2^a \setminus (2 * b) \Longleftrightarrow 2^{a-1} \setminus b, \text{ and lifting}\ \} \\ & (0 \mathbin{\dot{\geqslant}} s_1) \curlyvee (2^{s_2 - 1} \setminus nat') \\ = & \quad \{\ \text{specification } (18)\ \} \\ & (0 \mathbin{\dot{\geqslant}} s_1) \curlyvee (carry \mathbin{\dot{\geqslant}} s_2 - 1) \\ = & \quad \{\ \text{arithmetic}\ \} \end{array}$$

[6] The author is grateful to Roland Backhouse for suggesting this specification.

$$\begin{array}{ll} & (0 \mathbin{\dot{\geqslant}} s_1) \curlyvee (\mathit{carry}+1 \mathbin{\dot{\geqslant}} s_2) \\ = & \quad \{ \text{ abide law (11) and } s = s_1 \curlyvee s_2 \} \\ & (0 \curlyvee \mathit{carry}+1) \mathbin{\dot{\geqslant}} s \ . \end{array}$$

Since (18) uniquely determines *carry*, we have

$$\mathit{carry} = 0 \curlyvee \mathit{carry}+1 \ ,$$

which is, in fact, an executable program. (The form of the equation does not quite meet the requirements. We allow ourselves some liberty, as a simple unfolding turns it into an admissible form: $\mathit{carry} = 0 \prec \mathit{carry}+1 \curlyvee 0$. The unfolding works as long as the first argument of $\curlyvee$ is a sequence defined elsewhere.)

There is an intriguing connection to infinite binary trees. Consider the following definition.

$$\begin{array}{ll} \mathit{turn}\ 0 & = [\,] \\ \mathit{turn}\ (n+1) & = \mathit{turn}\ n +\!\!+ [n] +\!\!+ \mathit{turn}\ n \end{array}$$

The call *turn n* yields the heights of the nodes of a perfect binary tree of depth n. Stewart refers to this recursion pattern as the 'army method' with an $(n+1)$-star general delegating work to two n-star generals (1999). Now, imagine traversing an infinite binary tree starting at the leftmost leaf: visit the current node, visit its finite right subtree and then continue with its parent. The infinite tree is a so-called *sideways tree*: it has no root, but extends infinitely upwards. The following parametrised stream captures the traversal:

$$\mathit{tree}\ n = n \prec \mathit{turn}\ n \mathbin{\prec\!\!\!\prec} \mathit{tree}\ (n+1) \ ,$$

where $\prec\!\!\!\prec$ prepends a list to a sequence.

$$\begin{array}{l} (\prec\!\!\!\prec) \quad :: [\alpha] \to \mathit{Stream}\ \alpha \to \mathit{Stream}\ \alpha \\ \mathit{as} \prec\!\!\!\prec s = \mathit{foldr}\ (\prec)\ s\ \mathit{as} \end{array}$$

Here is the punch line: *tree* 0 also yields the binary carry sequence! The following table of binary numbers illustrates the correspondence.

$$\begin{array}{cccccccccccccccc} & & & & & & & & & & & & & & & 1 \\ & & & & & & & & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ & & & & & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ & & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \end{array}$$

For emphasis, prefixes of zeros are underlined. The lines correspond to the marks on a binary ruler; this is why *carry* is also called the ruler function. If we connect each 0-prefix of length n with the nearest 0-prefix of length $n+1$, we obtain the sideways tree.

Turning to the proof, let us try the obvious: we show that *tree* 0 satisfies the equation $x = 0 \curlyvee x+1$.

$$\begin{array}{ll} & 0 \curlyvee \mathit{tree}\ 0+1 \\ = & \quad \{ \text{ definition of } \curlyvee \} \\ & 0 \prec \mathit{tree}\ 0+1 \curlyvee 0 \end{array}$$

$$
\begin{array}{ll}
= & \quad \{ \text{ proof obligation, see below } \} \\
& 0 \prec \mathit{tree}\ 1 \\
= & \quad \{ \text{ definition of } \mathit{tree} \text{ and definition of } \mathit{turn} \ \} \\
& \mathit{tree}\ 0
\end{array}
$$

We are left with the proof obligation $\mathit{tree}\ 1 = \mathit{tree}\ 0 + 1 \curlyvee 0$. With some foresight, we generalise to $\mathit{tree}\ (k+1) = \mathit{tree}\ k + 1 \curlyvee 0$. The $\subset$-proof below makes essential use of the *mixed abide law*: if $\mathit{length}\ x = \mathit{length}\ y$, then

$$(x \prec\!\!\!\prec s) \curlyvee (y \prec\!\!\!\prec t) \quad = \quad (x \curlyvee y) \prec\!\!\!\prec (s \curlyvee t) \ , \tag{19}$$

where the operator in $x \curlyvee y$ denotes interleaving of two *lists* of the same length. Noting that $\mathit{length}\ (\mathit{turn}\ n) = 2^n - 1$, we reason

$$
\begin{array}{ll}
& \mathit{tree}\ (k+1) \\
= & \quad \{ \text{ definition of } \mathit{tree} \ \} \\
& k+1 \prec \mathit{turn}\ (k+1) \prec\!\!\!\prec \mathit{tree}\ (k+2) \\
\subset & \quad \{ \ x\, n = n+1 \prec \mathit{turn}\ (n+1) \prec\!\!\!\prec x\ (n+1) \text{ has a unique solution } \} \\
& k+1 \prec \mathit{turn}\ (k+1) \prec\!\!\!\prec (\mathit{tree}\ (k+1) + 1 \curlyvee 0) \\
= & \quad \{ \text{ proof obligation, see below } \} \\
& k+1 \prec (\mathit{replicate}\ 2^k\ 0 \curlyvee \mathit{turn}\ k + 1) \prec\!\!\!\prec (\mathit{tree}\ (k+1) + 1 \curlyvee 0) \\
= & \quad \{ \text{ definition of } \curlyvee \text{ and definition of } \prec\!\!\!\prec \ \} \\
& ((k+1 : \mathit{turn}\ k + 1) \curlyvee \mathit{replicate}\ 2^k\ 0) \prec\!\!\!\prec (\mathit{tree}\ (k+1) + 1 \curlyvee 0) \\
= & \quad \{ \text{ mixed abide law (19) } \} \\
& ((k+1 : \mathit{turn}\ k + 1) \prec\!\!\!\prec \mathit{tree}\ (k+1) + 1) \curlyvee (\mathit{replicate}\ 2^k\ 0 \prec\!\!\!\prec 0) \\
= & \quad \{ \text{ arithmetic and definition of } \prec\!\!\!\prec \ \} \\
& (k \prec \mathit{turn}\ k \prec\!\!\!\prec \mathit{tree}\ (k+1)) + 1 \curlyvee 0 \\
= & \quad \{ \text{ definition of } \mathit{tree} \ \} \\
& \mathit{tree}\ k + 1 \curlyvee 0
\end{array}
$$

It remains to show the finite version of the proof obligation: $\mathit{turn}\ (k+1) = \mathit{replicate}\ 2^k\ 0 \curlyvee \mathit{turn}\ k + 1$. We omit the straightforward induction, which relies on an abide law for lists.

3.2.4 Fractal sequences

The sequence *pot*, the 1s-counting sequence and the binary carry sequence are all examples of *fractal* or *self-similar* sequences: a subsequence is similar to the entire sequence. Another fractal sequence is *A*025480.

$$\mathit{frac} = \mathit{nat} \curlyvee \mathit{frac}$$

This sequence contains infinitely many copies of the natural numbers. The distance between equal numbers grows exponentially, 2^n, as we progress to the right. Like *carry*, *frac* is related to divisibility:

$$\mathit{god} = 2 * \mathit{frac} + 1$$

is the *greatest odd divisor* of nat'. Since we have $god = 2 * nat + 1 \curlyvee god$ and $nat' = 2 * nat + 1 \curlyvee 2 * nat'$, the property follows from a simple case analysis: the greatest odd divisor of an odd number, $2 * nat + 1$, is the number itself; the greatest odd divisor of an even number, $2 * nat'$, is the god of nat'.

Now, recall that 2^{carry} is the largest power of two dividing nat'. Putting these observations together, we have

$$2^{carry} * god \quad = \quad nat' \ .$$

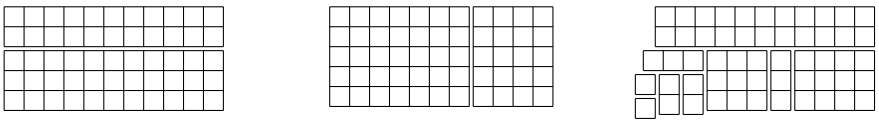
The proof is surprisingly straightforward.

$$\begin{array}{ll} & 2^{carry} * god \\ = & \quad \{ \text{ definition of } carry \text{ and definition of } god \ \} \\ & 2^{0 \curlyvee carry+1} * (2 * nat + 1 \curlyvee god) \\ = & \quad \{ \text{ arithmetic and abide law (11) } \} \\ & 2 * nat + 1 \curlyvee 2 * 2^{carry} * god \\ \subset & \quad \{ \ x = 2 * nat + 1 \curlyvee 2 * x \text{ has a unique solution } \} \\ & 2 * nat + 1 \curlyvee 2 * nat' \\ = & \quad \{ \ nat' = 2 * nat + 1 \curlyvee 2 * nat' \ \} \\ & nat' \end{array}$$

3.2.5 Sprague-Grundy numbers

Quadratisch. Praktisch. Gut.
Slogan of "Ritter Sport" (1970)

There is an intriguing connection between the sequence *frac* and the *chocolate game*. Two players participate in this game, eating up a chocolate bar according the following rule: the player whose turn it is breaks the bar in two pieces, either horizontally or vertically, and eats the smaller one — or the more appealing one in case of a tie.



The players make alternating moves; the game ends when the bar consists of a single square. The player whose turn it is loses and consequently has to pay for the chocolate bar.

Representing a game position by the two dimensions of the bar, a possible sequence of moves is, see picture on the right above:

$$(11,5) \mapsto (11,3) \mapsto (7,3) \mapsto (6,3) \mapsto (3,3) \mapsto (3,2) \mapsto (2,2) \mapsto (1,2) \mapsto (1,1) \ .$$

Since the number of moves is even, the first player loses. In fact, the first position, $(11,5)$, is a *losing position*: every move leads to a winning position. Conversely, from a winning position there is at least one move to a losing position.

How can we determine whether a position is a winning or a losing position? This is where the sequence *frac* enters the scene: (i, j) is a winning position iff $(frac)_i = (frac)_j$!

By this token, square bars — the famous Ritter Sport — are losing positions, but also $(7,3)$ and $(11,5)$ as $(frac)_7 = 0 = (frac)_3$ and $(frac)_{11} = 1 = (frac)_5$.

The chocolate game is an example of an *impartial two-person game*. For reasons of space, we can only sketch the necessary theory. For a more leisurely exposition we refer the interested reader to the treatment by Backhouse and Michaelis (2005). An impartial game is fully determined by a function $move :: Pos \to [Pos]$, where *Pos* is the type of game positions. The chocolate game can be viewed as a combination of two identical games, breaking a $1 \times n$ bar, whose individual *move* function is

$$move\ n = take\ (n\ \mathbf{div}\ 2)\ [n-1, n-2 \mathinner{..} 1]\ .$$

A position in the combined game is then a pair of positions. The combined game is known as a *sum game*, because a player can either make a move in the left or in the right component game. The central idea for determining winning and losing positions in a sum game is to assign a number to each component position, the so-called *Sprague-Grundy number*, such that (i, j) is a losing position iff $sg\ i = sg\ j$. The numbers are chosen so that every move from a losing position makes the numbers unequal, and for every winning position there is a move that makes them equal. One can show that the following definition of *sg* satisfies these requirements.

$$\begin{aligned} sg\ p &= mex\ \{sg\ q \mid q \leftarrow move\ p\} \\ mex\ x &= head\ \langle n \mid n \leftarrow nat, n \notin x \rangle \end{aligned}$$

Here, $mex\ x$ yields the *minimal excludent* of x, the least natural number that is not contained in the finite set x. The Sprague-Grundy number of p is then the minimal excludent of the Sprague-Grundy numbers of positions reachable from p.

The claim is that $frac'$ is the tabulation of *sg*:

$$map\ sg\ nat' = frac'\ .$$

(The positions only comprise the positive numbers; hence we have to take the tail of the two sequences.) This equation is not at all obvious, and, indeed, it takes some effort to prove it correct. The proof is, however, quite instructive, as it shows how to deal with course-of-value recursion and interleavings.

First observe that the executable specification of *sg* is not good for a program, as it is horribly inefficient: $sg\ p$ spawns $p\ \mathbf{div}\ 2$ recursive calls, each of which triggers another avalanche of calls. For instance, $sg\ 25$ generates a total of $8,986,539$ calls. The call structure of *sg* is an instance of *course-of-value recursion*. In this recursion scheme, the value of g_n is defined in terms of all previous values: $g_{n-1}, g_{n-2}, \ldots, g_0$. For tabulating functions defined by course-of-value recursion, it is useful to introduce a stream operator that maps a stream to the stream of its partial reverses:

$$\begin{aligned} course\ &:: Stream\ \alpha \to Stream\ [\alpha] \\ course\ s &= t\ \mathbf{where}\ t = [\,] \prec s : t\ , \end{aligned}$$

where : is lifted to streams. In what follows, we shall also lift the list processing functions *take*, *init*, *last*, *set* (which takes a list to a set), and the set operations $\{-\}$ (singleton set), $\cup$ (set union), and $\setminus$ (set difference) to streams.

The functioning of *course* is illustrated below.

$$\begin{array}{cccccccccccc}
t_0 & t_1 & t_2 & t_3 & t_4 & t_5 & t_6 & t_7 & t_8 & t_9 & t_{10} & \cdots \\
\| & \| & \| & \| & \| & \| & \| & \| & \| & \| & \| & \cdots \\
[\,] & s_0 & s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 & s_8 & s_9 & \cdots \\
 & : & : & : & : & : & : & : & : & : & : & \cdots \\
 & t_0 & t_1 & t_2 & t_3 & t_4 & t_5 & t_6 & t_7 & t_8 & t_9 & \cdots
\end{array}
\qquad
\begin{array}{ccc}
t & & \\
\| & & \\
[\,] & \prec & s \\
 & & : \\
 & & t
\end{array}$$

If f is the function that takes the list of previous values $[g_{n-1}, g_{n-2}, \ldots, g_0]$ to g_n, then *tabulate g* is the unique solution of

$$s = map\ f\ (course\ s)\ .$$

(The equation has a unique solution, as a simple unfolding of *course* and *map* turns it into the required syntactic form.) Again, we avoid index manipulations by introducing a suitable stream operator. In our example, f is essentially *mex* after *take*. Some straightforward calculations turn the recursive definition of *sg* into a recursion equation over streams: *sg*'s tabulation, *map sg nat′*, satisfies

$$x = map\ mex\ (set\ (take\ (nat'\ \mathbf{div}\ 2)\ (course\ x))).$$

Since the equation has a unique solution, we are left with the task of showing that *frac′* also satisfies the equation, which is what we tackle next.

In a succession of calculations we determine $r = course\ frac'$, $s = take\ (nat'\ \mathbf{div}\ 2)\ r$, $t = set\ s$, and finally $map\ mex\ t$. Observe that both $nat'\ \mathbf{div}\ 2$ and *frac′* involve interleavings: $nat'\ \mathbf{div}\ 2 = nat \curlyvee nat'$ and $frac' = frac \curlyvee nat'$. To deal effectively with interleavings, it is helpful to recall that the equation $x \curlyvee y = a \prec f\ x \curlyvee g\ y$ uniquely determines x and y, see Section 2.3. With a single equation we capture three sequences: the entire stream, the stream of elements at even positions and the stream of elements at odd positions.

Now, setting $re \curlyvee ro = course\ frac'$, we reason

$$\begin{array}{ll}
 & re \curlyvee ro \\
= & \quad \{\ \text{definition of } re \curlyvee ro\ \} \\
 & course\ frac' \\
= & \quad \{\ \text{definition of } course\ \} \\
 & [\,] \prec frac' : course\ frac' \\
= & \quad \{\ \text{definition of } frac' \text{ and definition of } re \curlyvee ro\ \} \\
 & [\,] \prec (frac \curlyvee nat') : (re \curlyvee ro) \\
= & \quad \{\ \text{abide law (11)}\ \} \\
 & [\,] \prec frac : re \curlyvee nat' : ro\ .
\end{array}$$

Next, let $se \curlyvee so = take\ (nat \curlyvee nat')\ (re \curlyvee ro)$. Using an analogous calculation we can show that $se \curlyvee so = [\,] \prec frac : se \curlyvee nat' : init\ so$. Finally, let $te \curlyvee to = set\ (se \curlyvee so)$, then

$$\begin{array}{ll}
 & te \curlyvee to \\
= & \quad \{\ \text{definition of } te \curlyvee to\ \} \\
 & set\ (se \curlyvee so)
\end{array}$$

$$
\begin{array}{ll}
= & \quad \{ \text{ see above } \} \\
& \mathit{set}\ ([\,] \prec \mathit{frac} : \mathit{se} \curlyvee \mathit{nat}' : \mathit{init}\ \mathit{so}) \\
= & \quad \{ \text{ definition of } \mathit{set}\text{: } \mathit{set}\ [\,] = \{\} \text{ and } \mathit{set}\ (a:x) = \{a\} \cup \mathit{set}\ x \ \} \\
& \{\} \prec \{\mathit{frac}\} \cup \mathit{set}\ \mathit{se} \curlyvee \{\mathit{nat}'\} \cup \mathit{set}\ (\mathit{init}\ \mathit{so}) \ .
\end{array}
$$

We are stuck. We have to express $\mathit{set}\ (\mathit{init}\ \mathit{so})$ in terms of $\mathit{set}\ \mathit{so}$. We can rewrite the former stream expression to $\mathit{set}\ \mathit{so} \setminus \{\mathit{last}\ \mathit{so}\}$, where $\setminus$ denotes set difference. However, to make further progress we need to know $\mathit{last}\ \mathit{so}$. So let us peek at some values.

$$
\begin{array}{l}
\gg\ \mathit{last}\ \mathit{so} \\
\langle 0,1,0,2,1,3,0,4,2,5,1,6,3,7,0,8,\ldots\rangle \\
\gg\ \mathit{frac}' \\
\langle 0,1,0,2,1,3,0,4,2,5,1,6,3,7,0,8,\ldots\rangle
\end{array}
$$

Somewhat surprisingly, $\mathit{last}\ \mathit{so}$ seems to be frac'! We leave this conjecture as a proof obligation and finish off ($\setminus$ binds more tightly than $\cup$).

$$
\begin{array}{ll}
= & \quad \{ \text{ proof obligation: } \mathit{last}\ \mathit{so} = \mathit{frac}' \ \} \\
& \{\} \prec \{\mathit{frac}\} \cup \mathit{set}\ \mathit{se} \curlyvee \{\mathit{nat}'\} \cup \mathit{set}\ \mathit{so} \setminus \{\mathit{frac}'\} \\
= & \quad \{ \text{ definition of } \mathit{te} \curlyvee \mathit{to} \text{ and abide law (11) } \} \\
& \{\} \prec \{\mathit{frac}\} \cup \mathit{te} \curlyvee \{\mathit{nat}'\} \cup \mathit{to} \setminus \{\mathit{frac}'\}
\end{array}
$$

We are nearly done. It is not hard to see that $\mathit{te} \curlyvee \mathit{to} = \{\} \prec \{\mathit{frac}\} \cup \mathit{te} \curlyvee \{\mathit{nat}'\} \cup \mathit{to} \setminus \{\mathit{frac}'\}$ has the unique solution $\{0\,..\,\mathit{nat}\} \setminus \{\mathit{frac}\} \curlyvee \{0\,..\,\mathit{nat}\}$. (Here we also lift the mixfix notation $\{a\,..\,b\}$ to streams.) Consequently,

$$
\begin{array}{ll}
& \mathit{map}\ \mathit{mex}\ (\mathit{te} \curlyvee \mathit{to}) \\
= & \quad \{ \text{ see above } \} \\
& \mathit{map}\ \mathit{mex}\ (\{0\,..\,\mathit{nat}\} \setminus \{\mathit{frac}\} \curlyvee \{0\,..\,\mathit{nat}\}) \\
= & \quad \{ \text{ abide law (11) } \} \\
& \mathit{map}\ \mathit{mex}\ (\{0\,..\,\mathit{nat}\} \setminus \{\mathit{frac}\}) \curlyvee \mathit{map}\ \mathit{mex}\ \{0\,..\,\mathit{nat}\} \\
= & \quad \{ \text{ definition of } \mathit{mex} \text{ and } (\mathit{frac} \mathbin{\dot{<}} \mathit{nat}') = \mathit{true} \ \} \\
& \mathit{frac} \curlyvee \mathit{nat}' \\
= & \quad \{ \text{ definition of } \mathit{frac}' \ \} \\
& \mathit{frac}' \ .
\end{array}
$$

Hurray! The second but last step depends on the crucial property that frac is smaller than nat' — this follows from the fact that $\mathit{god} = 2 * \mathit{frac} + 1$ is the greatest odd divisor of nat'.

We still have to discharge the proof obligation. Since the stream so is given by the equation $\mathit{so} = [0] \prec \mathit{frac}' : \mathit{nat}' : \mathit{init}\ \mathit{so}$, we have to show that $\mathit{last}\ \mathit{so} = \mathit{frac}'$. This is, in fact, an instance of a more general property: if x is given by $x = [\mathit{head}\ s] \prec \mathit{tail}\ s : t : \mathit{init}\ x$, then $\mathit{last}\ x = s \curlyvee t$. The mostly straightforward proof is left as an exercise to the reader.

3.2.6 Josephus problem

Our final example is a variant of the *Josephus problem* (Graham *et al.*, 1994). Imagine n people numbered 1 to n forming a circle. Every second person is killed until only one survives. Our task is to determine the survivor's number.

Now, if there is only one person, then this person survives. For an even number of persons the martial process starts as follows: 1 2 3 4 5 6 becomes 1 ~~2~~ 3 ~~4~~ 5 ~~6~~. Renumbering 1 3 5 to 1 2 3, we observe that if *nat* survives in the sequence of first-round survivors, then $2 * nat - 1$ survives in the original sequence. Likewise for odd numbers: 1 2 3 4 5 6 7 becomes ~~1~~ ~~2~~ 3 ~~4~~ 5 ~~6~~ 7 — since the number is odd, the first person is killed, as well. Renumbering 3 5 7 to 1 2 3, we observe that if *nat* survives in the remaining sequence, then $2 * nat + 1$ survives in the original sequence. The verbal description is captured by the stream equation below.

$$jos = 1 \prec 2 * jos - 1 \curlyvee 2 * jos + 1$$

The nth element of *jos* determines the survivor's number, when there are n persons initially. It is quite revealing to inspect the sequence.

$$\begin{array}{l} \gg\ jos \\ \langle 1,1,3,1,3,5,7,1,3,5,7,9,11,13,15,1,\ldots\rangle \\ \gg\ (jos-1)/2 \\ \langle 0,0,1,0,1,2,3,0,1,2,3,4,5,6,7,0,\ldots\rangle \end{array}$$

Since the even numbers are eliminated in the first round, *jos* only contains odd numbers. If we divide $jos - 1$ by two, we obtain a sequence we have encountered before: $nat' - msb$. Indeed, *jos* and *msb* are related by

$$jos \quad = \quad 2 * (nat' - msb) + 1 \ .$$

In terms of bit operations, *jos* implements a *cyclic left shift*: $nat' - msb$ removes the most significant bit, $2*$ shifts to the left and $+1$ sets the least significant bit.

$$\begin{array}{ll} & 2 * (nat' - msb) + 1 \\ = & \quad \{ \text{ definition of } msb \text{ and property of } nat' \ \} \\ & 2 * ((1 \prec 2 * nat' \curlyvee 2 * nat' + 1) - (1 \prec 2 * msb \curlyvee 2 * msb)) + 1 \\ = & \quad \{ \text{ abide law (11) } \} \\ & 2 * (0 \prec 2 * nat' - 2 * msb \curlyvee 2 * nat' + 1 - 2 * msb) + 1 \\ = & \quad \{ \text{ arithmetic } \} \\ & 1 \prec 2 * (2 * (nat' - msb) + 1) - 1 \curlyvee 2 * (2 * (nat' - msb) + 1) + 1 \end{array}$$

4 Finite calculus (Δ, Σ)

Let us move on to another application of streams: *finite calculus* (Graham *et al.*, 1994). Finite calculus is the discrete counterpart of infinite calculus, where finite difference replaces the derivative and summation replaces integration. We shall see that difference and summation can be easily recast as stream operators.

4.1 Finite difference

A common type of puzzle asks the reader to continue a given sequence of numbers. A first routine step towards solving the puzzle is to calculate the difference of subsequent elements. This stream operator, *finite difference* or *forward difference*, enjoys a simple, non-recursive definition.

$$\begin{aligned}&\Delta \quad :: (Num\ \alpha) \Rightarrow Stream\ \alpha \to Stream\ \alpha\\&\Delta\ s = tail\ s - s\end{aligned}$$

Here are some examples (*A*003215, *A*000079, *A*094267, not listed).

$$\begin{aligned}&\gg\ \Delta\ nat^3\\&\langle 1,7,19,37,61,91,127,169,217,271,331,397,469,547,\ldots\rangle\\&\gg\ \Delta\ 2^{nat}\\&\langle 1,2,4,8,16,32,64,128,256,512,1024,2048,4096,8192,\ldots\rangle\\&\gg\ \Delta\ carry\\&\langle 1,-1,2,-2,1,-1,3,-3,1,-1,2,-2,1,-1,4,-4,\ldots\rangle\\&\gg\ \Delta\ jos\\&\langle 0,2,-2,2,2,2,-6,2,2,2,2,2,2,2,-14,2,\ldots\rangle\end{aligned}$$

The finite difference of *carry* exhibits an interesting pattern, so let us calculate the stream as a warm-up exercise.

$$\begin{aligned}&\Delta\ carry\\=&\quad \{\ \text{definition of } \Delta\ \}\\&tail\ carry - carry\\=&\quad \{\ \text{definition of carry}\ \}\\&(carry + 1\ \curlyvee\ 0) - (0\ \curlyvee\ carry + 1)\\=&\quad \{\ \text{abide law (11)}\ \}\\&(carry + 1)\ \curlyvee\ -(carry + 1)\end{aligned}$$

Infinite calculus has a simple rule for the derivative of a power: $x^{n+1}\frac{d}{dx} = (n+1)x^n$. Unfortunately, the first example above shows that finite difference does not interact nicely with ordinary powers: $\Delta\ nat^3$ is not $3 * nat^2$. Can we find a different notion that enjoys an analogous rule? Let us try. Writing $x^{\underline{n}}$ for the new power and its lifted variant, we calculate

$$\begin{aligned}&\Delta\ (nat^{\underline{n+1}})\\=&\quad \{\ \text{definition of } \Delta\ \}\\&tail\ (nat^{\underline{n+1}}) - nat^{\underline{n+1}}\\=&\quad \{\ \text{definition of } nat\ \}\\&(nat + 1)^{\underline{n+1}} - nat^{\underline{n+1}}\end{aligned}\qquad .$$

Starting at the other end, we obtain

$$\begin{aligned}&(n+1) * nat^{\underline{n}}\\=&\quad \{\ \text{arithmetic}\ \}\\&nat^{\underline{n}} + n * nat^{\underline{n}}\end{aligned}$$

Table 2. *Converting between powers and falling factorial powers.*

x^0	$=$	$x^{\underline{0}}$	$x^{\underline{0}}$	$=$	x^0
x^1	$=$	$x^{\underline{1}}$	$x^{\underline{1}}$	$=$	x^1
x^2	$=$	$x^{\underline{2}}+x^{\underline{1}}$	$x^{\underline{2}}$	$=$	x^2-x^1
x^3	$=$	$x^{\underline{3}}+3*x^{\underline{2}}+x^{\underline{1}}$	$x^{\underline{3}}$	$=$	$x^3-3*x^2+2*x^1$
x^4	$=$	$x^{\underline{4}}+6*x^{\underline{3}}+7*x^{\underline{2}}+x^{\underline{1}}$	$x^{\underline{4}}$	$=$	$x^4-6*x^3+11*x^2-6*x^1$

Table 3. *Laws for finite difference (c, k and n are constant streams).*

$\Delta\,(\mathit{tail}\,s)$	$=$	$\mathit{tail}\,(\Delta\,s)$	$\Delta\,(s+t)$	$=$	$\Delta\,s+\Delta\,t$
$\Delta\,(a\prec s)$	$=$	$\mathit{head}\,s-a\prec\Delta\,s$	$\Delta\,(s*t)$	$=$	$s*\Delta\,t+\Delta\,s*\mathit{tail}\,t$
$\Delta\,(s\curlyvee t)$	$=$	$(t-s)\curlyvee(\mathit{tail}\,s-t)$	$\Delta\,c^{\mathit{nat}}$	$=$	$(c-1)*c^{\mathit{nat}}$
$\Delta\,n$	$=$	0	$\Delta\,(\mathit{nat}^{\underline{n+1}})$	$=$	$(n+1)*\mathit{nat}^{\underline{n}}$
$\Delta\,(n*s)$	$=$	$n*\Delta\,s$	$\Delta\binom{\mathit{nat}}{k+1}$	$=$	$\binom{\mathit{nat}}{k}$

$$
\begin{aligned}
&= \quad \{\text{ introduce } \mathit{nat}*\mathit{nat}^{\underline{n}}\ \}\\
&\quad (\mathit{nat}+1)*\mathit{nat}^{\underline{n}}-\mathit{nat}^{\underline{n}}*(\mathit{nat}-n)
\end{aligned}
$$

We can connect the loose ends, if the new power satisfies both $x^{\underline{n+1}} = x*(x-1)^{\underline{n}}$ and $x^{\underline{n+1}} = x^{\underline{n}}*(x-n)$. That is easy to arrange, we use the first equation as a definition. (It is not hard to see that the definition then also satisfies the second equation.)

$$
\begin{aligned}
x^{\underline{0}} &= 1\\
x^{\underline{n+1}} &= x*(x-1)^{\underline{n}}
\end{aligned}
$$

The new powers are, of course, well-known: they are called *falling factorial powers* (Graham *et al.*, 1994). The following session shows that they behave as expected.

$\gg\ \mathit{nat}^{\underline{3}}$
$\langle 0,0,0,6,24,60,120,210,336,504,720,990,1320,1716,2184,2730,\ldots\rangle$
$\gg\ \Delta\,(\mathit{nat}^{\underline{3}})$
$\langle 0,0,6,18,36,60,90,126,168,216,270,330,396,468,546,630,\ldots\rangle$
$\gg\ 3*\mathit{nat}^{\underline{2}}$
$\langle 0,0,6,18,36,60,90,126,168,216,270,330,396,468,546,630,\ldots\rangle$

One can convert mechanically between powers and falling factorial powers using Stirling numbers (Graham *et al.*, 1994). The details are beyond the scope of this paper. For reference, Table 2 displays the correspondence up to the fourth power.

4.1.1 Laws

Table 3 lists the rules for finite differences. First of all, Δ is a *linear operator*: it distributes over sums. The stream 2^{nat} is the discrete analogue of e^x as $\Delta\,2^{\mathit{nat}} = 2^{\mathit{nat}}$. In general,

$$\Delta\,c^{\mathit{nat}}$$

$$
\begin{array}{ll}
= & \quad \{ \text{ definition of } \Delta \} \\
& \mathit{tail}\ (c^{\mathit{nat}}) - c^{\mathit{nat}} \\
= & \quad \{ \ c \text{ is constant and definition of } \mathit{nat} \ \} \\
& c^{\mathit{nat}+1} - c^{\mathit{nat}} \\
= & \quad \{ \text{ arithmetic } \} \\
& (c-1) * c^{\mathit{nat}} \quad .
\end{array}
$$

The product rule is similar to the product rule of infinite calculus except for an occurrence of *tail* on the right-hand side.

$$
\begin{array}{ll}
& \Delta\,(s * t) \\
= & \quad \{ \text{ definition of } \Delta \text{ and definition of } * \} \\
& \mathit{tail}\ s * \mathit{tail}\ t - s * t \\
= & \quad \{ \text{ arithmetic } \} \\
& s * \mathit{tail}\ t - s * t + \mathit{tail}\ s * \mathit{tail}\ t - s * \mathit{tail}\ t \\
= & \quad \{ \text{ distributivity } \} \\
& s * (\mathit{tail}\ t - t) + (\mathit{tail}\ s - s) * \mathit{tail}\ t \\
= & \quad \{ \text{ definition of } \Delta \} \\
& s * \Delta\, t + \Delta\, s * \mathit{tail}\ t
\end{array}
$$

We can avoid the unpleasant asymmetry of the law, if we further massage $\Delta\, s * \mathit{tail}\, t$.

$$
\begin{array}{ll}
= & \quad \{ \text{ definition of } \Delta \} \\
& s * \Delta\, t + \Delta\, s * (t + \Delta\, t) \\
= & \quad \{ \text{ arithmetic } \} \\
& s * \Delta\, t + \Delta\, s * t + \Delta\, s * \Delta\, t
\end{array}
$$

The rule for binomial coefficients is a special case of the rule for falling factorial powers as $\binom{n}{k} = n^{\underline{k}} / k!$.

4.1.2 Example: Josephus problem, continued

Let us get back to the Josephus problem: the interactive session in Section 4.1 suggests that $\Delta\, \mathit{jos}$ is almost always 2, except for pots, the powers of two. We can express this property using a *stream conditional*:

$$\Delta\, \mathit{jos} = (\mathit{pot}' \rightarrow -\mathit{nat}\,; 2) \quad ,$$

where $(_ \rightarrow _\,; _)$ is **if** _ **then** _ **else** _ lifted to streams. The stream conditional enjoys the standard laws, such as $(\mathit{false} \rightarrow s\,; t) = t$, and a ternary version of the abide law.

$$((s_1 \curlyvee s_2) \rightarrow (t_1 \curlyvee t_2)\,; (u_1 \curlyvee u_2)) \quad = \quad (s_1 \rightarrow t_1\,; u_1) \curlyvee (s_2 \rightarrow t_2\,; u_2) \qquad (20)$$

Both laws are used in the proof of the property above.

$$\Delta\, \mathit{jos}$$

$$
\begin{array}{ll}
= & \quad \{\ \Delta \text{ law and arithmetic } \} \\
& 0 \prec 2 \curlyvee 2 * (\mathit{tail}\ \mathit{jos} - \mathit{jos}) - 2 \\
= & \quad \{\ \text{definition of } \Delta\ \} \\
& 0 \prec 2 \curlyvee 2 * \Delta\, \mathit{jos} - 2 \\
\subset & \quad \{\ x = 0 \prec 2 \curlyvee 2 * x - 2 \text{ has a unique solution } \} \\
& 0 \prec 2 \curlyvee 2 * (\mathit{pot}' \rightarrow -\mathit{nat}\,;2) - 2 \\
= & \quad \{\ \text{arithmetic and definition of } \mathit{nat}'\ \} \\
& 0 \prec 2 \curlyvee (\mathit{pot}' \rightarrow -(2 * \mathit{nat}')\,;2) \\
= & \quad \{\ \text{definition of } \mathit{nat}, \text{ definition of } \mathit{pot} \text{ and definition of } \curlyvee\ \} \\
& (\mathit{pot} \rightarrow -(2 * \mathit{nat})\,;2) \curlyvee 2 \\
= & \quad \{\ \text{conditional and abide law (20)}\ \} \\
& ((\mathit{pot} \curlyvee \mathit{false}) \rightarrow (-(2 * \mathit{nat} \curlyvee 2 * \mathit{nat} + 1))\,;(2 \curlyvee 2)) \\
= & \quad \{\ \text{definition of } \mathit{pot}' \text{ and characterisation of } \mathit{nat}\ \} \\
& (\mathit{pot}' \rightarrow -\mathit{nat}\,;2)
\end{array}
$$

The formula suggests a simple strategy to survive a shootout of increasing size: If the circle is extended by one person, move two positions clockwise, unless the total number is a power of two, in which case move to the first position.

4.1.3 Higher-order differences

An interesting pattern emerges, when we repeatedly apply the difference operator to a stream. A few quick calculations give the following formulas for higher-order differences (recall that $f^{(n)}$ is n-fold composition).

$$
\begin{array}{lcl}
\Delta^{(0)}\, s & = & \mathit{tail}^{(0)}\, s \\
\Delta^{(1)}\, s & = & \mathit{tail}^{(1)}\, s - \mathit{tail}^{(0)}\, s \\
\Delta^{(2)}\, s & = & \mathit{tail}^{(2)}\, s - 2 * \mathit{tail}^{(1)}\, s + \mathit{tail}^{(0)}\, s \\
\Delta^{(3)}\, s & = & \mathit{tail}^{(3)}\, s - 3 * \mathit{tail}^{(2)}\, s + 3 * \mathit{tail}^{(1)}\, s - \mathit{tail}^{(0)}\, s
\end{array}
$$

The factors of the tails appear to be binomial coefficients with alternating signs. To reveal the connection, let us rewrite Δ in a point-free style.

$$\Delta = \mathit{tail} - \mathit{id}$$

Subtraction is now doubly lifted: first to functions and then to streams, both of which are idioms. Indeed, if we provide a *Num* instance for the environment idiom[7], then the definition above is even legal Haskell!

Under this view the formulas for higher-order differences are simply instances of the Binomial Theorem lifted to stream operators.

$$(\mathit{tail} - \mathit{id})^{(n)} \quad = \quad \sum_k \mathit{repeat}\ \tbinom{n}{k} * (\mathit{tail}^{(k)} \cdot (-\mathit{id})^{(n-k)}) \tag{21}$$

[7] In principle, every idiom can be made an instance of *Num*, see Section 2.1.

(The unbounded sum on the right is really finite, as all terms are zero except those with $0 \leqslant k \leqslant n$.) The Binomial Theorem is applicable, because *tail* and $-id$ are linear operators and because they commute with each other. To see where these two conditions originate, let us prove the higher-order version of the Binomial Theorem for $n = 2$.

$$\begin{array}{ll} & (f+g) \cdot (f+g) \\ = & \quad \{ \cdot \text{ right-distributes over } + \} \\ & f \cdot (f+g) + g \cdot (f+g) \\ = & \quad \{ f \text{ and } g \text{ are linear} \} \\ & f \cdot f + f \cdot g + g \cdot f + g \cdot g \\ = & \quad \{ f \cdot g = g \cdot f \} \\ & f \cdot f + 2 * (f \cdot g) + g \cdot g \end{array}$$

4.2 Binomial transforms

Here is a little puzzle: continue the sequence $u = \langle 0,1,4,12,32,80,192,448,\ldots \rangle$. Play a bit with the numbers, before you proceed.

As noted before, a first routine step is to look at the higher-order differences. For instance, if $\Delta^{(n)}\ s$ is constant for some n, then the unknown stream s is a polynomial.

$\gg\ \Delta^{(0)}\ u$
$\langle 0,1,4,12,32,80,192,448,1024,2304,5120,11264,24576,53248,114688,245760,\ldots \rangle$
$\gg\ \Delta^{(1)}\ u$
$\langle 1,3,8,20,48,112,256,576,1280,2816,6144,13312,28672,61440,131072,\ldots \rangle$
$\gg\ \Delta^{(2)}\ u$
$\langle 2,5,12,28,64,144,320,704,1536,3328,7168,15360,32768,69632,147456,\ldots \rangle$
$\gg\ \Delta^{(3)}\ u$
$\langle 3,7,16,36,80,176,384,832,1792,3840,8192,17408,36864,77824,163840,\ldots \rangle$

Unfortunately, the higher-order differences do not become smaller. However, their heads form a familiar sequence: $\mathscr{B}^{(-1)}\ u = nat$ where

$$\begin{array}{l} \mathscr{B}^{(-1)} :: (Num\ \alpha) \Rightarrow Stream\ \alpha \to Stream\ \alpha \\ \mathscr{B}^{(-1)} = unfold\ head\ \Delta\ . \end{array}$$

If we view the stream of higher-order differences *iterate* Δ as a matrix, then $\mathscr{B}^{(-1)}$ maps the first row to the first column. As a first step towards solving the puzzle, let us derive the inverse transformation, which maps the first column to the first row, characterised as follows.

$$\begin{array}{lcl} head \cdot \mathscr{B} & = & head \\ \Delta \cdot \mathscr{B} & = & \mathscr{B} \cdot tail \end{array}$$

The first row and the first column share the left, upper corner; transforming the tail of the first column yields the second row, which is the difference of the first row. This data uniquely determines $\mathscr{B}$ as a quick calculation shows.

$$\mathscr{B}\ (tail\ s) = \Delta\ (\mathscr{B}\ s)$$

$$\begin{array}{ll} \iff & \{ \text{ definition of } \Delta \ \} \\ & \mathscr{B}\ (\mathit{tail}\ s) = \mathit{tail}\ (\mathscr{B}\ s) - \mathscr{B}\ s \\ \iff & \{ \text{ arithmetic } \} \\ & \mathit{tail}\ (\mathscr{B}\ s) = \mathscr{B}\ s + \mathscr{B}\ (\mathit{tail}\ s) \end{array}$$

Factoring out $\mathscr{B}\ s$ for efficiency, we obtain the following definition of $\mathscr{B}$.

$$\begin{array}{l} \mathscr{B} \quad :: (\mathit{Num}\ \alpha) \Rightarrow \mathit{Stream}\ \alpha \to \mathit{Stream}\ \alpha \\ \mathscr{B}\ s = t \ \textbf{where}\ t = \mathit{head}\ s \prec t + \mathscr{B}\ (\mathit{tail}\ s) \end{array}$$

The stream operator $\mathscr{B}$ is known as the *binomial transform*[8], as it sends $\langle s_0, s_1, s_2, s_3, \ldots \rangle$ to $\langle s_0, s_0 + s_1, s_0 + 2 * s_1 + s_2, s_0 + 3 * s_1 + 3 * s_2 + s_3, \ldots \rangle$. In general, we have

$$(\mathscr{B}^{(-1)}\ s)_n \quad = \quad \sum_k \tbinom{n}{k} * (s)_k * (-1)^{n-k} \qquad \text{and} \qquad (\mathscr{B}\ s)_n \quad = \quad \sum_k \tbinom{n}{k} * (s)_k \ . \tag{22}$$

The first formula is simply the pointwise version of (21). The characterisation of $\mathscr{B}$ implies that geometric progressions have a particularly simple binomial transform:

$$\mathscr{B}\ c^{nat} \quad = \quad (c+1)^{nat} \ ,$$

which is, in fact, a special case of the Binomial Theorem. The property can also be shown using the fact that the binomial transform is a linear transformation. (The proofs are left as exercises to the reader.)

$$\begin{array}{lll} \mathscr{B}\ (n * s) & = & n * \mathscr{B}\ s \\ \mathscr{B}\ (s + t) & = & \mathscr{B}\ s + \mathscr{B}\ t \end{array}$$

We are finally in a position to tackle the puzzle.

$$\begin{array}{ll} & \mathscr{B}\ \mathit{nat} \\ = & \quad \{ \text{ definition of } \mathscr{B} \text{ and definition of } \mathit{nat} \ \} \\ & 0 \prec \mathscr{B}\ \mathit{nat} + \mathscr{B}\ (\mathit{nat} + 1) \\ = & \quad \{ \ \mathscr{B} \text{ is linear } \} \\ & 0 \prec 2 * \mathscr{B}\ \mathit{nat} + \mathscr{B}\ 1 \\ = & \quad \{ \ \mathscr{B}\ 1 = \mathscr{B}\ (1^{nat}) = 2^{nat} \ \} \\ & 0 \prec 2 * \mathscr{B}\ \mathit{nat} + 2^{nat} \end{array}$$

We have to solve the recursion equation $x = 0 \prec 2 * x + 2^{nat}$. We shall later show how to do this systematically. For now, we just check that $\mathit{nat} * 2^{nat-1}$ is the solution of the equation and hence of the puzzle.

$$\begin{array}{ll} & \mathit{nat} * 2^{nat-1} \\ = & \quad \{ \text{ definition of } * \text{ and definition of } \mathit{nat} \ \} \\ & 0 \prec (\mathit{nat} + 1) * 2^{nat} \end{array}$$

[8] There is another transformation that goes under the same name: Knuth (1998) defines a variant where the tail is the *difference* of $\mathscr{B}\ s$ and $\mathscr{B}\ (\mathit{tail}\ s)$, rather than the sum.

$$\begin{aligned} &= \quad \{ \text{ arithmetic } \} \\ &\quad 0 \prec 2 * (nat * 2^{nat-1}) + 2^{nat} \end{aligned}$$

4.3 Newton series

The binomial transform $\mathscr{B}^{(-1)}$ is essentially the finite difference applied repeatedly. What happens if we in turn apply the binomial transform repeatedly? This sounds a bit scary, but it turns out that the operator is very well behaved. We consider its inverse first.

$$\begin{aligned} &\mathscr{B}\ (\mathscr{B}\ s) \\ = &\quad \{ \text{ definition of } \mathscr{B}, \text{ twice } \} \\ &\mathit{head}\ s \prec \mathscr{B}\ (\mathscr{B}\ s) + \mathscr{B}\ (\mathscr{B}\ s + \mathscr{B}\ (\mathit{tail}\ s)) \\ = &\quad \{ \mathscr{B} \text{ is linear } \} \\ &\mathit{head}\ s \prec \mathscr{B}\ (\mathscr{B}\ s) + \mathscr{B}\ (\mathscr{B}\ s) + \mathscr{B}\ (\mathscr{B}\ (\mathit{tail}\ s)) \\ = &\quad \{ \text{ arithmetic } \} \\ &\mathit{head}\ s \prec 2 * \mathscr{B}\ (\mathscr{B}\ s) + \mathscr{B}\ (\mathscr{B}\ (\mathit{tail}\ s)) \end{aligned}$$

The resulting recursion equation is identical to the definition of $\mathscr{B}$, except for an additional factor of two. If we apply $\mathscr{B}$ a third time, the factor increases to three. Abstracting away from the constant, we define a parametric version of the binomial transform.

$$\begin{aligned} &\mathscr{B} \quad :: (\mathit{Num}\ \alpha) \Rightarrow \alpha \to \mathit{Stream}\ \alpha \to \mathit{Stream}\ \alpha \\ &\mathscr{B}_n\ s = t \ \textbf{where}\ t = \mathit{head}\ s \prec \mathit{repeat}\ n * t + \mathscr{B}_n\ (\mathit{tail}\ s) \end{aligned}$$

The derivation makes clear that $\mathscr{B}_n$ is the n-fold composition of the binomial transform: $\mathscr{B}_n = \mathscr{B}^{(n)}$. Consequently, we have $\mathscr{B}_0 = \mathit{id}$ and $\mathscr{B}_m \cdot \mathscr{B}_n = \mathscr{B}_{m+n}$. The correspondence can even be extended to the integers, in particular, $\mathscr{B}_{-1} = \mathscr{B}^{(-1)}$. The unique fixed-point proof of this fact is too good to be missed: the calculation below shows that $\mathscr{B}_{-1}\ s$ satisfies the recursion equation of $\mathscr{B}^{(-1)} = \mathit{unfold}\ \mathit{head}\ \Delta$.

$$\begin{aligned} &\mathscr{B}_{-1}\ s \\ = &\quad \{ \text{ definition of } \mathscr{B}_{-1} \} \\ &\mathit{head}\ s \prec -\mathscr{B}_{-1}\ s + \mathscr{B}_{-1}\ (\mathit{tail}\ s) \\ = &\quad \{ \mathscr{B}_{-1} \text{ is linear and definition of } \Delta \} \\ &\mathit{head}\ s \prec \mathscr{B}_{-1}\ (\Delta\ s) \end{aligned}$$

This in turn implies that $\mathscr{B}$ and $\mathscr{B}^{(-1)}$ are mutually inverse.

In the introduction to Section 4.2 we remarked that if the unknown sequence is a polynomial, it can be easily identified: the higher-order differences eventually become constant. But can we reconstruct the polynomial from the data obtained? Let us tackle a concrete example (p is some unknown polynomial).

$$\begin{aligned} &\gg \ \mathscr{B}^{(-1)}\ p \\ &\langle 0, 1, 14, 36, 24, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, \ldots \rangle \end{aligned}$$

Since every element following $(\mathscr{B}^{(-1)}\ p)_4$ is zero, we know that the unknown sequence p is a polynomial of degree 4. But which one? To answer the question, let us put together

what we have found out so far.

$$
\begin{array}{ll}
 & s \\
= & \quad \{\ \mathscr{B} \cdot \mathscr{B}^{(-1)} = id\ \} \\
 & \mathscr{B}\ (\mathscr{B}^{(-1)}\ s) \\
= & \quad \{\ \text{extensionality (16)}\ \} \\
 & \langle (\mathscr{B}\ (\mathscr{B}^{(-1)}\ s))_n \mid n \leftarrow nat \rangle \\
= & \quad \{\ \text{pointwise characterisation of } \mathscr{B}\ (22)\ \} \\
 & \langle \sum_k \binom{n}{k} * (\mathscr{B}^{(-1)}\ s)_k \mid n \leftarrow nat \rangle \\
= & \quad \{\ \text{idioms, see below}\ \} \\
 & \sum_k \binom{nat}{k} * repeat\ (\mathscr{B}^{(-1)}\ s)_k \\
= & \quad \{\ \text{definition of binomial coefficients: } \binom{n}{k} = n^{\underline{k}} / k!\ \} \\
 & \sum_k repeat\ (\mathscr{B}^{(-1)}\ s / fac)_k * nat^{\underline{k}}
\end{array}
$$

(The second but last step is a bit daring, since we turn an infinite sum of elements into an infinite sum of streams. However, infinite summation can be made precise (Rutten, 2005), so let us not worry about this.) The calculation shows that if we divide the sequence $\mathscr{B}^{(-1)}\ s$ by *fac*, then we obtain the coefficients of a polynomial of falling factorial powers.

$$
\begin{array}{l}
\gg\ \ \mathscr{B}^{(-1)}\ p\ /\ fac \\
\langle 0,1,7,6,1,0,0,0,0,0,0,0,0,0,0,0,\ldots \rangle
\end{array}
$$

Table 2 then tells us that $p = nat^{\underline{1}} + 7 * nat^{\underline{2}} + 6 * nat^{\underline{3}} + nat^{\underline{4}} = nat^4$.

As an aside, the sum of binomial coefficients in the second but last line of the derivation is called the *Newton series* of s, which is the finite calculus counterpart of infinite calculus' Taylor series. So, in a nutshell, $\mathscr{B}^{(-1)}$ maps a sequence to the sequence of coefficients of its Newton series.

4.4 Summation

Finite difference Δ has a right-inverse: the *anti-difference* or *summation* operator Σ. We can easily derive its definition.

$$
\begin{array}{ll}
 & \Delta\ (\Sigma\ s) = s \\
\Longleftrightarrow & \quad \{\ \text{definition of } \Delta\ \} \\
 & tail\ (\Sigma\ s) - \Sigma\ s = s \\
\Longleftrightarrow & \quad \{\ \text{arithmetic}\ \} \\
 & tail\ (\Sigma\ s) = s + \Sigma\ s
\end{array}
$$

Setting $head\ (\Sigma\ s) = 0$, we obtain

$$
\begin{array}{l}
\Sigma\ \ :: (Num\ \alpha) \Rightarrow Stream\ \alpha \rightarrow Stream\ \alpha \\
\Sigma\ s = t\ \textbf{where}\ t = 0 \prec s + t\ \ .
\end{array}
$$

We have additionally applied λ-dropping (Danvy, 1999), turning the higher-order equation for Σ into a first-order equation for $\Sigma\, s$ with s fixed. The firstification of the definition enables sharing of computations as illustrated below.

$$\begin{array}{cccccccccccc}
t_0 & t_1 & t_2 & t_3 & t_4 & t_5 & t_6 & t_7 & t_8 & t_9 & t_{10} & \cdots \\
\| & \| & \| & \| & \| & \| & \| & \| & \| & \| & \| & \cdots \\
0 & s_0 & s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 & s_8 & s_9 & \cdots \\
 & + & + & + & + & + & + & + & + & + & + & \cdots \\
 & t_0 & t_1 & t_2 & t_3 & t_4 & t_5 & t_6 & t_7 & t_8 & t_9 & \cdots
\end{array}
\qquad
\begin{array}{ccc}
t & & \\
\| & & \\
0 & \prec & s \\
 & & + \\
 & & t
\end{array}$$

The illustration also makes clear that $(\Sigma\, s)_n$ is the sum of the first $n-1$ elements of s, excluding $(s)_n$. There is also a variant of Σ that includes $(s)_n$.

$$\begin{aligned}
&\Sigma' \;::\; (\mathit{Num}\;\alpha) \Rightarrow \mathit{Stream}\;\alpha \to \mathit{Stream}\;\alpha \\
&\Sigma'\; s = t \;\mathbf{where}\; t = \mathit{head}\; s \prec \mathit{tail}\; s + t
\end{aligned}$$

The two variants are related by $\Sigma\, s = 0 \prec \Sigma'\, s$ and $\Sigma'\, s = \mathit{tail}\;(\Sigma\, s)$. Usually, Σ is preferable over Σ', as it has more attractive calculational properties. The following pointwise characterisations relate the stream operators to summation.

$$(\Sigma\, s)_n \;=\; \sum_i (s)_i * [0 \leqslant i < n] \qquad \text{and} \qquad (\Sigma'\, s)_n \;=\; \sum_i (s)_i * [0 \leqslant i \leqslant n]$$

The formulas on the right-hand sides make use of *Iverson's convention*: the square brackets[9] turn a Boolean value into a number, with $[\mathit{false}] = 0$ and $[\mathit{true}] = 1$.

Here are some example summations (*A*004520, *A*000290, *A*011371, *A*002275).

$$\begin{aligned}
&\gg\; \Sigma\,(0 \curlyvee 1) \\
&\langle 0,0,1,1,2,2,3,3,4,4,5,5,6,6,7,7,\ldots\rangle \\
&\gg\; \Sigma\,(2 * \mathit{nat} + 1) \\
&\langle 0,1,4,9,16,25,36,49,64,81,100,121,144,169,196,225,\ldots\rangle \\
&\gg\; \Sigma\, \mathit{carry} \\
&\langle 0,0,1,1,3,3,4,4,7,7,8,8,10,10,11,11,\ldots\rangle \\
&\gg\; \Sigma\, 10^{\mathit{nat}} \\
&\langle 0,1,11,111,1111,11111,111111,1111111,11111111,111111111,1111111111,\ldots\rangle
\end{aligned}$$

The definition of Σ suggests an unusual approach for determining the sum of a sequence: if we observe that a stream satisfies $t = 0 \prec s + t$, then we may conclude that $\Sigma\, s = t$. For example, $\Sigma\, 1 = \mathit{nat}$ as $\mathit{nat} = 0 \prec \mathit{nat} + 1$, $\Sigma\,(2 * \mathit{nat} + 1) = \mathit{nat}^2$ as $\mathit{nat}^2 = 0 \prec \mathit{nat}^2 + 2 * \mathit{nat} + 1$, and $\Sigma\,(1 \prec \mathit{fib}) = \mathit{fib}$ as $\mathit{fib} = 0 \prec (1 \prec \mathit{fib}) + \mathit{fib}$. This is *summation by happenstance*. The latter example also shows that $1 \prec \mathit{fib}$ solves the *difference equation* $x = 1 \prec \Sigma\, x$.

Of course, if we already know the sum, we can use the definition of Σ to verify our conjecture. As an example, let us prove $\Sigma\, \mathit{fib}'^2 = \mathit{fib} * \mathit{fib}'$.

$$\begin{array}{ll}
 & \mathit{fib} * \mathit{fib}' \\
= & \quad \{\ \text{definition of } \mathit{fib} \text{ and definition of } \mathit{fib}'\ \}
\end{array}$$

[9] Unfortunately, Iverson's convention clashes with Haskell's notation for singleton lists. Both notations are used in this paper, but not simultaneously in the same section.

$$
\begin{aligned}
& (0 \prec \mathit{fib}') * (1 \prec \mathit{fib} + \mathit{fib}') \\
= \quad & \{ \text{ arithmetic } \} \\
& 0 \prec \mathit{fib}'^2 + \mathit{fib} * \mathit{fib}'
\end{aligned}
$$

The unique fixed-point proof avoids the inelegant case analysis of a traditional inductive proof.

4.4.1 Laws

The *Fundamental Theorem of finite calculus* relates Δ and Σ.

$$\Sigma\, t = s - \mathit{repeat}\ (\mathit{head}\ s) \iff t = \Delta\, s \tag{23}$$

The implication from right to left is easy to show using $\Delta\,(\Sigma\, t) = t$ and $\Delta\, c = 0$. For the reverse direction, we reason

$$
\begin{aligned}
& \Sigma\,(\Delta\, s) \\
= \quad & \{ \text{ definition of } \Sigma \ \} \\
& 0 \prec \Sigma\,(\Delta\, s) + \Delta\, s \\
\subset \quad & \{\ x = 0 \prec x + \Delta\, s \text{ has a unique solution } \} \\
& 0 \prec s - \mathit{repeat}\ (\mathit{head}\ s) + \Delta\, s \\
= \quad & \{ \text{ definition of } \Delta \text{ and arithmetic } \} \\
& (\mathit{head}\ s \prec \mathit{tail}\ s) - \mathit{repeat}\ (\mathit{head}\ s) \\
= \quad & \{ \text{ extensionality (16) } \} \\
& s - \mathit{repeat}\ (\mathit{head}\ s) \quad .
\end{aligned}
$$

For instance, $\Sigma\, 2^{\mathit{nat}} = 2^{\mathit{nat}} - 1$, since $2^{\mathit{nat}} = \Delta\, 2^{\mathit{nat}}$ and $\mathit{head}\ (2^{\mathit{nat}}) = 1$. Put differently, 2^{nat} solves the difference equation $x = 1 + \Sigma\, x$.

In Section 3.2.5, we briefly discussed course-of-value recursion, where the value of g_n is defined in terms of all previous values: $g_{n-1}, g_{n-2}, \ldots, g_0$. If we implemented g naîvely, how many recursive calls would g_n actually generate? A moment's reflection suggests that the total number is given by the difference equation $x = \mathit{nat} + \Sigma\, x$, the number of direct calls plus the sum of all indirect calls (note that $\mathit{map\ length} \cdot \mathit{course} = \mathit{nat}$ and $\mathit{map\ sum} \cdot \mathit{course} = \Sigma$).

$$
\begin{aligned}
& x = \mathit{nat} + \Sigma\, x \\
= \quad & \{ \text{ arithmetic } \} \\
& \Sigma\, x = x - \mathit{nat} \\
= \quad & \{ \text{ Fundamental Theorem (23) and } \mathit{head}\ (x - \mathit{nat}) = 0 \ \} \\
& x = \Delta\,(x - \mathit{nat}) \\
= \quad & \{ \ \Delta \text{ is linear and } \Delta\, \mathit{nat} = 1 \ \} \\
& x = \Delta\, x - 1
\end{aligned}
$$

The resulting equation is solved by $2^{\mathit{nat}} - 1$, which shows why a naîve implementation of g is prohibitive.

Table 4. *Laws for summation (c, k and n are constant streams).*

$$
\begin{array}{lcl}
\Sigma\,(\mathit{tail}\,s) & = & \mathit{tail}\,(\Sigma\,s) - \mathit{repeat}\,(\mathit{head}\,s) \\
\Sigma\,(a \prec s) & = & 0 \prec \mathit{repeat}\,a + \Sigma\,s \\
\Sigma\,(s \curlyvee t) & = & (\Sigma\,s + \Sigma\,t) \curlyvee (s + \Sigma\,s + \Sigma\,t) \\
\Sigma\,n & = & n * \mathit{nat} \\
\Sigma\,(s * \Delta\,t) & = & s*t - \Sigma\,(\Delta\,s * \mathit{tail}\,t) \\
 & & \quad - \mathit{repeat}\,(\mathit{head}\,(s*t))
\end{array}
\qquad
\begin{array}{lcl}
\Sigma\,(n*s) & = & n * \Sigma\,s \\
\Sigma\,(s+t) & = & \Sigma\,s + \Sigma\,t \\
\Sigma\,c^{\mathit{nat}} & = & (c^{\mathit{nat}} - 1)\,/\,(c-1) \\
\Sigma\,(\mathit{nat}^{\underline{n}}) & = & \mathit{nat}^{\underline{n+1}}\,/\,(n+1) \\
\Sigma\,\binom{\mathit{nat}}{k} & = & \binom{\mathit{nat}}{k+1}
\end{array}
$$

Using the Fundamental Theorem we can transform the rules in Table 3 into rules for summation, see Table 4. As an example, the rule for products, *summation by parts*, can be derived from the product rule of Δ. Let $c = \mathit{repeat}\,(\mathit{head}\,(s*t))$, then

$$
\begin{array}{ll}
 & s*\Delta\,t + \Delta\,s*\mathit{tail}\,t = \Delta\,(s*t) \\
\Longleftrightarrow & \quad \{\ \text{Fundamental Theorem (23)}\ \} \\
 & \Sigma\,(s*\Delta\,t + \Delta\,s*\mathit{tail}\,t) = s*t - c \\
\Longleftrightarrow & \quad \{\ \Sigma \text{ is linear}\ \} \\
 & \Sigma\,(s*\Delta\,t) + \Sigma\,(\Delta\,s*\mathit{tail}\,t) = s*t - c \\
\Longleftrightarrow & \quad \{\ \text{arithmetic}\ \} \\
 & \Sigma\,(s*\Delta\,t) = s*t - \Sigma\,(\Delta\,s*\mathit{tail}\,t) - c \quad .
\end{array}
$$

Unlike the others, this law is not compositional: $\Sigma\,(s*t)$ is not given in terms of $\Sigma\,s$ and $\Sigma\,t$, a situation familiar from infinite calculus.

The only slightly tricky derivation is the one for interleaving.

$$
\begin{array}{ll}
 & (t-s) \curlyvee (\mathit{tail}\,s - t) = \Delta\,(s \curlyvee t) \\
= & \quad \{\ \text{Fundamental Theorem (23) and } \mathit{head}\,(s \curlyvee t) = \mathit{head}\,s\ \} \\
 & \Sigma\,((t-s) \curlyvee (\mathit{tail}\,s - t)) = (s \curlyvee t) - \mathit{repeat}\,(\mathit{head}\,s)
\end{array}
$$

At first glance, we are stuck. To make progress, let us introduce two fresh variables: $x = t - s$ and $y = \mathit{tail}\,s - t$. If we can express s and t in terms of x and y, then we have found the desired formula.

$$
\begin{array}{ll}
 & t - s = x \ \text{ and } \ \mathit{tail}\,s - t = y \\
\Longleftrightarrow & \quad \{\ \text{arithmetic}\ \} \\
 & \mathit{tail}\,s - s = x + y \ \text{ and } \ t = x + s \\
\Longleftrightarrow & \quad \{\ \text{definition of } \Delta\ \} \\
 & \Delta\,s = x + y \ \text{ and } \ t = x + s \\
\Longleftarrow & \quad \{\ \Delta\,(\Sigma\,s) = s\ \} \\
 & s = \Sigma\,x + \Sigma\,y \ \text{ and } \ t = x + \Sigma\,x + \Sigma\,y
\end{array}
$$

Since $\mathit{head}\,s = 0$, the interleaving rule follows.

4.4.2 Examples

Using the rules in Table 4 we can mechanically calculate summations of polynomials. The main effort goes into converting between ordinary and falling factorial powers. Here is a formula for the sum of the first n squares, the *square pyramidal numbers* ($0 \prec A000330$).

$$
\begin{aligned}
& \Sigma\, nat^2 \\
= \quad & \{ \text{ converting to falling factorial powers (Table 2) } \} \\
& \Sigma\, (nat^{\underline{2}} + nat^{\underline{1}}) \\
= \quad & \{ \text{ summation laws (Table 4) } \} \\
& \tfrac{1}{3} * nat^{\underline{3}} + \tfrac{1}{2} * nat^{\underline{2}} \\
= \quad & \{ \text{ converting to ordinary powers (Table 2) } \} \\
& \tfrac{1}{3} * (nat^3 - 3 * nat^2 + 2 * nat) + \tfrac{1}{2} * (nat^2 - nat) \\
= \quad & \{ \text{ arithmetic } \} \\
& \tfrac{1}{6} * (nat - 1) * nat * (2 * nat - 1)
\end{aligned}
$$

Calculating the summation of a product, say, $\Sigma\, (nat * c^{nat})$ is often more involved. Recall that the rule for products, *summation by parts*, is imperfect: to be able to apply it, we have to spot a difference among the factors. In the example above, there is an obvious candidate: c^{nat}. Let us see how it goes.

$$
\begin{aligned}
& \Sigma\, (nat * c^{nat}) \\
= \quad & \{ \Delta\, c^{nat} = (c-1) * c^{nat} \} \\
& \Sigma\, (nat * \Delta\, c^{nat} \,/\, (c-1)) \\
= \quad & \{ \Sigma \text{ is linear } \} \\
& \Sigma\, (nat * \Delta\, c^{nat}) \,/\, (c-1) \\
= \quad & \{ \text{ summation by parts (Table 4) } \} \\
& (nat * c^{nat} - \Sigma\, (\Delta\, nat * tail\, c^{nat})) \,/\, (c-1) \\
= \quad & \{ \Delta\, nat = 1, c \text{ is constant, and definition of } nat \} \\
& (nat * c^{nat} - c * \Sigma\, c^{nat}) \,/\, (c-1) \\
= \quad & \{ \text{ summation laws (Table 4) } \} \\
& (nat * c^{nat} - c * (c^{nat} - 1) \,/\, (c-1)) \,/\, (c-1) \\
= \quad & \{ \text{ arithmetic } \} \\
& (((c-1) * nat - c) * c^{nat} + c) \,/\, (c-1)^2
\end{aligned}
$$

That was not too hard.

The summation rule for interleaving is useful to solve summations that involve alternating signs.

$$
\begin{aligned}
& \Sigma\, ((-1)^{nat} * nat) \\
= \quad & \{ (-1)^{nat} = 1 \,\Upsilon\, -1 \text{ and characterisation of } nat \} \\
& \Sigma\, (2 * nat \,\Upsilon\, -(2 * nat + 1)) \\
= \quad & \{ \text{ summation laws (Table 4) } \}
\end{aligned}
$$

$$\begin{array}{ll} & \Sigma\,(-1)\ \curlyvee\ (2*nat+\Sigma\,(-1)) \\ = & \quad \{\ \text{summation laws (Table 4) and arithmetic}\ \} \\ & -nat\ \curlyvee\ nat \end{array}$$

As a final example, let us tackle another sum that involves the interleaving operator: $\Sigma\,carry$ (*A*011371). The sum is important, as it determines the amortised running time of the binary increment. Going back to the interactive session, Section 4.4, we observe that the sum is always at most *nat*, which would imply that the amortised running time, $\Sigma\,carry\,/\,nat$, is constant. That is nice, but can we actually quantify the difference? Let us approach the problem from a different angle. The binary increment changes the number of 1s, so we might hope to relate *carry* to *ones*. The increment flips the leading 1s to 0s and flips the first 0 to 1. Now, since *carry* defines the number of leading 0s, we obtain the following alternative definition of *ones*.

$$ones = 0 \prec ones + 1 - carry$$

We omit the proof that both definitions are indeed equal. (If you want to try, use a $\subset$-proof, showing that they both satisfy the recursion equation $x = 0 \prec 1 \prec x+1-(carry\ \curlyvee\ carry)$.) Now we can invoke the *summation by happenstance* rule — the fact that $\Sigma\,s$ is the unique solution of $x = 0 \prec s+x$.

$$\begin{array}{ll} & ones = 0 \prec ones + (1 - carry) \\ \Longleftrightarrow & \quad \{\ \text{summation by happenstance}\ \} \\ & \Sigma\,(1-carry) = ones \\ \Longleftrightarrow & \quad \{\ \text{arithmetic}\ \} \\ & \Sigma\,carry = nat - ones \end{array}$$

Voilà. We have found a closed form for $\Sigma\,carry$.

That was fun. But surely, the interleaving rule would yield the result directly, wouldn't it? Let us try.

$$\begin{array}{ll} & \Sigma\,carry \\ = & \quad \{\ \text{definition of } carry\ \} \\ & \Sigma\,(0\ \curlyvee\ carry+1) \\ = & \quad \{\ \text{summation laws (Table 4)}\ \} \\ & \Sigma\,(carry+1)\ \curlyvee\ \Sigma\,(carry+1) \\ = & \quad \{\ \Sigma \text{ is linear and } \Sigma\,1 = nat\ \} \\ & (\Sigma\,carry+nat)\ \curlyvee\ (\Sigma\,carry+nat) \end{array}$$

This is quite a weird property. Since we know where we are aiming at, let us determine $nat-\Sigma\,carry$.

$$\begin{array}{ll} & nat-\Sigma\,carry \\ = & \quad \{\ \text{property of nat and } \Sigma\,carry\ \} \\ & (2*nat\ \curlyvee\ 2*nat+1)-((\Sigma\,carry+nat)\ \curlyvee\ (\Sigma\,carry+nat)) \\ = & \quad \{\ \text{arithmetic}\ \} \end{array}$$

$$(nat - \Sigma\, carry) \curlyvee (nat - \Sigma\, carry) + 1$$

Voilà again. The sequence $nat - \Sigma\, carry$ satisfies $x = x \curlyvee x + 1$, which implies that $nat - \Sigma\, carry = ones$. For the sake of completeness, we should also check that $head\ ones = head\ (nat - \Sigma\, carry)$, which is indeed the case.

4.4.3 Perturbation method

The Fundamental Theorem has another easy consequence, which is the basis of the *perturbation method*. Setting $t = tail\ s - s$ and applying the theorem from left to right we obtain

$$\Sigma\, s \quad = \quad \Sigma\, (tail\ s) - s + repeat\ (head\ s) \quad . \tag{24}$$

The idea of the method is to try to express $\Sigma\, (tail\ s)$ in terms of $\Sigma\, s$. Then we obtain an equation whose solution is the sum we seek. Let us try the method on a sum we have done before.

$$\begin{array}{ll} & \Sigma\, (nat * c^{nat}) \\ = & \quad \{ \text{ perturbation (24) and } head\ (nat * c^{nat}) = 0 \ \} \\ & \Sigma\, (tail\ (nat * c^{nat})) - nat * c^{nat} \\ = & \quad \{ \text{ definition of } nat \ \} \\ & \Sigma\, ((nat + 1) * c^{nat+1}) - nat * c^{nat} \\ = & \quad \{ \text{ summation laws (Table 4) } \} \\ & c * \Sigma\, (nat * c^{nat}) + c * \Sigma\, c^{nat} - nat * c^{nat} \\ = & \quad \{ \text{ summation laws (Table 4) } \} \\ & c * \Sigma\, (nat * c^{nat}) + c * (c^{nat} - 1) \,/\, (c - 1) - nat * c^{nat} \end{array}$$

The sum $\Sigma\, (nat * c^{nat})$ appears again on the right-hand side. All that is left to do is to solve the resulting equation, which yields the result we have seen in Section 4.4.2:

$$\begin{array}{ll} & \Sigma\, (nat * c^{nat}) \\ = & \quad \{ \text{ see above } \} \\ & (c * (c^{nat} - 1) \,/\, (c - 1) - nat * c^{nat}) \,/\, (1 - c) \\ = & \quad \{ \text{ arithmetic } \} \\ & (((c - 1) * nat - c) * c^{nat} + c) \,/\, (c - 1)^2 \quad . \end{array}$$

As an aside, the perturbation method also suggests an alternative definition of Σ, this time as a second-order fixed point.

$$\Sigma\, s = 0 \prec repeat\ (head\ s) + \Sigma\, (tail\ s)$$

The code implements the naîve way of summing: the nth element is computed using n additions not re-using any previous results.

4.4.4 Example: Moessner's Theorem

In the 1950s Alfred Moessner discovered the following intriguing scheme for generating the natural nth powers (1951): From the sequence of natural numbers, delete every nth number and form the sequence of partial sums. From the resulting sequence, delete every $(n-1)$-st number and form again the partial sums. Repeat this step $n-1$ times.

If we repeat the transformation 0 times, we obtain the 1st powers of the naturals. The second simplest instance of the process yields the squares by summing up the odd numbers.

$$\begin{array}{cccccccccc} 1 & \not{2} & 3 & \not{4} & 5 & \not{6} & 7 & \not{8} & \ldots \\ 1 & & 4 & & 9 & & 16 & & \ldots \end{array}$$

For generating the cubes we perform two deletion-and-summation steps.

$$\begin{array}{cccccccccccccc} 1 & 2 & \not{3} & 4 & 5 & \not{6} & 7 & 8 & \not{9} & 10 & 11 & \not{12} & \ldots \\ 1 & \not{3} & & 7 & \not{12} & & 19 & \not{27} & & 37 & \not{48} & & \ldots \\ 1 & & & 8 & & & 27 & & & 64 & & & \ldots \end{array} \tag{25}$$

The second sequence is probably unfamiliar — the numbers are the "three-quarter squares" ($A077043$) — but the final sequence is the desired sequence of cubes. Actually, we can add another deletion-and-summation step and start off with a sequence of ones.

$$\begin{array}{c|cccc|cccc|cccc|c} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & \\ 0 & & 1 & 2 & 3 & & 4 & 5 & 6 & & 7 & 8 & 9 & \\ 0 & & & 1 & 3 & & & 7 & 12 & & & 19 & 27 & \cdots \\ 0 & & & & 1 & & & & 8 & & & & 27 & \end{array} \tag{26}$$

As before, we have replaced the deleted elements by spaces in the rows below, so that the numbers form a sequence of triangles. To further emphasise the structure, we have shifted the triangles to the right: ◸ becomes ◹ — the following calculations are slightly more attractive then. We have also added a 'seed' column to the left, so that every element of (25) is the sum of its north-western and its western neighbour in (26). Note that the first triangle is now a prefix of Pascal's triangle rotated 90° counterclockwise.

Turning to the specification of Moessner's process, the main challenge is to model the deletion step. One approach is simply to name the elements.

$$\begin{array}{c|cccc|cccc|cccc|c} 1 & a^0_{0,0} & a^0_{1,0} & \cdots & a^0_{n,0} & a^1_{0,0} & a^1_{1,0} & \cdots & a^1_{n,0} & a^2_{0,0} & a^2_{1,0} & \cdots & a^2_{n,0} & \\ 0 & & a^0_{1,1} & \cdots & a^0_{n,1} & & a^1_{1,1} & \cdots & a^1_{n,1} & & a^2_{1,1} & \cdots & a^2_{n,1} & \\ \vdots & & & \ddots & \vdots & & & \ddots & \vdots & & & \ddots & \vdots & \cdots \\ 0 & & & & a^0_{n,n} & & & & a^1_{n,n} & & & & a^2_{n,n} & \end{array}$$

The index variable i in $a^k_{i,j}$ ranges over the columns of a triangle, j over the rows, and k over the triangles ($0 \leqslant j \leqslant i \leqslant n$ and $k \geqslant 0$). The 'name and conquer' approach with its use of three index variables is not too attractive, even though this is the approach taken by most of the proofs in the literature (Perron, 1951; Paasche, 1952; Salié, 1952). Fortunately, since the process generates an infinite number of triangles, we can eliminate the index variable k

and define a triangle of streams instead:

$$\begin{array}{c|cccc} r_0 & s_{0,0} & s_{1,0} & \cdots & s_{n,0} \\ r_1 & & s_{1,1} & \cdots & s_{n,1} \\ \vdots & & & \ddots & \vdots \\ r_n & & & & s_{n,n} \end{array} ,$$

where $(s_{i,j})_k = a^k_{i,j}$. The streams $r_0, r_1, \ldots, r_n$ model the columns to the left of the triangles: their heads form the seed column $(1, 0, \ldots, 0)$ and their tails equal the last column of the preceding triangles $(s_{n,0}, s_{n,1}, \ldots, s_{n,n})$.

$$r_j \quad = \quad [j=0] \prec s_{n,j} \ . \tag{27}$$

(Recall that the square brackets are syntax for Iverson's convention.) The following equations capture the manual process of calculating the sequences $s_{i,j}$.

$$\begin{array}{lcl} s_{i,0} & = & r_0 \\ s_{i+1,j+1} & = & s_{i,j} + r_{j+1} * [i=j] + s_{i,j+1} * [i>j] \end{array} \tag{28}$$

Each of the streams $s_{i+1,j+1}$ is the sum of its north-western $(s_{i,j})$ and its western neighbour $(r_{j+1} * [i=j] + s_{i,j+1} * [i>j])$. Moessner's Theorem then states that

$$r_n = nat^n \qquad \text{or, equivalently,} \qquad s_{n,n} = (nat+1)^n \ .$$

Turning to the proof, first observe that r_j depends on $s_{n,j}$, which in turn (indirectly) depends on r_j. So, we have effectively turned the verbal description of Moessner's process into a system of recursion equations. To illustrate, here is the system for $n = 3$:

$$\begin{array}{llll} r_0 = 1 \prec s_{3,0}; & s_{0,0} = r_0; & s_{1,0} = r_0; & s_{2,0} = r_0; \quad s_{3,0} = r_0; \\ r_1 = 0 \prec s_{3,1}; & & s_{1,1} = s_{0,0} + r_1; & s_{2,1} = s_{1,0} + s_{1,1}; \quad s_{3,1} = s_{2,0} + s_{2,1}; \\ r_2 = 0 \prec s_{3,2}; & & & s_{2,2} = s_{1,1} + r_2; \quad s_{3,2} = s_{2,1} + s_{2,2}; \\ r_3 = 0 \prec s_{3,3}; & & & \quad s_{3,3} = s_{2,2} + r_3 \ . \end{array}$$

The form of the equations does not quite meet the requirements of Section 2.3. This can be remedied, however, if we repeatedly in-line the definitions of $s_{i,j}$. For $n = 3$, the resulting equations for $s_{i,j}$ are (for reasons of space we have omitted the multiplication symbol $*$):

$$\begin{array}{llll} s_{0,0} = 1r_0; & s_{1,0} = 1r_0; & s_{2,0} = 1r_0; & s_{3,0} = 1r_0; \\ & s_{1,1} = 1r_0 + 1r_1; & s_{2,1} = 2r_0 + 1r_1; & s_{3,1} = 3r_0 + 1r_1; \\ & & s_{2,2} = 1r_0 + 1r_1 + 1r_2; & s_{3,2} = 3r_0 + 2r_1 + 1r_2; \\ & & & s_{3,3} = 1r_0 + 1r_1 + 1r_2 + 1r_3 \ . \end{array}$$

The coefficients in each column form a prefix of Pascal's triangle rotated 90° clockwise. In general, the streams $s_{i,j}$ and r_j are related by

$$s_{i,j} \quad = \quad \sum_k \tbinom{i-k}{j-k} * r_k \ . \tag{29}$$

The proof that (29) is equivalent to (28) is a straightforward nested induction, making essential use of the addition formula for binomial coefficients. It is worth noting that relation (29) holds for arbitrary r_j. (Also, r_j and $s_{i,j}$ do not have to be streams.) Using

the relation we can simplify the original system of recursion equations, (27) and (28), to

$$r_j \quad = \quad [j=0] \prec \sum_k \tbinom{n-k}{j-k} * r_k \ . \tag{30}$$

For our running example ($n=3$), we obtain the system on the left — we have simply inlined the definitions of $s_{n,j}$ into the equations $r_j = [j=0] \prec s_{n,j}$.

$$\begin{array}{lcl} r_0 & = & 1 \prec 1 * r_0; \\ r_1 & = & 0 \prec 3 * r_0 + 1 * r_1; \\ r_2 & = & 0 \prec 3 * r_0 + 2 * r_1 + 1 * r_2; \\ r_3 & = & 0 \prec 1 * r_0 + 1 * r_1 + 1 * r_2 + 1 * r_3; \end{array} \qquad \begin{array}{lcl} r_0 & = & 1; \\ r_1 & = & \Sigma\,(3 * r_0); \\ r_2 & = & \Sigma\,(3 * r_0 + 2 * r_1); \\ r_3 & = & \Sigma\,(1 * r_0 + 1 * r_1 + 1 * r_2) \ . \end{array}$$

Having eliminated the doubly indexed stream variable $s_{i,j}$, we are left with the task of solving a system of n equations. Since r_0 is the constant stream 1 and since the other streams are partial sums (see above on the right)

$$r_{j+1} \quad = \quad \Sigma\,(\sum_k \tbinom{n-k}{j+1-k} * r_k * [k \leqslant j]) \ , \tag{31}$$

we can systematically solve the recursion system top to bottom. Here are the calculations for $n=3$:

$$\begin{array}{lcllcll} r_0 & = & 1; & & & & \\ r_1 & = & \Sigma\,(\tbinom{3}{1} * r_0) & = & 3 * \Sigma\,1 & = & 3 * nat; \\ r_2 & = & \Sigma\,(\tbinom{3}{2} * r_0 + \tbinom{2}{1} * r_1) & = & 3 * \Sigma\,(1 + 2 * nat) & = & 3 * nat^2; \\ r_3 & = & \Sigma\,(\tbinom{3}{3} * r_0 + \tbinom{2}{2} * r_1 + \tbinom{1}{1} * r_2) & = & \Sigma\,(1 + 3 * nat + 3 * nat^2) & = & nat^3 \ . \end{array}$$

The summations are easy to calculate by hand. In particular, we do not need to refer to Table 4. (Do you see why?)

Actually, it is not too hard to prove Moessner's Theorem in its full glory. The following calculation, which attempts to show $tail\ r_n = (nat+1)^n$, motivates a formula for r_j.

$$\begin{array}{ll} & tail\ r_n \\ = & \quad \{ \text{ characterisation of } r_n \ (30) \ \} \\ & \sum_k \tbinom{n-k}{n-k} * r_k \\ = & \quad \{ \ \tbinom{i}{i} = [0 \leqslant i] \ \} \\ & \sum_k r_k * [k \leqslant n] \\ = & \quad \{ \text{ conjecture: } r_j = \tbinom{n}{j} * nat^j \ \} \\ & \sum_k \tbinom{n}{k} * nat^k * [k \leqslant n] \\ = & \quad \{ \text{ Binomial Theorem } \} \\ & (nat+1)^n \end{array}$$

Summarising our findings, we conjecture

$$r_j = \tbinom{n}{j} * nat^j \qquad \text{and} \qquad s_{i,j} = \sum_k \tbinom{i-k}{j-k} * \tbinom{n}{k} * nat^k \ .$$

For the proof we show that the streams $\binom{n}{j} * nat^j$ are indeed the unique solutions of (30).

$$
\begin{aligned}
& [j=0] \prec \sum_k \tbinom{n-k}{j-k} * \tbinom{n}{k} * nat^k \\
= \quad & \{ \text{ trinomial revision (Graham et al., 1994, p. 174) } \} \\
& [j=0] \prec \sum_k \tbinom{n}{j} * \tbinom{j}{k} * nat^k \\
= \quad & \{ \text{ distributive law } \} \\
& [j=0] \prec \tbinom{n}{j} * \sum_k \tbinom{j}{k} * nat^k \\
= \quad & \{ \text{ Binomial Theorem } \} \\
& [j=0] \prec \tbinom{n}{j} * (nat+1)^j \\
= \quad & \{ \, [j=0] = 0^j \, \} \\
& 0^j \prec \tbinom{n}{j} * (nat+1)^j \\
= \quad & \{ \text{ definition of } nat \, \} \\
& \tbinom{n}{j} * nat^j
\end{aligned}
$$

Since $r_n = nat^n$, Moessner's Theorem follows. The proof illustrates the importance of avoiding unnecessary detail: the central step in the derivation is the elimination of the stream variables $s_{i,j}$ from the original system of recursion equations.

Section 5.3.3 presents an alternative proof of this theorem, which avoids double subscripts and binomial coefficients. Instead, it builds on the theory of generating functions, which we introduce next.

5 Generating functions ($\times$, $\div$)

In this section, we look at number sequences from a different perspective: we take the view that a sequence, a_0, a_1, a_2 ..., represents a power series, $a_0 + a_1 z + a_2 z^2 + a_3 z^3 + \cdots$, in some formal variable z. It is an alternative view and we shall see that it provides us with additional operators and techniques for manipulating numeric streams.

5.1 Power series

Let us put on the 'power series glasses'. The simplest series, the constant function a_0 and the identity z (*A*063524), are given by

$$
\begin{aligned}
& const \quad :: (Num\ \alpha) \Rightarrow \alpha \to Stream\ \alpha \\
& const\ n = n \prec repeat\ 0 \\
& z :: (Num\ \alpha) \Rightarrow Stream\ \alpha \\
& z = 0 \prec 1 \prec repeat\ 0 \ .
\end{aligned}
$$

The sum of two power series is implemented by lifted addition. The successor function, for instance, is $const\ 1 + z$. The product of two series, however, is not given by lifted multiplication, since, for example, $(const\ 1 + z) * (const\ 1 + z) = const\ 1 + z$. Let us introduce a new operator for the product of two series, say, $\times$ and derive its implementation. The

point of departure is *Horner's rule* for evaluating a polynomial, rephrased as an identity on streams.

$$s \quad = \quad const\ (head\ s) + z \times tail\ s \tag{32}$$

The rule implies $z \times s = 0 \prec s$. The derivation of $\times$ proceeds as follows — we assume that $\times$ is associative and distributes over addition.

$$\begin{array}{ll}
 & s \times t \\
= & \quad \{\ \text{Horner's rule (32)}\ \} \\
 & (const\ (head\ s) + z \times tail\ s) \times t \\
= & \quad \{\ \text{arithmetic}\ \} \\
 & const\ (head\ s) \times t + z \times tail\ s \times t \\
= & \quad \{\ \text{Horner's rule (32)}\ \} \\
 & const\ (head\ s) \times (const\ (head\ t) + z \times tail\ t) + z \times tail\ s \times t \\
= & \quad \{\ \text{arithmetic}\ \} \\
 & const\ (head\ s) \times const\ (head\ t) + const\ (head\ s) \times z \times tail\ t + z \times tail\ s \times t \\
= & \quad \{\ const\ a \times const\ b = const\ (a * b) \text{ and arithmetic}\ \} \\
 & const\ (head\ s * head\ t) + z \times (const\ (head\ s) \times tail\ t + tail\ s \times t) \\
= & \quad \{\ \text{Horner's rule (32)}\ \} \\
 & head\ s * head\ t \prec const\ (head\ s) \times tail\ t + tail\ s \times t
\end{array}$$

The first line jointly with the last one serves as a perfectly valid implementation. However, $\times$ is a costly operation; we can improve the efficiency somewhat, if we replace $const\ k \times s$ by $repeat\ k * s$ (this law also follows from Horner's rule).

$$\begin{array}{l}
(\times)\ ::\ (Num\ \alpha) \Rightarrow Stream\ \alpha \to Stream\ \alpha \to Stream\ \alpha \\
s \times t = head\ s * head\ t \prec repeat\ (head\ s) * tail\ t + tail\ s \times t
\end{array}$$

Here are some examples ($A014824$, $0 \prec A099670$, $tail A002275$).

$$\begin{array}{l}
\gg\ nat \times 10^{nat} \\
\langle 0, 1, 12, 123, 1234, 12345, 123456, 1234567, 12345678, 123456789, \ldots \rangle \\
\gg\ 9 * (nat \times 10^{nat}) \\
\langle 0, 9, 108, 1107, 11106, 111105, 1111104, 11111103, 111111102, 1111111101, \ldots \rangle \\
\gg\ 9 * (nat \times 10^{nat}) + nat' \\
\langle 1, 11, 111, 1111, 11111, 111111, 1111111, 11111111, 111111111, 1111111111, \ldots \rangle
\end{array}$$

The operator $\times$ is also called *convolution product*. The first example suggests a pointwise characterisation: the convolution product of the streams $\langle s_0, s_1, s_2, \ldots \rangle$ and $\langle t_0, t_1, t_2, \ldots \rangle$ is $\langle s_0 * t_0, s_0 * t_1 + s_1 * t_0, s_0 * t_2 + s_1 * t_1 + s_2 * t_0, \ldots \rangle$. In general, we have

$$(s \times t)_n \quad = \quad \sum_{i,j} (s)_i * (t)_j * [i + j = n]\ .$$

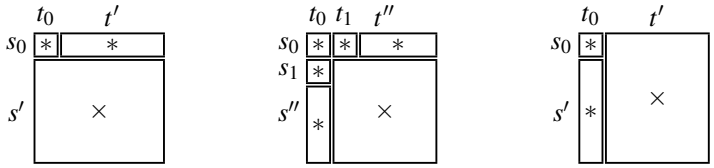
So the nth element of the convolution product is the sum of all products whose subscripts add up to n.

There is a nice non-mathematical explanation of the convolution product. Imagine two rows of people shaking hands while passing in opposite directions. Firstly, the two leaders shake hands; then the first shakes hand with the second of the other row and vice versa; then the first shakes hand with the third of the other row and so forth.

$$\begin{array}{ccc}
\ldots s_3\,s_3\,s_1\,s_0 \longrightarrow & \ldots s_3\,s_2\,s_1\quad s_0 \longrightarrow & \ldots s_3 s_2\quad s_1\quad s_0 \longrightarrow \\
\quad * & \quad * + * & \quad * + * + * \\
\longleftarrow t_0\,t_1\,t_2\,t_3\,\ldots & \longleftarrow t_0\quad t_1\,t_2\,t_3\,\ldots & \longleftarrow t_0\quad t_1\quad t_2 t_3\,\ldots
\end{array}$$

The handshake corresponds to a multiplication, the results of which are added up.

Unfortunately, the symmetry of the description is lost in the implementation above. There are, at least, two other ways to set up the corecursion (we abbreviate *head s* by s_0 and *tail s* by s').



The first element of the convolution product is $s_0 * t_0$. Then we can either add *repeat* $s_0 * t'$ to $s' \times t$ (diagram on the left) or $s \times t'$ to $s' * \mathit{repeat}\ t_0$ (diagram on the right). For a symmetric definition (diagram in the middle), we have to expand s and t twice. Fortunately, since addition is associative, all three variants are equivalent (Hinze, 2009b).

Let us complete our repertoire of arithmetic operators with reciprocal and division. Convolution was a little more complicated than the other operations, so it is wise to derive reciprocal from a specification (*recip* is a member of the class *Fractional*).

$$s \times \mathit{recip}\ s \quad = \quad \mathit{const}\ 1$$

We reason

$$\begin{array}{ll}
 & \mathit{head}\ s * \mathit{head}\ (\mathit{recip}\ s) = 1 \\
\iff & \quad \{\ \text{arithmetic}\ \} \\
 & \mathit{head}\ (\mathit{recip}\ s) = \mathit{recip}\ (\mathit{head}\ s)
\end{array}$$

and

$$\begin{array}{ll}
 & \mathit{const}\ (\mathit{head}\ s) \times \mathit{tail}\ (\mathit{recip}\ s) + \mathit{tail}\ s \times \mathit{recip}\ s = 0 \\
\iff & \quad \{\ \text{arithmetic}\ \} \\
 & -\mathit{const}\ (\mathit{head}\ s) \times \mathit{tail}\ (\mathit{recip}\ s) = \mathit{tail}\ s \times \mathit{recip}\ s \\
\iff & \quad \{\ \text{arithmetic}\ \} \\
 & \mathit{tail}\ (\mathit{recip}\ s) = \mathit{const}\ (-\mathit{recip}\ (\mathit{head}\ s)) \times \mathit{tail}\ s \times \mathit{recip}\ s\ \ .
\end{array}$$

Again replacing *const* $k \times s$ by *repeat* $k * s$, we obtain

$$\begin{array}{l}
\mathit{recip}\ s = t\ \textbf{where}\ a = \mathit{recip}\ (\mathit{head}\ s) \\
\qquad\qquad\qquad\quad t\ = a \prec \mathit{repeat}\ (-a) * (\mathit{tail}\ s \times t) \\
s \div t = s \times \mathit{recip}\ t\ \ .
\end{array}$$

Note that *recip s* is not defined if *head* $s = 0$. We use $s^{\lfloor n \rfloor}$, where n is a natural number, to denote iterated convolution with $s^{\lfloor 0 \rfloor} = const\ 1$ and set $s^{\lfloor -n \rfloor} = (recip\ s)^{\lfloor n \rfloor}$. Like *recip s*, the negative power $s^{\lfloor -n \rfloor}$ is not defined if *head* $s = 0$. For instance, $z^{\lfloor -1 \rfloor}$ does not exist.

5.2 Laws

The familiar arithmetic laws also hold for *const n*, $+$, $-$, $\times$ and $\div$. Perhaps surprisingly, we can reformulate the numeric streams we have introduced so far in terms of these operators. In other words, we view them with our new 'power series glasses'. Mathematically speaking, this conversion corresponds to finding the *generating function* (gf) of a sequence. The good news is that we need not leave our stamping ground: everything can be accomplished within the world of streams. The only caveat is that we have to be careful not to confuse *const n*, $\times$, and $s^{\lfloor n \rfloor}$ with *repeat n*, $*$, and s^n.

As a start, let us determine the generating function for *repeat a*.

$$\begin{array}{ll} & repeat\ a = a \prec repeat\ a \\ \iff & \quad \{\ \text{Horner's rule (32)}\ \} \\ & repeat\ a = const\ a + z \times repeat\ a \\ \iff & \quad \{\ \text{arithmetic}\ \} \\ & const\ 1 \times repeat\ a - z \times repeat\ a = const\ a \\ \iff & \quad \{\ \text{arithmetic}\ \} \\ & (const\ 1 - z) \times repeat\ a = const\ a \\ \iff & \quad \{\ \text{arithmetic}\ \} \\ & repeat\ a = const\ a \div (const\ 1 - z) \end{array}$$

The form of the resulting equation, $s = const\ h \div (const\ 1 - u)$, is quite typical reflecting the shape of stream equations, $s = h \prec t$.

Geometric sequences are not much harder.

$$\begin{array}{ll} & (repeat\ a)^{nat} \\ = & \quad \{\ \text{definition of exponentiation and definition of } nat\ \} \\ & 1 \prec repeat\ a * (repeat\ a)^{nat} \\ = & \quad \{\ \text{Horner's rule (32) and } repeat\ k * s = const\ k \times s\ \} \\ & const\ 1 + z \times const\ a \times (repeat\ a)^{nat} \end{array}$$

Consequently, $(repeat\ a)^{nat} = const\ 1 \div (const\ 1 - const\ a \times z)$.

We can even derive a formula for the sum of a sequence.

$$\begin{array}{ll} & \Sigma\ s = 0 \prec \Sigma\ s + s \\ \iff & \quad \{\ \text{Horner's rule (32)}\ \} \\ & \Sigma\ s = z \times (\Sigma\ s + s) \\ \iff & \quad \{\ \text{arithmetic}\ \} \\ & \Sigma\ s = s \times z \div (const\ 1 - z) \end{array}$$

Table 5. *Streams and their generating functions.*

$\mathit{repeat}\ a$	$=$	$\mathit{const}\ a \div (\mathit{const}\ 1 - z)$
$(\mathit{repeat}\ a)^{\mathit{nat}}$	$=$	$\mathit{const}\ 1 \div (\mathit{const}\ 1 - \mathit{const}\ a \times z)$
$\Sigma\, s$	$=$	$s \times z \div (\mathit{const}\ 1 - z)$
$\Sigma^{(n)}\ s$	$=$	$s \times z^{\lfloor n \rfloor} \div (\mathit{const}\ 1 - z)^{\lfloor n \rfloor}$
nat	$=$	$z \div (\mathit{const}\ 1 - z)^{\lfloor 2 \rfloor}$
$\mathit{nat} + 1$	$=$	$\mathit{const}\ 1 \div (\mathit{const}\ 1 - z)^{\lfloor 2 \rfloor}$
$\binom{n}{\mathit{nat}}$	$=$	$(\mathit{const}\ 1 + z)^{\lfloor n \rfloor}$
$\binom{n}{\mathit{nat}} * (\mathit{repeat}\ a)^{\mathit{nat}}$	$=$	$(\mathit{const}\ 1 + \mathit{const}\ a \times z)^{\lfloor n \rfloor}$
$\binom{\mathit{nat}}{k}$	$=$	$z^{\lfloor k \rfloor} \div (\mathit{const}\ 1 - z)^{\lfloor k+1 \rfloor}$

This implies that the generating function of the natural numbers is $\mathit{nat} = \Sigma\ (\mathit{repeat}\ 1) = z \div (\mathit{const}\ 1 - z)^{\lfloor 2 \rfloor}$. If we repeatedly sum the stream of ones, then we obtain the *columns* of Pascal's triangle.

$$\binom{\mathit{nat}}{k} \quad = \quad z^{\lfloor k \rfloor} \div (\mathit{const}\ 1 - z)^{\lfloor k+1 \rfloor}$$

The formula for a *row* is an immediate consequence of the Binomial Theorem.

$$\binom{n}{\mathit{nat}} \quad = \quad (z + \mathit{const}\ 1)^{\lfloor n \rfloor}$$

Repeated summation of an arbitrary stream is not any harder.

$$\Sigma^{(n)}\ s \quad = \quad s \times z^{\lfloor n \rfloor} \div (\mathit{const}\ 1 - z)^{\lfloor n \rfloor}$$

Table 5 summarises our findings.

Of course, there is no reason for jubilation: the formula for the sum does not immediately provide us with a closed form for the *coefficients* of the generating function. In fact, to be able to read off the coefficients, we have to reduce the generating function to a known stream, for instance, $\mathit{repeat}\ a$, $(\mathit{repeat}\ a)^{\mathit{nat}}$ or nat. This is what we do in the next sections.

5.3 Examples

5.3.1 Vandermonde's convolution

Many identities between binomial coefficients have straightforward explanations in terms of generating functions. *Vandermonde's convolution* serves as a particularly beautiful illustration of this observation. Graham et. al. (1994) phrase the identity as a sum of products of binomial coefficients.

$$\sum_k \binom{r}{m+k} * \binom{s}{n-k} \quad = \quad \binom{r+s}{m+n}$$

The bound variable k appears in two sub-terms, $m+k$ and $n-k$, whose sum is constant. In other words, the expression on the left-hand side is a convolution product in disguise. Using this insight, we can rephrase the identity as an equation over streams.

$$\binom{r}{\mathit{nat}} \times \binom{s}{\mathit{nat}} \quad = \quad \binom{r+s}{\mathit{nat}}$$

The proof of the law is a breeze, if we express the streams in terms of generating functions.

$$\begin{array}{ll} & \binom{r}{nat} \times \binom{s}{nat} \\ = & \quad \{ \text{ Table 5 } \} \\ & (const\ 1 + z)^{\lfloor r \rfloor} \times (const\ 1 + z)^{\lfloor s \rfloor} \\ = & \quad \{ \text{ arithmetic: laws of exponentials } \} \\ & (const\ 1 + z)^{\lfloor r+s \rfloor} \\ = & \quad \{ \text{ Table 5 } \} \\ & \binom{r+s}{nat} \end{array}$$

5.3.2 Repunits

The session in Section 5.1 shows an intriguing way to generate the sequence of repunits $\langle 1, 11, 111, \ldots \rangle$, where a repunit contains only the digit 1. The example is based on the decimal number system, but it can be readily generalised to an arbitrary base b.

$$repeat\ (b-1) * (nat \times (repeat\ b)^{nat}) + nat + 1 \quad = \quad tail\ (\Sigma\ (repeat\ b)^{nat})$$

The sum on the right-hand side models the sequence of base b repunits $\langle 1_b, 11_b, 111_b, \ldots \rangle$. The proof of the identity again uses generating functions (we abbreviate *const* 1 by $\dot{1}$).

$$\begin{array}{ll} & repeat\ (b-1) * (nat \times (repeat\ b)^{nat}) + nat + 1 \\ = & \quad \{\ repeat\ k * s = const\ k \times s\ \} \\ & const\ (b-1) \times nat \times (repeat\ b)^{nat} + nat + 1 \\ = & \quad \{ \text{ Table 5 } \} \\ & const\ (b-1) \times z \div (\dot{1} - z)^{\lfloor 2 \rfloor} \times \dot{1} \div (\dot{1} - const\ b \times z) + \dot{1} \div (\dot{1} - z)^{\lfloor 2 \rfloor} \\ = & \quad \{ \text{ arithmetic } \} \\ & \dot{1} \div (\dot{1} - z) \times \dot{1} \div (\dot{1} - const\ b \times z) \\ = & \quad \{\ tail\ (z \times s) = s\ \} \\ & tail\ (z \div (\dot{1} - z) \times \dot{1} \div (\dot{1} - const\ b \times z)) \\ = & \quad \{ \text{ Table 5 } \} \\ & tail\ (\Sigma\ (repeat\ b)^{nat}) \end{array}$$

5.3.3 Moessner's Theorem, revisited

In Section 4.4.4 we have presented a proof of Moessner's Theorem, which involved some binomial wizardry. In this section we follow an alternative route, tackling the problem using convolutions and generating functions. The resulting proof is more calculational, in particular, it avoids double subscripts and binomial coefficients. For the calculations, it will be convenient to abbreviate *const* $(s)_n$ by $(s)_n$ and *const* $[b]$ by $[b]$, where the square brackets are syntax for Iverson's convention.

As a reminder, here is the process that generates the cubes.

1	1	1	1	1	1	1	1	1	1	1	1	1	…
0		1	2	3		4	5	6		7	8	9	…
0			1	3			7	12			19	27	…
0				1				8				27	…

The verbal description suggests that the sequences are formed from top to bottom. An alternative view is that they are formed from left to right: We start with the sequence $\langle 1,0,0,0,0\ldots\rangle$ (leftmost column read from top to bottom). The sequence goes through a *triangle transformation*, which yields the sequence $\langle 1,3,3,1,0\ldots\rangle$. This sequence goes through the same transformation yielding $\langle 1,6,12,8,0\ldots\rangle$, and so forth. The sequence of elements at position 3 are the cubes. In a sense, we have already justified this alternative view in Section 4.4.4, when we showed that the streams $s_{i,j}$ can be defined solely in terms of r_j — recall that the streams r_j model the columns to the left of the triangles. The slight twist here is that we switch from row-major to column-major order.

If we view the columns through our 'power series glasses', the summation steps can be modelled by a simple multiplication: if g_i is some column, then $g_i \times (\dot{1} + z)$ is the next column (again we abbreviate *const* 1 by $\dot{1}$). Recall that each element of a triangle is the sum of its western and its north-western neighbour; g_i represents the former and $g_i \times z$ the latter (g_i shifted a row downwards). Additionally taking the 'input column', called f, into account, the columns $g_1, \ldots, g_n$ of a triangle are related by

$$\begin{aligned} g_0 &= (f)_0 \times z^{\lfloor 0 \rfloor}; \\ g_{i+1} &= (f)_{i+1} \times z^{\lfloor i+1 \rfloor} + g_i \times (\dot{1} + z) \ . \end{aligned}$$

For $n = 3$, we obtain the system of equations on the left. For instance, if the input column f is $\langle 1,6,12,8,0\ldots\rangle$, then the output column g_3 is $\langle 1,9,27,27,0\ldots\rangle$.

$$\begin{aligned} g_0 &= (f)_0 \times z^{\lfloor 0 \rfloor} \\ g_1 &= (f)_1 \times z^{\lfloor 1 \rfloor} + g_0 \times (\dot{1} + z) \\ g_2 &= (f)_2 \times z^{\lfloor 2 \rfloor} + g_1 \times (\dot{1} + z) \\ g_3 &= (f)_3 \times z^{\lfloor 3 \rfloor} + g_2 \times (\dot{1} + z) \end{aligned}$$

f	g_0	g_1	g_2	g_3
1	1	1	1	1
6		7	8	9
12			19	27
8				27 .

Distributing the products over the sums, it is not hard to see that the generating functions satisfy the invariant

$$g_{i+1} \times (\dot{1} + z)^{\lfloor m-(i+1) \rfloor} = (f)_{i+1} \times z^{\lfloor i+1 \rfloor} \times (\dot{1} + z)^{\lfloor m-(i+1) \rfloor} + g_i \times (\dot{1} + z)^{\lfloor m-i \rfloor} \ ,$$

which allows us to define g_i solely in terms of f:

$$g_i = \sum_k (f)_k \times z^{\lfloor k \rfloor} \times (\dot{1} + z)^{\lfloor i-k \rfloor} \times [k \leqslant i] \ .$$

The factor $(\dot{1} + z)^{\lfloor i-k \rfloor}$ indicates that binomial coefficients are lurking behind the scenes and, indeed,

$$(g_i)_j = \sum_k (f)_k * \tbinom{i-k}{i-j} * [k \leqslant i] = \sum_k (f)_k * \tbinom{i-k}{j-k} \ ,$$

which is Equation (29) modulo naming of variables. Fortunately, we do not need the pointwise characterisation in what follows. The central idea is to view the triangle transformation as a *function* that maps the input column f to the output column g_n:

$$\nabla_n\ f \quad = \quad \sum_k (f)_k \times z^{\lfloor k \rfloor} \times (\dot{1}+z)^{\lfloor n-k \rfloor} \times [k \leqslant n] \ .$$

Moessner's process can be formalised by repeatedly applying the triangle transformation to the power series $\dot{1}$; the coefficients at position n then form the sequence of nth powers.

$$\mathit{moessner}\ n = \langle (f)_n \mid f \leftarrow \dot{1}, \nabla_n\ f \,.. \rangle$$

Turning to the proof of Moessner's Theorem, let us first study the stream operator ∇_n in more detail. The formula for ∇_n looks a bit daunting, but, as we shall see, the triangle transformation enjoys a number of pleasant properties. The first non-trivial instance gives

$$\begin{array}{lcl} \nabla_1\ \dot{1} & = & \dot{1}+z\ ; \\ \nabla_1\ z & = & z\ . \end{array}$$

Furthermore, it is not hard to see that the operator is a linear transformation:

$$\begin{array}{lcl} \nabla_n\ (\mathit{const}\ k \times f) & = & \mathit{const}\ k \times \nabla_n\ f\ ; \\ \nabla_n\ (f+g) & = & \nabla_n\ f + \nabla_n\ g\ . \end{array}$$

For instance, using *const* 2 rather than *const* 1 as the initial seed yields the natural powers doubled. Perhaps surprisingly, the triangle transformation also distributes over the convolution product: if $(f)_i = 0$ for $i > m$ and $(g)_j = 0$ for $j > n$, then

$$\nabla_{m+n}\ (f \times g) \quad = \quad \nabla_m\ f \times \nabla_n\ g\ .$$

The side condition requires that f is a polynomial of degree at most m and, likewise, that g has degree at most n. Table 6 displays the proof of this property. An immediate consequence is $\nabla_n\ \dot{1} = \nabla_n\ (\dot{1}^{\lfloor n \rfloor}) = (\nabla_1\ \dot{1})^{\lfloor n \rfloor} = (\dot{1}+z)^{\lfloor n \rfloor}$, or, more generally,

$$\nabla_n\ ((\dot{1}+\mathit{const}\ i \times z)^{\lfloor n \rfloor}) = (\nabla_1\ (\dot{1}+\mathit{const}\ i \times z))^{\lfloor n \rfloor} = (\dot{1}+\mathit{const}\ (i+1) \times z)^{\lfloor n \rfloor}\ . \quad (33)$$

For instance, the cube-generating process yields

$\dot{1}$				$(\dot{1}+z)^{\lfloor 3 \rfloor}$				$(\dot{1}+\dot{2} \times z)^{\lfloor 3 \rfloor}$				$(\dot{1}+\dot{3} \times z)^{\lfloor 3 \rfloor}$	…
1	1	1	1	1	1	1	1	1	1	1	1	1	…
0		1	2	3		4	5	6		7	8	9	…
0			1	3			7	12			19	27	…
0				1				8				27	…

In general, after i triangle transformations we obtain the sequence $(\dot{1}+\mathit{const}\ i \times z)^{\lfloor n \rfloor}$:

$$\mathit{iterate}\ \nabla_n\ \dot{1} \quad = \quad \langle (\dot{1}+\mathit{const}\ i \times z)^{\lfloor n \rfloor} \mid i \leftarrow \mathit{nat} \rangle\ .$$

Here is a simple unique fixed-point proof of this fact. We show that the right-hand side satisfies the equation $x = \dot{1} \prec \mathit{map}\ \nabla_n\ x$.

$$\langle (\dot{1}+\mathit{const}\ i \times z)^{\lfloor n \rfloor} \mid i \leftarrow \mathit{nat} \rangle$$

Table 6. *Proof that ∇ distributes over $\times$.*

$$
\begin{array}{ll}
 & \nabla_{m+n}\,(f\times g)\\
= & \quad \{\text{ definition of } \nabla \}\\
 & \sum_k (f\times g)_k \times z^{\lfloor k\rfloor} \times (\dot{1}+z)^{\lfloor m+n-k\rfloor} \times [k\leqslant m+n]\\
= & \quad \{\text{ pointwise characterisation of } \times \}\\
 & \sum_k (\sum_i (f)_i \times (g)_{k-i}) \times z^{\lfloor k\rfloor} \times (\dot{1}+z)^{\lfloor m+n-k\rfloor} \times [k\leqslant m+n]\\
= & \quad \{\text{ distributive law and interchanging the order of summation }\}\\
 & \sum_{i,k} (f)_i \times (g)_{k-i} \times z^{\lfloor k\rfloor} \times (\dot{1}+z)^{\lfloor m+n-k\rfloor} \times [k\leqslant m+n]\\
= & \quad \{\text{ replace } k \text{ by } i+j \}\\
 & \sum_{i,j} (f)_i \times (g)_j \times z^{\lfloor i+j\rfloor} \times (\dot{1}+z)^{\lfloor m+n-(i+j)\rfloor} \times [i+j\leqslant m+n]\\
= & \quad \{\text{ assumption: } (f)_i = 0 \text{ for } i>m \text{ and } (g)_j = 0 \text{ for } j>n \}\\
 & \sum_{i,j} (f)_i \times (g)_j \times z^{\lfloor i+j\rfloor} \times (\dot{1}+z)^{\lfloor m-i+n-j\rfloor} \times [i\leqslant m \wedge j\leqslant n]\\
= & \quad \{\text{ laws of exponentials and } [b\wedge c] = [b]\times[c] \}\\
 & \sum_{i,j} (f)_i \times z^{\lfloor i\rfloor} \times (\dot{1}+z)^{\lfloor m-i\rfloor} \times [i\leqslant m] \times (g)_j \times z^{\lfloor j\rfloor} \times (\dot{1}+z)^{\lfloor n-j\rfloor} \times [j\leqslant n]\\
= & \quad \{\text{ distributive law }\}\\
 & (\sum_i (f)_i \times z^{\lfloor i\rfloor} \times (\dot{1}+z)^{\lfloor m-i\rfloor} \times [i\leqslant m]) \times (\sum_j (g)_j \times z^{\lfloor j\rfloor} \times (\dot{1}+z)^{\lfloor n-j\rfloor} \times [j\leqslant n])\\
= & \quad \{\text{ definition of } \nabla \}\\
 & \nabla_m\, f \times \nabla_n\, g
\end{array}
$$

$$
\begin{array}{ll}
= & \quad \{\text{ definition of } nat \}\\
 & (\dot{1}+const\ 0\times z)^{\lfloor n\rfloor} \prec \langle (\dot{1}+const\ i\times z)^{\lfloor n\rfloor} \mid i \leftarrow nat+1 \rangle\\
= & \quad \{\text{ arithmetic and functor law (4) }\}\\
 & \dot{1} \prec \langle (\dot{1}+const\ (i+1)\times z)^{\lfloor n\rfloor} \mid i \leftarrow nat \rangle\\
= & \quad \{\text{ (33) }\}\\
 & \dot{1} \prec \langle \nabla_n\, (\dot{1}+const\ i\times z)^{\lfloor n\rfloor} \mid i \leftarrow nat \rangle\\
= & \quad \{\text{ functor law (4) }\}\\
 & \dot{1} \prec map\ \nabla_n \langle (\dot{1}+const\ i\times z)^{\lfloor n\rfloor} \mid i \leftarrow nat \rangle
\end{array}
$$

We are now in a position to re-prove Moessner's Theorem. Since the kth coefficient of $(\dot{1}+const\ i\times z)^{\lfloor n\rfloor}$ is $\binom{n}{k} * i^k$, see Table 5, we have

$$
\begin{array}{ll}
 & moessner\ n\\
= & \quad \{\text{ definition of } moessner \}\\
 & \langle (f)_n \mid f \leftarrow iterate\ \nabla_n\ \dot{1} \rangle
\end{array}
$$

$$
\begin{array}{ll}
= & \quad \{ \text{ see above } \} \\
& \langle (f)_n \mid f \leftarrow \langle (\dot{1} + const\ i \times z)^{\lfloor n \rfloor} \mid i \leftarrow nat \rangle \rangle \\
= & \quad \{ \text{ functor law (4) } \} \\
& \langle ((\dot{1} + const\ i \times z)^{\lfloor n \rfloor})_n \mid i \leftarrow nat \rangle \\
= & \quad \{ ((\dot{1} + const\ i \times z)^{\lfloor n \rfloor})_n = \binom{n}{n} * i^n = i^n \ \} \\
& \langle i^n \mid i \leftarrow nat \rangle \\
= & \quad \{ \text{ extensionality (16) } \} \\
& nat^n \quad .
\end{array}
$$

Voilà. The introduction of the stream operator ∇_n illustrates the important idea of avoiding unnecessary detail. The triangle transformation hides away the generating functions of the intermediate columns, which are not immediately needed to establish the overall result.

5.4 Solving recurrences

Generating functions are jolly useful to solve numeric recurrences, that is, to turn a recursive stream definition into a non-recursive one. Let us illustrate the approach using a simple recurrence, the closed form of which we can determine with an informed guess.

$$hanoi = 0 \prec hanoi + 1 + hanoi$$

The stream *hanoi* yields the number of moves to solve the Tower of Hanoï problem with *nat* discs. It is not hard to see that $hanoi = 2^{nat} - 1$.

As a first step, we express *hanoi* in terms of $\times$ and friends.

$$
\begin{array}{ll}
& hanoi = 0 \prec hanoi + 1 + hanoi \\
= & \quad \{ \text{ Horner's rule (32) } \} \\
& hanoi = z \times (hanoi + 1 + hanoi) \\
= & \quad \{ \text{ Table 5 } \} \\
& hanoi = z \times (hanoi + (const\ 1 \div (const\ 1 - z)) + hanoi) \\
= & \quad \{ \text{ arithmetic } \} \\
& hanoi = z \div ((const\ 1 - z) \times (const\ 1 - const\ 2 \times z))
\end{array}
$$

To reiterate, we have to be careful not to confuse *const n* and $\times$ with *repeat n* and $*$. In particular, recall that the numeral 1 is shorthand for *repeat* 1, so its generating function is $const\ 1 \div (const\ 1 - z)$, *not* $const\ 1$.

For the second step, we have to turn the right-hand side of the resulting equation into a generating function or a sum of generating functions whose coefficients we know. The following algebraic identity points us into the right direction ($\alpha \neq \beta$).

$$\frac{x}{(1-\alpha x)(1-\beta x)} = \frac{1}{\alpha - \beta}\left(\frac{1}{1-\alpha x} - \frac{1}{1-\beta x}\right) \tag{34}$$

Inspecting Table 5 we realise that we know the stream expression for the right-hand side:

$$repeat\ (1\ /\ \alpha - \beta) * ((repeat\ \alpha)^{nat} - (repeat\ \beta)^{nat}) \quad .$$

For our running example, $\alpha = 1$ and $\beta = 2$. Substituting the values and applying some algebraic simplifications, we obtain the expected result.

$$hanoi \quad = \quad 2^{nat} - 1$$

A noteworthy feature of the derivation is that it stays entirely within the world of streams.

Now, let us try to find a closed form for our all-time favourite, the Fibonacci sequence. The first step, determining the generating function, is probably routine by now.

$$\begin{aligned} & fib = 0 \prec (1 \prec fib) + fib \\ = & \quad \{ \text{ Horner's rule (32) } \} \\ & fib = z \times ((const\ 1 + z \times fib) + fib) \\ = & \quad \{ \text{ arithmetic } \} \\ & fib = z \div (const\ 1 - z - z^{\lfloor 2 \rfloor}) \end{aligned}$$

To be able to apply (34), we have to transform $1 - z - z^2$ into the form $(1 - \alpha z)(1 - \beta z)$. It turns out that the roots of $z^2 - z - 1$ are the reciprocals of the roots of $1 - z - z^2$. (The trick of reversing the coefficients works in general, see (Graham et al., 1994, p. 339).) A quick calculation shows that $\phi = \frac{1}{2}(1 + \sqrt{5})$, the golden ratio $\frac{a+b}{a} = \frac{a}{b}$, and $\hat{\phi} = \frac{1}{2}(1 - \sqrt{5})$ are the roots we seek: $z^2 - z - 1 = (z - \phi)(z - \hat{\phi})$. Consequently, $1 - z - z^2 = (1 - \phi z)(1 - \hat{\phi} z)$. Since furthermore $\phi - \hat{\phi} = \sqrt{5}$, we have inferred that

$$fib = repeat\ (1\ /\ sqrt\ 5) * ((repeat\ \phi)^{nat} - (repeat\ \hat{\phi})^{nat}) \quad .$$

We can reformulate the result in more familiar terms

$$\mathscr{F}_n \quad = \quad \frac{1}{\sqrt{5}} \left(\phi^n - \hat{\phi}^n \right) \quad ,$$

but this is only cosmetics.

Many recurrences can be solved along those lines. In general, the main effort goes into finding the *partial fraction expansion* of the rational function obtained as the result of the first step. Very briefly, every rational function (which is defined at 0) can be expressed as the sum of a polynomial and a linear combination of functions of the form

$$const\ 1 \div (const\ 1 - const\ a \times z)^{\lfloor k+1 \rfloor} \quad = \quad \tbinom{nat+k}{k} * (repeat\ a)^{nat} \quad . \tag{35}$$

For the details, we refer the interested reader to (Graham et al., 1994, Section 7.3). As a quick example, consider the recursion equation $x = 0 \prec 2 * x + 2^{nat}$, which captures the puzzle posed in Section 4.2. Its generating function is $z \div (const\ 1 - const\ 2 \times z)^{\lfloor 2 \rfloor}$, which by (35) is equal to

$$z \times \left(\tbinom{nat+1}{1} * 2^{nat} \right) = 0 \prec (nat + 1) * 2^{nat} = nat * 2^{nat-1} \quad .$$

5.5 Counting

In Section 3.2.1 we have turned a context-free grammar (in Backus-Naur Form) into a system of stream equations — for each non-terminal X we introduced a stream x whose nth element specified the number of words of length n produced by X. Using generating functions we can nicely mechanise the translation: the grammar equation $X = E$ is turned

into the stream equation $x = e$ whose right-hand side is formed according to

BNF expression			stream expression
	$\emptyset$	$const\ 0$	
	ε	$const\ 1$	
	a	z	
	X	x	
	$E_1 \mid E_2$	$e_1 + e_2$	
	$E_1\ E_2$	$e_1 \times e_2$.	

Each terminal symbol a is replaced by z — we are only interested in the length of the words — each non-terminal symbol X is replaced by the corresponding stream variable x. Alternation corresponds to lifted addition and concatenation to the convolution product. To illustrate, here is the original grammar

$$\begin{aligned} Nao &= \varepsilon \mid 0\ Nao \mid 1\ Nap\ ; \\ Nap &= \varepsilon \mid 0\ Nao\ , \end{aligned}$$

and here is the corresponding system of stream equations

$$\begin{aligned} nao &= const\ 1 + z \times nao + z \times nap\ ; \\ nap &= const\ 1 + z \times nao\ . \end{aligned}$$

Horner's rule (32) immediately gives us the stream equations of Section 3.2.1.

There is one caveat, however. The translation tacitly assumes that $E_1 \mid E_2$ is a *disjoint* union. Otherwise, we do not count the number of words of some length, but rather the number of their parse trees. Fortunately, in our example, the languages generated by ε, $0\ Nao$ and $1\ Nap$ are clearly disjoint.

Essentially the same technique can also be applied to parametric datatypes, so-called *container types*, to count the number of containers of some given size. Consider as an example the inductive datatype of binary trees.

$$\mathbf{data}\ Tree\ \alpha = Empty \mid Node\ (Tree\ \alpha, \alpha, Tree\ \alpha)$$

There is one tree of size 0, one tree of size 1, two trees of size 3, five trees of size 4, and so forth. In general, the number of trees of a given size satisfies

$$tree = const\ 1 + tree \times z \times tree\ .$$

Using Horner's rule (32) this specification translates into the executable stream equation

$$tree = 1 \prec tree \times tree\ .$$

The correspondence between datatype declarations and stream equations can be seen more clearly if we write the former in a point-free or functorial style:

$$Tree = K\ 1 \dot{+} Tree \mathbin{\dot{\times}} Id \mathbin{\dot{\times}} Tree\ ,$$

where K is the constant type constructor, $K\ \alpha\ \beta = \alpha$, Id is the identity, $Id\ \alpha = \alpha$, and $\dot{+}$ and $\mathbin{\dot{\times}}$ are lifted sum and product, $(\iota \dot{+} \kappa)\ \alpha = Either\ (\iota\ \alpha)\ (\kappa\ \alpha)$ and $(\iota \mathbin{\dot{\times}} \kappa)\ \alpha = (\iota\ \alpha, \kappa\ \alpha)$. As a second example, the functional programmer's favourite datatype, the inductive type of lists,

$$\mathbf{data}\ List\ \alpha = Nil \mid Cons\ (\alpha, List\ \alpha)$$

enjoys the following point-free definition.

$$List = K\ 1 \dot{+} Id \mathbin{\dot{\times}} List$$

The corresponding stream equation is then

$$list = const\ 1 + z \times list \quad .$$

Horner's rule implies $list = 1 \prec list$ and, consequently, $list = repeat\ 1$. For each size, there is exactly one list container, as expected.

Quite amazingly, the technique also works for so-called *non-regular* or *nested datatypes* (Bird & Meertens, 1998). Consider as an example the type of perfect trees (Hinze, 2000a).

$$\textbf{data}\ Perfect\ \alpha = Zero\ \alpha \mid Succ\ (Perfect\ (\alpha, \alpha))$$

A perfect tree of height 0 is a singleton; a perfect tree of height $h+1$ is a perfect tree of height h whose elements are pairs. Thus, a tree of the form $Succ^h\ (Zero\ t)$ has exactly 2^h elements. It will be interesting to see how this property translates into the world of streams. As a first step, we put the type definition into a point-free form:

$$Perfect = Id \dot{+} Perfect \circ (Id \mathbin{\dot{\times}} Id) \quad ,$$

where $\circ$ is type composition, $(\iota \circ \kappa)\ \alpha = \iota\ (\kappa\ \alpha)$. Now, which stream operator models composition of type constructors? A moment's reflection reveals that the corresponding operator is simply composition of generating functions. Loosely speaking, the composition $s \circ t$ replaces the formal variable z in s by t, for instance, $(z \times z) \circ t = t \times t$. Let us postpone the implementation of composition and first return to our application. The number of perfect trees of some size then satisfies the stream equation

$$perfect \quad = \quad z + perfect \circ (z \times z) \quad . \tag{36}$$

As an intermediate summary, to count containers we transform the functor equation $X = E$ into the stream equation $x = e$ whose right-hand side is formed according to

functor expression		stream expression
$K\ 0$	$const\ 0$	
$K\ 1$	$const\ 1$	
Id	z	
X	x	
$F_1 \dot{+} F_2$	$e_1 + e_2$	
$F_1 \mathbin{\dot{\times}} F_2$	$e_1 \times e_2$	
$F_1 \circ F_2$	$e_1 \circ e_2$	

This time there is no caveat, since $\dot{+}$ forms a disjoint union by definition. Of course, there is no guarantee that the resulting system of stream equations has a unique solution or even a solution at all: $x = x$ has infinitely many solutions, $x = const\ 1 + z \times x$ has a unique solution, and $x = z + x$ has no solution.

Now, let us derive an implementation of composition assuming the laws listed in Table 7 ($\circ$ binds more tightly than $\times$).

$$\begin{array}{ll} & s \circ t \\ = & \quad \{\ \text{Horner's rule (32)}\ \} \end{array}$$

Table 7. *Properties of composition.*

$z \circ s$	$=$	s	$(s \times t) \circ u$	$=$	$s \circ u \times t \circ u$
$s \circ z$	$=$	s	$(s \div t) \circ u$	$=$	$s \circ u \div t \circ u$
$(s \circ t) \circ u$	$=$	$s \circ (t \circ u)$	$recip\ s \circ u$	$=$	$recip\ (s \circ u)$
$const\ a \circ u$	$=$	$const\ a$	$s^{\lfloor n \rfloor} \circ u$	$=$	$(s \circ u)^{\lfloor n \rfloor}$
$(s + t) \circ u$	$=$	$s \circ u + t \circ u$	$s \curlyvee t$	$=$	$(s \circ z^{\lfloor 2 \rfloor}) + z \times (t \circ z^{\lfloor 2 \rfloor})$

$$
\begin{aligned}
& (const\ (head\ s) + z \times tail\ s) \circ t) \\
= & \quad \{ \circ \text{ right-distributes over } + \text{ and } \times \} \\
& const\ (head\ s) \circ t + z \circ t \times tail\ s \circ t \\
= & \quad \{ const\ a \circ u = const\ a \text{ and } z \circ u = u \} \\
& const\ (head\ s) + t \times tail\ s \circ t \\
= & \quad \{ \textbf{assumption:}\ head\ t = 0 \} \\
& const\ (head\ s) + z \times tail\ t \times tail\ s \circ t \\
= & \quad \{ \text{Horner's rule (32)} \} \\
& head\ s \prec tail\ t \times tail\ s \circ t
\end{aligned}
$$

In order to make progress, we have to assume that $head\ t = 0$. In general, the first coefficient of $s \circ t$ is given by an infinite sum, which possibly exists, but which we cannot compute mechanically in Haskell. Like *recip*, composition is a partial operation.

$$
\begin{aligned}
& (\circ)\ ::\ (Num\ \alpha) \Rightarrow Stream\ \alpha \to Stream\ \alpha \to Stream\ \alpha \\
& s \circ t = head\ s \prec tail\ t \times tail\ s \circ t
\end{aligned}
$$

Composition is associative with z as its neutral element; and it right-distributes over the arithmetic operations ($+$, $\times$, $\div$). Furthermore, there is an intriguing cross-connection to interleaving. Consider the generating function $s \circ z^{\lfloor 2 \rfloor}$; its even coefficients are those of s, its odd coefficients are zero. Consequently, we can express interleaving using composition:

$$s \curlyvee t \quad = \quad (s \circ z^{\lfloor 2 \rfloor}) + z \times (s \circ z^{\lfloor 2 \rfloor})\ .$$

Table 7 summarises our findings.

We can use the properties to turn the specification of *perfect* into an executable form. First of all, the stream equation is ambiguous as it does not determine the head of *perfect*. Since there is no empty perfect tree, we set $head\ perfect = 0$. For the tail, we reason

$$
\begin{aligned}
& tail\ perfect \\
= & \quad \{ \text{specification of } perfect \text{ (36)} \} \\
& tail\ (z + perfect \circ z^{\lfloor 2 \rfloor}) \\
= & \quad \{ \text{definition of } z \text{ and } \circ \} \\
& const\ 1 + z \times (tail\ perfect \circ z^{\lfloor 2 \rfloor}) \\
= & \quad \{ s \circ z^{\lfloor 2 \rfloor} = s \curlyvee 0 \text{ (Table 7)} \} \\
& const\ 1 + z \times (tail\ perfect \curlyvee 0)
\end{aligned}
$$

$$\begin{aligned} &= \quad \{ \text{ Horner's rule (32) } \} \\ &\quad 1 \prec \mathit{tail}\,\mathit{perfect} \curlyvee 0 \ . \end{aligned}$$

Thus, *tail perfect* is essentially *pot* — if we identify 0 with *false* and 1 with *true*.

$$\begin{aligned} \mathit{perfect} &= 0 \prec \mathit{perfect}' \\ \mathit{perfect}' &= 1 \prec \mathit{perfect}' \curlyvee 0 \end{aligned}$$

It is high time to see the definitions in action (*A*000012, *A*000108, not listed).

$\gg$ *list*
$\langle 1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,\ldots\rangle$
$\gg$ *tree*
$\langle 1,1,2,5,14,42,132,429,1430,4862,16796,58786,208012,742900,2674440,\ldots\rangle$
$\gg$ *perfect*
$\langle 0,1,1,0,1,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,\ldots\rangle$

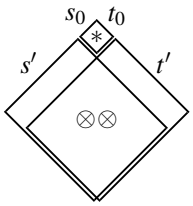
The stream *tree* captures a well-known combinatorial sequence, the *Catalan* or *Segner* numbers. Can you derive a closed form?

5.6 Exponential generating functions

The implementation of the convolution product exhibits an unpleasant asymmetry. There is a related operator, called *shuffle product*, whose definition is perfectly symmetric.

$$s \otimes t = \mathit{head}\, s * \mathit{head}\, t \prec \mathit{tail}\, s \otimes t + s \otimes \mathit{tail}\, t$$

The two-dimensional illustration of the corecursion scheme below shows that the shuffle products in the tail overlap, in a way reminiscent of the recurrence underlying Pascal's triangle (the picture is rotated 45° degrees clockwise to emphasise the correspondence).



For instance, the term $s_1 * t_1$ appears once in $\mathit{tail}\, s \otimes t$ and once in $s \otimes \mathit{tail}\, t$, and hence twice in $s \otimes t$.

The shuffle product implements the product of two *exponential generating functions* (egfs). An egf is similar to a gf except that the nth coefficient is additionally divided by n factorial: $a_0 + a_1 z + a_2 z^2/2! + a_3 z^3/3! + \cdots$. Like the convolution product, the shuffle product is associative, commutative, it distributes over sums, $\mathit{const}\, 1 \otimes s = s$, and furthermore $(c * s) \otimes t = c * (s \otimes t) = s \otimes (c * t)$.

To turn a stream representing a gf into a stream representing an egf, we simply have to multiply the former by *fac*.

$$\Lambda\, s = s * \mathit{fac}$$

The two products are then related by

$$\Lambda\,(s \times t) \quad = \quad \Lambda\, s \otimes \Lambda\, t \ .$$

The formula enables the derivation of the following pointwise characterisation of the shuffle product, which links in with the earlier observation that the corecursion scheme of $\otimes$ is related to Pascal's triangle. Since $(i+j)\,!\,/(i\,!*j!) = \binom{i+j}{i}$, we have

$$(s \otimes t)_n \quad = \quad \sum_{i,j} \binom{n}{i} * (s)_i * (t)_j * [i+j=n] \ .$$

So, the shuffle product is similar to the convolution product, except that each point-level product is multiplied by a binomial coefficient (which suggests a more efficient implementation of $\otimes$). Because of that, there is a revealing cross-connection to binomial transforms:

$$\mathscr{B}_n\, s \quad = \quad n^{nat} \otimes s \ .$$

Indeed, if we unfold $n^{nat} \otimes s$, we obtain the recursion equation of $\mathscr{B}_n$. In particular, $\mathscr{B}\, s = 1 \otimes s$. All the properties of the binomial transform listed in Section 4.2 follow directly from this correspondence.

First of all, here is the stream version of the Binomial Theorem.

$$m^{nat} \otimes n^{nat} \quad = \quad (m+n)^{nat}$$

Egfs are well-behaved, when their coefficients are geometric progressions, hence the name. The unique fixed-point proof of the above property is remarkable, as it does not mention binomial coefficients at all.

$$\begin{array}{ll} & m^{nat} \otimes n^{nat} \\ = & \quad \{\ \text{definition of } \otimes\ \} \\ & m^0 * n^0 \prec m^{nat+1} \otimes n^{nat} + m^{nat} \otimes n^{nat+1} \\ = & \quad \{\ \text{arithmetic and } (c*s) \otimes t = c*(s \otimes t) = s \otimes (c*t)\ \} \\ & 1 \prec (m+n) * (m^{nat} \otimes n^{nat}) \end{array}$$

Back to the correspondence between shuffle product and binomial transforms: Proving, for instance, that $\mathscr{B}$ and $\mathscr{B}^{(-1)}$ are mutually inverse is now a breeze.

$$\mathscr{B}\,(\mathscr{B}^{(-1)}\, s) = 1^{nat} \otimes (-1)^{nat} \otimes s = 0^{nat} \otimes s = const\ 1 \otimes s = s$$

The other laws can be shown using similar arguments.

The moral of the story is that the *right* generalisation often makes our life easier.

6 Related work

As mentioned in the introduction, the two major sources of inspiration were Rutten's work on stream calculus (2003; 2005) and the textbook on concrete mathematics (Graham *et al.*, 1994). Rutten introduces streams and stream operators using coinductive definitions, which he calls *behavioural differential equations*. As an example, the Haskell definition of lifted addition

$$s + t = head\ s + head\ t \prec tail\ s + tail\ t$$

translates to

$$(s+t)(0) = s(0)+t(0) \quad \text{and} \quad (s+t)' = s'+t' \ ,$$

where $s(0)$ denotes the head of s, its initial value, and s' the tail of s, its stream derivative. (The notation goes back to Hoare.) Rutten shows the existence and uniqueness of solutions for a particular system of behavioural differential equations (2003, Theorem 3.1). (The conference version of this paper rephrases the proof using type classes and generalised algebraic data types (Hinze, 2008).) Bartels proves uniqueness for the general case in a categorical setting (2003).

Interestingly, Rutten relies on coinduction as the main proof technique and emphasises the 'power series' view of streams. In fact, we have given power series and generating functions only a cursory treatment, as there are already a number of papers on that subject, most notably, (Karczmarczuk, 1997; McIlroy, 1999; McIlroy, 2001). Both Karczmarczuk and McIlroy mention the proof technique of unique fixed points in passing by: Karczmarczuk sketches a proof of $\mathit{iterate}\ f \cdot f = \mathit{map}\ f \cdot \mathit{iterate}\ f$ and McIlroy shows $1/e^x = e^{-x}$. As an aside, both use lazy lists to represent streams, resulting in additional code to cover the empty list.

Various proof methods for corecursive programs are discussed by Gibbons and Hutton (2005). The technique of unique fixed points is not among them.[10] Unique fixed-point proofs are closely related to the principle of *guarded induction* (Coquand, 1994), which goes back to the work on process algebra (Milner, 1989). Loosely speaking, the guarded condition ensures that functions are productive by restricting the context of a recursive call to one ore more constructors. For instance,

$$\mathit{nat} = 1 \prec \mathit{nat}+1$$

is not guarded as $+$ is not a constructor. However, *nat* can be defined by $\mathit{iterate}\ (+1)\ 0$ as *iterate* is guarded. The proof method then allows us to show that $\mathit{iterate}\ (+1)\ 0$ is the unique solution of $x = x \prec x+1$ by constructing a suitable proof transformer using guarded equations. Indeed, the central idea underlying guarded induction is to express proofs as lazy functional programs.

Recently, Niqui and Rutten introduced various operations for partitioning, projecting and merging streams (2010). While the operators generalise *even*, *odd* and Υ, it is not clear under which conditions they can be used in other definitions — as we have pointed out, the equation $x = 0 \prec \mathit{even}\ x$, for instance, has infinitely many solutions.

The use of different structural views — idioms, tabulations and final coalgebras — to organise operations, laws and proofs appears to be original. Categorically, idioms are *lax monoidal functors* (Mac Lane, 1998) with strength. Programmatically, idioms arose as an interface for parsing combinators (Röjemo, 1995). McBride and Paterson (2008) introduced the notion to a wider audience. Idioms capture the notion of lifting. The type of streams is a very special idiom in that a lifted operator inherits the properties of the base-level operator. The recent paper by the author (2010a) abstracts away from streams

[10] The minutes of the 2003 Meetings of the Algebra of Programming Research Group, 21st November, seem to suggest that the authors were aware of the technique, but were not sure of constraints on applicability, see `http://www.comlab.ox.ac.uk/research/pdt/ap/minutes/minutes2003.html#21nov`.

and answers the questions: "What base-level identities can be lifted through any idiom?" and "Which idioms satisfy every lifted base-level identity?" I previously discussed memo tables in a datatype-generic setting (2000b; 2010b). The observation that the memo table of an inductive datatype is given by a coinductive datatype is due to Altenkirch (2001).

This paper features two different proofs of Moessner's Theorem: the first is loosely modelled after a proof by Salié (1952), the second is based on a proof by Paasche (1952). A third alternative proof can be found in a recent paper of mine (2009b), which generalises the number-theoretic operators Σ and $\times$ to polymorphic, higher-order combinators — scans and convolutions. In fact, the paper proves Paasche's generalisation of Moessner's Theorem (1952).

The proof techniques presented in this paper can be readily generalised to other coinductive datatypes. Silva and Rutten adopt coinductive definition and proof principles to infinite binary trees (2010). Independently, the author adopted the unique fixed-point techniques to the same structures, applying the techniques to the problem of enumerating the rationals (Hinze, 2009a).

7 Conclusion

I hope you enjoyed the journey. Lazy functional programming has proven its worth: with a couple of one-liners we have built a small domain-specific language for manipulating infinite sequences. Suitably restricted, stream equations possess unique fixed points, a property that can be exploited to redevelop the theory of recurrences, finite calculus and generating functions.

Acknowledgements

Thanks are due to Nils Anders Danielsson, Jeremy Gibbons, Tom Harper, Daniel W. H. James, and the anonymous referees of ICFP 2008 for improving my English and for pointing out several typos. I owe a special debt of gratitude to the anonymous referees of this special issue, who went over and above the call of duty. Their suggestions greatly helped to improve the quality of the paper. Roland Backhouse provided intensive feedback — nineteen pages of TEX'ed comments. He pointed out that the approach is more general than I initially thought and advocated the consequential use of lifting and overloading. He also challenged me to derive implementation from specifications, in order to close the gap between the verbal descriptions and their incarnations in stream calculus. Thank you Roland.

References

Altenkirch, T. (2001) Representations of first order function types as terminal coalgebras. *Typed Lambda Calculi and Applications, TLCA 2001*. Lecture Notes in Computer Science 2044, pp. 62–78. Springer-Verlag.

Backhouse, R. (2001) *Galois Connections and Fixed Point Calculus*. Lecture Notes.

Backhouse, R. and Fokkinga, M. (2001) The associativity of equivalence and the Towers of Hanoi problem. *Information Processing Letters* **77**:71–76.

Backhouse, R. and Michaelis, D. (2005) *A Calculational Presentation of Impartial Two-Person Games*. Abstract presented at RelMiCS8.

Bartels, F. (2003) Generalised coinduction. *Math. Struct. in Comp. Science* **13**:321—-348.

Bird, R. (1998) *Introduction to Functional Programming using Haskell*. 2nd edn. Prentice Hall Europe.

Bird, R. and Meertens, L. (1998) Nested datatypes. Jeuring, J. (ed), *Fourth International Conference on Mathematics of Program Construction, MPC'98, Marstrand, Sweden*. Lecture Notes in Computer Science 1422, pp. 52–67. Springer-Verlag.

Bird, R. (1987) An introduction to the theory of lists. Broy, M. (ed), *Proceedings of the NATO Advanced Study Institute on Logic of programming and calculi of discrete design, Marktoberdorf, Germany* pp. 5–42. Springer-Verlag.

Coquand, T. (1994) Infinite objects in type theory. Barendregt, H. and Nipkow, T. (eds), *Types for Proofs and Programs, International Workshop TYPES'93, Nijmegen, The Netherlands, May 24-–28, 1993, Selected Papers*. Lecture Notes in Computer Science 806, pp. 62–78. Springer-Verlag.

Danvy, O. (1999) An extensional characterization of lambda-lifting and lambda-dropping. Middeldorp, A. and Sato, T. (eds), *4th Fuji International Symposium on Functional and Logic Programming (FLOPS'99), Tsukuba, Japan*. Lecture Notes in Computer Science 1722, pp. 241–250. Springer-Verlag.

Fokkinga, M. M. (1992) *Law and Order in Algorithmics*. PhD thesis, University of Twente.

Gibbons, J. and Hutton, G. (2005) Proof methods for corecursive programs. *Fundamenta Informaticae (Special Issue on Program Transformation)* **66**(4):353–366.

Gill, A. and Hutton, G. (2009) The worker/wrapper transformation. *J. Functional Programming* **19**(2):227–251.

Graham, R. L., Knuth, D. E. and Patashnik, O. (1994) *Concrete mathematics*. 2nd edn. Addison-Wesley Publishing Company.

Hinze, R. (2000a) Functional Pearl: Perfect trees and bit-reversal permutations. *Journal of Functional Programming* **10**(3):305–317.

Hinze, R. (2000b) Memo functions, polytypically! Jeuring, J. (ed), *Proceedings of the 2nd Workshop on Generic Programming, Ponte de Lima, Portugal* pp. 17–32. The proceedings appeared as a technical report of Universiteit Utrecht, UU-CS-2000-19.

Hinze, R. (2008) Functional Pearl: Streams and unique fixed points. Thiemann, P. (ed), *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming (ICFP '08)* pp. 189–200. ACM Press.

Hinze, R. (2009a) Functional Pearl: The Bird tree. *J. Functional Programming* **19**(5):491–508.

Hinze, R. (2009b) Scans and convolutions—a calculational proof of Moessner's theorem. Scholz, S.-B. (ed), *Post-proceedings of the 20th International Symposium on the Implementation and Application of Functional Languages (IFL 2008), University of Hertfordshire, UK, September 10–12, 2008*. Lecture Notes in Computer Science 5836. Springer-Verlag.

Hinze, R. (2010a) *Lifting Operators and Laws*. Available from `http://www.comlab.ox.ac.uk/ralf.hinze/Lifting.pdf`.

Hinze, R. (2010b) Type fusion. Pavlovic, D. and Johnson, M. (eds), *Thirteenth International Conference on Algebraic Methodology And Software Technology (AMAST2010)*. Lecture Notes in Computer Science. Springer-Verlag. to appear.

Hinze, R. and Löh, A. (2008) *Guide2lhs2TeX (for version 1.13)*. `http://people.cs.uu.nl/andres/lhs2tex/`.

Karczmarczuk, J. (1997) Generating power of lazy semantics. *Theoretical Computer Science (Special volume on computer algebra)* **187**(1-2):203–219.

Knuth, D. E. (1998) *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd edn. Addison-Wesley Publishing Company.

Mac Lane, S. (1998) *Categories for the Working Mathematician*. 2nd edn. Graduate Texts in Mathematics. Springer-Verlag.

McBride, C. and Paterson, R. (2008) Functional Pearl: Applicative programming with effects. *Journal of Functional Programming* **18**(1):1–13.

McIlroy, M. D. (1999) Power series, power serious. *J. Functional Programming* **3**(9):325–337.

McIlroy, M. D. (2001) The music of streams. *Information Processing Letters* **77**(2-4):189–195.

Milner, R. (1989) *Communication and Concurrency*. International Series in Computer Science. Prentice Hall International.

Moessner, A. (1951) Eine Bemerkung über die Potenzen der natürlichen Zahlen. *Aus den Sitzungsberichten der Bayerischen Akademie der Wissenschaften, Mathematisch-naturwissenschaftliche Klasse 1951 Nr. 3* March:29.

Niqui, M. and Rutten, J. (2010) Sampling, splitting and merging in coinductive stream calculus. Bolduc, C., Desharnais, J. and Ktari, B. (eds), *10th International Conference on Mathematics of Program Construction (MPC '10)*. Lecture Notes in Computer Science 6120, pp. 310–330. Springer-Verlag.

Paasche, I. (1952) Ein neuer Beweis des Moessnerschen Satzes. *Aus den Sitzungsberichten der Bayerischen Akademie der Wissenschaften, Mathematisch-naturwissenschaftliche Klasse 1952 Nr. 1* February:1–5.

Perron, O. (1951) Beweis des Moessnerschen Satzes. *Aus den Sitzungsberichten der Bayerischen Akademie der Wissenschaften, Mathematisch-naturwissenschaftliche Klasse 1951 Nr. 4* May:31–34.

Peyton Jones, S. (2003) *Haskell 98 Language and Libraries*. Cambridge University Press.

Rutten, J. (2003) Fundamental study: Behavioural differential equations: A coinductive calculus of streams, automata, and power series. *Theoretical Computer Science* **308**:1–53.

Rutten, J. (2005) A coinductive calculus of streams. *Math. Struct. in Comp. Science* **15**:93–147.

Röjemo, N. (1995) *Garbage collection, and memory efficiency, in lazy functional languages*. PhD thesis, Chalmers University of Technology.

Salié, H. (1952) Bemerkung zu einem Satz von Moessner. *Aus den Sitzungsberichten der Bayerischen Akademie der Wissenschaften, Mathematisch-naturwissenschaftliche Klasse 1952 Nr. 2* February:7–11.

Silva, A. and Rutten, J. (2010) A coinductive calculus of binary trees. *Information and Computation* **208**:578–593.

Sloane, N. J. A. (2009) *The On-Line Encyclopedia of Integer Sequences [online]*. Available at: `http://www.research.att.com/~njas/sequences/`.

Stewart, I. (1999) *The Magical Maze: Seeing the World Through Mathematical Eyes*. Wiley. ISBN 0471350656.